

MATERIAL AUTORAL

Criação de Agentes Autônomos

Disciplina 4 • Engenharia de Software com IA Aplicada

Autoria: Thiago Bussola

Edição didática completa • conteúdo integral • GitHub • práticas • diagramas • checkpoints

Como usar este material

Este material foi reconstruído a partir do conteúdo integral das aulas da disciplina. O texto-base e as afirmações conceituais seguem a transcrição fornecida; a edição acrescenta organização didática, recursos visuais, integração com o GitHub e atividades de revisão sem substituir o conteúdo apresentado pelo professor.

PRINCÍPIO DE FIDELIDADE

- Todo o conteúdo textual das aulas foi preservado, com correções editoriais de ortografia, pontuação, concordância, capitalização e repetições acidentais.
- Gráficos numéricos só aparecem quando os valores estão explicitamente presentes na transcrição; os demais recursos visuais apenas reorganizam conceitos já apresentados.
- Os links do GitHub apontam para o repositório oficial e para o snapshot correspondente a cada unidade.

REPOSITÓRIO OFICIAL DA DISCIPLINA

[Criação de Agentes Autônomos no GitHub](#)

O diretório está organizado em nove snapshots funcionais. Use o repositório junto com o texto: compreenda a decisão de engenharia, abra o snapshot e reproduza o comportamento.



COMO ESTUDAR CADA AULA

- Leia os objetivos e observe o fluxo visual da unidade antes de avançar.
- Estude o conteúdo integral, agora dividido em blocos e subtítulos para facilitar a leitura.
- Abra o snapshot correspondente no GitHub e localize os artefatos citados na aula.
- Responda ao checkpoint sem consultar o texto; volte apenas aos pontos em que ainda houver dúvida.
- Adapte o exercício ou a decisão de engenharia a um cenário diferente para consolidar o aprendizado.

Introdução da disciplina

Ao longo deste material, eu vou construir e evoluir um agente de software até chegar a uma arquitetura muito mais próxima de um sistema que poderia ser utilizado em um cenário real de operação. O objetivo não é apenas mostrar como chamar um modelo de linguagem ou como escrever prompts melhores. Eu quero trabalhar a engenharia necessária para transformar um modelo em um agente capaz de raciocinar, utilizar ferramentas, manter estado, lidar com memória, controlar custos, operar com resiliência e deixar uma trilha auditável das decisões que tomou.

O projeto que acompanha esse percurso é o OpsPilot, um copiloto de plantão voltado para cenários de suporte e resposta a incidentes. Ele começa simples, mas vai acumulando responsabilidades ao longo dos capítulos. Essa progressão é importante porque permite observar onde surgem os problemas reais. Quando uma aplicação passa a depender de ferramentas externas, memória, múltiplas estratégias e ações que alteram estado, não basta que ela responda corretamente em uma demonstração. Eu preciso pensar em arquitetura, contratos, persistência, segurança, contexto, observabilidade e governança.

Por isso, o material foi organizado como uma sequência de decisões de engenharia. Cada capítulo acrescenta uma peça e, ao mesmo tempo, reaproveita os fundamentos anteriores. Em vez de tratar cada conceito de agentes de forma isolada, eu mostro como eles se conectam dentro de uma mesma aplicação.

Context engineering e desenvolvimento orientado por especificação

Nos primeiros capítulos, eu começo pela relação entre o agente de código e o contexto que entregamos para ele. A discussão parte de context engineering, porque a qualidade de um agente não depende apenas do

prompt escrito naquele instante. Ela depende do conjunto de instruções, arquivos, restrições, permissões e referências que entram na janela de contexto.

A partir daí, eu trato o contexto como um recurso que precisa ser curado. Instruções permanentes do projeto, convenções de arquitetura, comandos permitidos, limites de segurança e regras de teste passam a ser registrados de forma persistente. O agente deixa de depender exclusivamente de uma conversa longa e passa a receber contratos estáveis sobre como o projeto deve ser desenvolvido.

Esse ponto se conecta diretamente ao Spec-Driven Development. Em vez de pedir uma feature inteira em uma única conversa e esperar que o agente descubra sozinho o que fazer, eu separo o trabalho em especificação, planejamento, tarefas e implementação. A spec registra intenção e critérios. O plano transforma essa intenção em arquitetura. As tasks quebram o trabalho em unidades menores. A implementação acontece somente depois dessas etapas.

Nos capítulos seguintes, esse fluxo é inicialmente montado de forma manual para que eu consiga entender cada artefato. Depois, ele é substituído pelo Spec Kit, que formaliza o mesmo processo e passa a acompanhar o restante do projeto. Essa disciplina reduz ambiguidade, melhora a revisão humana e cria uma memória de projeto que sobrevive à troca de chats, ferramentas e modelos.

Guardrails, permissões e revisão humana

Outra parte importante do início do material é a diferença entre instrução e permissão. Uma instrução pode orientar o agente a não executar determinada ação. Uma permissão aplicada pelo ambiente consegue impedir que ela aconteça.

Por isso, eu introduzo allow lists, deny lists, aprovação de comandos, hooks e verificações determinísticas. O agente pode sugerir ou tentar uma operação, mas o ambiente de execução continua responsável por controlar aquilo que realmente pode ser feito.

Também mantenho testes, type checking e hooks de Git como guardrails independentes do modelo. Isso é importante porque um agente pode esquecer uma instrução ou interpretar um pedido de forma diferente. Um mecanismo determinístico continua funcionando mesmo quando a parte probabilística falha.

Esse princípio de revisão humana aparece durante todo o material. O agente de código acelera o desenvolvimento, mas especificações, planos, implementações e erros continuam sendo revisados. Quando o modelo gera uma arquitetura inadequada, altera um contrato ou cria um detalhe diferente do esperado, eu volto aos artefatos, corrijo a direção e sigo a partir de evidências.

Os padrões de raciocínio do agente

Depois de estabelecer o processo de desenvolvimento, eu passo para o raciocínio do próprio OpsPilot. O agente de código está sendo utilizado para construir outro agente, e agora a pergunta deixa de ser somente como gerar software. Eu quero saber como o OpsPilot pensa e executa uma solicitação.

Três padrões aparecem como base: ReAct, Plan-and-Execute e Reflection. ReAct trabalha em ciclos de raciocínio, ação e observação. Ele é adequado para solicitações mais diretas e consegue reagir dinamicamente ao retorno das tools. Plan-and-Execute separa planejamento e execução, criando uma sequência explícita de passos antes de agir. Reflection acrescenta uma camada crítica sobre uma estratégia já existente, avaliando a resposta e podendo solicitar uma regeneração quando necessário.

Essas estratégias não são apresentadas como vencedoras universais. Eu comparo chamadas ao modelo, latência, custo, correção e estado real da aplicação. O benchmark mostra que uma resposta convincente não é suficiente se a tool alterou o estado errado. Também mostra que uma estratégia mais sofisticada pode custar muito mais para resolver um problema simples.

Ao longo dessa parte, trace e métricas começam a ganhar importância. Não quero apenas a resposta final. Quero saber quantas chamadas aconteceram, quanto tempo foi gasto e quais decisões levaram ao resultado.

Tools, API, persistência e fronteiras externas

Com as estratégias funcionando, o agente passa a operar sobre ferramentas de verdade. Eu crio um contrato comum para a store, começo com dados em memória e evoluo para SQLite. Incidentes, alertas, serviços e runbooks passam a sobreviver ao reinício da aplicação.

Essa persistência permite validar um ponto essencial: a operação executada pelo agente precisa alterar o mesmo estado que depois será consultado pela API, pelo terminal ou por outra interface. Não quero uma demonstração em que cada canal possui uma cópia separada do mundo.

A API HTTP também passa a funcionar como uma porta para o agente. O cliente envia uma mensagem em linguagem natural, escolhe ou deixa o sistema escolher uma estratégia e recebe resposta, trace e métricas. Os testes de integração utilizam estratégias determinísticas para não depender de rede em CI, enquanto arena e benchmark continuam responsáveis pelos testes com LLM real.

Quando entram provedores externos, eu introduzo timeout, retry e validação com Zod. Uma falha de rede não deve necessariamente derrubar toda a execução. Em muitos casos, ela pode virar uma observação que o agente lê e utiliza para decidir se tenta outro caminho ou informa a limitação ao plantonista.

Essa parte reforça um princípio que aparece repetidamente: toda fronteira precisa ser tratada como não confiável. Entrada HTTP, resposta do modelo, retorno de API externa e dados persistidos precisam de contratos e validação.

MCP e interoperabilidade

Depois que as tools já existem, aparece outro problema: como reutilizá-las em agentes diferentes sem criar uma integração específica para cada cliente.

A resposta adotada é o MCP, Model Context Protocol. O OpsPilot passa a expor operações como listar alertas, abrir incidentes e resolver incidentes por meio de um servidor MCP.

O ponto arquitetural mais importante é que o MCP não cria um novo backend. Ele funciona como outra porta para a mesma aplicação. A API HTTP, o MCP e as demais interfaces precisam reutilizar a mesma store, os mesmos schemas e as mesmas regras.

Ao longo dessa implementação, também aparece uma regra específica do transporte stdio: stdout pertence ao protocolo. Logs arbitrários nesse canal podem corromper a comunicação. Esse tipo de detalhe mostra como protocolos e infraestrutura influenciam o desenho do agente mesmo quando a lógica de negócio permanece a mesma.

Depois da configuração, o mesmo OpsPilot pode ser consumido por clientes diferentes, como Cursor e Copilot, sem duplicação das operações centrais.

Memória episódica e memória semântica

A partir da unidade de memória, eu enfrento um defeito típico de agentes construídos apenas sobre requests independentes: o modelo esquece tudo entre uma chamada e outra.

Primeiro eu implemento memória episódica. Cada conversa recebe um identificador, suas mensagens são persistidas no SQLite e uma janela das mensagens mais recentes volta para o prompt. Isso cria continuidade mesmo quando a aplicação é reiniciada.

Depois entra a memória semântica. Em vez de recuperar somente mensagens em ordem cronológica, eu quero guardar fatos e preferências do usuário e encontrá-los por significado. Para isso, utilizo embeddings locais e uma busca vetorial simples, adequada à escala do projeto.

A memória passa a ter deduplicação por similaridade, recall dos itens mais relevantes e um limiar mínimo para evitar injetar ruído no contexto. Um refletor de aprendizado também analisa os turnos e pode destilar fatos duráveis que valem a pena lembrar.

Essa arquitetura permite que uma preferência aprendida em um episódio anterior influencie uma nova conversa. O agente pode, por exemplo, lembrar que determinado usuário prefere incidentes críticos primeiro, mesmo quando a solicitação atual não repete exatamente as mesmas palavras.

Contexto como orçamento

Quando a memória, histórico, tools e instruções começam a crescer, surge outro problema: contexto demais.

Eu volto ao conceito de context rot, agora dentro do próprio OpsPilot. O contexto ativo passa a ser tratado como um orçamento formado por system prompt, memórias recuperadas, histórico recente, definições de tools, mensagem atual e, em estratégias interativas, observações acumuladas.

Primeiro eu meço. Existe uma contagem real de tokens informada pelo modelo depois da chamada e uma estimativa que pode ser utilizada antes do envio para decidir cortes. Depois eu introduzo pruning e sumarização.

Mensagens antigas que saem da janela recente não são simplesmente apagadas. Elas podem ser resumidas, preservando decisões, fatos, datas, preferências, incidentes e pendências. O resumo novo incorpora o anterior, criando uma compressão incremental da conversa.

Em seguida, entra o context stitching. Cada seção recebe teto, prioridade e regra de corte. Histórico pode remover mensagens mais antigas. Memórias podem eliminar itens de menor score. Resumos e system prompt recebem políticas próprias. O ContextBuilder centraliza essa montagem para que todas as estratégias utilizem a mesma política.

A partir daí, o contexto deixa de ser apenas uma janela grande e passa a ser um recurso administrado deliberadamente.

LangGraph, roteamento e fallback

Com várias estratégias disponíveis, eu retiro do cliente a responsabilidade de escolher manualmente qual delas deve ser usada.

Um grafo de produção em LangGraph passa a representar as etapas de execução. O roteador recebe a solicitação, produz uma decisão estruturada e encaminha o estado para ReAct, Plan-and-Execute ou Reflection.

O campo de estratégia continua podendo ser utilizado como override em testes e depuração, mas o comportamento padrão passa a ser automático.

O grafo também melhora o tracing e organização da resiliência. Cada nó representa uma etapa conhecida e as transições ficam explícitas.

Em paralelo, eu introduzo o conceito de fallback de modelo. O modelo principal pode receber tentativas limitadas e, se falhar, uma segunda opção assume. Caso nenhuma opção esteja disponível, a aplicação precisa responder de forma coerente com uma indisponibilidade temporária, em vez de esconder a falha.

Observabilidade e autonomia

À medida que o agente se torna mais autônomo, a observabilidade deixa de ser opcional.

Eu persisto requests e eventos de trace no SQLite, adiciono logs estruturados e correlaciono tudo por requestId. A partir desse identificador, consigo reconstruir a execução completa: rota escolhida, modelo, tokens, latência, nós percorridos, tools e resposta final.

Também introduzo uma matriz de autonomia. Algumas ações podem ser executadas livremente. Outras podem ser executadas, mas precisam ficar registradas. Algumas exigem aprovação humana. E determinadas capacidades simplesmente não devem existir, como apagar a própria trilha de auditoria ou vaziar dados do usuário.

Depois da auditoria individual, eu adiciono métricas agregadas para observar total de requests, erros, tokens, custo e latências por rota e modelo. Essa base permite comparar comportamento, investigar regressões e decidir onde o agente precisa ser otimizado.

War Room e publicação da interface

Depois de trabalhar por bastante tempo com terminal, API e MCP, eu construo uma interface web para o OpsPilot.

A War Room utiliza React e Vite e funciona como cliente da API existente. Ela apresenta chat, trace e espaço para ações que podem exigir aprovação.

Também aproveito essa etapa para voltar à memória do usuário e validar uma preferência dentro da própria interface. O pedido para organizar o plantão passa a considerar aquilo que o agente aprendeu anteriormente.

A War Room é publicada no GitHub Pages pela GitHub Actions. Como o Pages hospeda apenas o front-end estático, o backend continua local durante a demonstração e é exposto temporariamente por Cloudflare Tunnel.

Essa separação reforça a arquitetura do projeto. A interface é apenas mais um cliente. O raciocínio, a memória, o banco, o grafo e as tools continuam centralizados no backend.

Multiagentes

Na parte final, eu evoluo o OpsPilot para um sistema multiagente.

Uma resposta a incidente completa pode envolver investigação, planejamento e execução. Em vez de exigir que um único agente desempenhe todos esses papéis ao mesmo tempo, eu crio um time especializado.

O supervisor coordena, mas não executa. O analista trabalha em leitura e levanta fatos. O planejador transforma o diagnóstico em plano de mitigação. O executor aplica o plano utilizando as tools autorizadas.

A comunicação entre os papéis acontece por handoffs registrados no trace, enquanto o estado do grafo funciona como blackboard compartilhado. Também aparece o padrão de consenso, em que agentes independentes podem produzir pareceres e um juiz pode arbitrar uma decisão.

O interessante é que essa etapa reutiliza praticamente tudo que foi construído antes. Grafo, memória, tools, observabilidade, budgets, persistência e guardrails continuam válidos. Multiagente não significa começar outra arquitetura. Significa reorganizar responsabilidades sobre uma base que já possui controle e auditabilidade.

Ao chegar aos últimos capítulos, o projeto deixa de ser apenas um exemplo de integração com LLM. Ele passa a reunir os principais problemas que aparecem quando um agente precisa atuar como software: contexto, estado, memória, protocolos, fronteiras externas, custos, decisões probabilísticas, observabilidade, interfaces e coordenação entre múltiplos papéis.

Mapa da disciplina

Navegue por uma unidade ou aula. Cada unidade também possui um link direto para o snapshot correspondente no GitHub.

Unidade 1: Arquitetura de Agentes de Código (GitHub Copilot) [↗ ir para a unidade](#)

[Aula 1: O Agente de Código Por Dentro \(GitHub Copilot\)](#)

[Aula 2: Context Engineering e o Contrato de Permissões](#)

[Aula 3: Spec-Driven Development do Zero Parte 1](#)

[Aula 4: Spec-Driven Development do Zero Parte 2](#)

[Aula 5: Guardrails, Revisor e Delegação](#)

Unidade 2: Padrões de Raciocínio e Execução [↗ ir para a unidade](#)

[Aula 1: Raciocínio e Tomada de Decisão em Agentes - Os Três Padrões de Raciocínio](#)

[Aula 2: Configurando o Spec Kit](#)

[Aula 3: Criando a Estrutura Inicial do Projeto com Spec Kit](#)

[Aula 4: Definindo o Padrão ReAct Para o Agente](#)

[Aula 5: Definindo o Padrão Plan-and-Execute Para o Agente](#)

[Aula 6: Críticas e Benchmark com Padrão Reflection](#)

[Aula 7: Uma API que Também é um Agente](#)

Unidade 3: Function Calling e Tool Use [↗ ir para a unidade](#)

[Aula 1: Especificando a Integração com Banco de Dados](#)[Aula 2: Criando e Consultando Incidentes no Banco com Nosso Agente](#)[Aula 3: Validando Disponibilidade de Provedores Externos](#)[Aula 4: Disponibilizando Nosso Agente Via MCP](#)**Unidade 4: Memória e Reflexão em Agentes Autônomos** [↗ ir para a unidade](#)[Aula 1: Memória Episódica Para Nosso Agente](#)[Aula 2: O Vector Store: Spec, Implementação e a Memória Funcionando](#)**Unidade 5: Gerenciamento de Contextos** [↗ ir para a unidade](#)[Aula 1: O Contexto como Orçamento: Anatomia, Medição e Sumarização](#)[Aula 2: Context Stitching e os Paralelos com a Ferramenta](#)**Unidade 6: LangGraph e Workflows Complexos** [↗ ir para a unidade](#)[Aula 1: LangGraph e Fallback de modelo](#)**Unidade 7: Observabilidade e Limites de Autonomia** [↗ ir para a unidade](#)[Aula 1: Criando Estratégias de Observabilidade](#)**Unidade 8: Projeto Prático: OpsPilot Publicado** [↗ ir para a unidade](#)[Aula 1: Implementando a War Room e Modificando a Memória do Agente](#)[Aula 2: Publicando a War Room no GitHub Pages](#)**Unidade 9: Multi-Agent Systems** [↗ ir para a unidade](#)[Aula 1: Implementando Multiagentes](#)

Mapa do GitHub

No repositório, cada pasta representa um snapshot funcional ao final da unidade. Os snapshots do OpsPilot são cumulativos: a unidade seguinte mantém o que já foi construído e acrescenta novas capacidades.

Unidade	Conteúdo	GitHub
1	Arquitetura de Agentes de Código (GitHub Copilot)	Abrir snapshot
2	Padrões de Raciocínio e Execução	Abrir snapshot
3	Function Calling e Tool Use	Abrir snapshot
4	Memória e Reflexão em Agentes Autônomos	Abrir snapshot
5	Gerenciamento de Contextos	Abrir snapshot
6	LangGraph e Workflows Complexos	Abrir snapshot
7	Observabilidade e Limites de Autonomia	Abrir snapshot
8	Projeto Prático: OpsPilot Publicado	Abrir snapshot
9	Multi-Agent Systems	Abrir snapshot

Unidade 1

Arquitetura de Agentes de Código (GitHub Copilot)

Operação profissional de agentes de código: contexto, permissões, especificação, revisão e delegação com GitHub Copilot.

FLUXO VISUAL DA UNIDADE					
Contexto →	Constitution / Spec →	Plan →	Tasks →	Implement →	Revisão + Guardrails

GITHUB DA UNIDADE 1
Abrir no GitHub • Ler UNIDADE.md

Aula	Tema
1	O Agente de Código Por Dentro (GitHub Copilot)
2	Context Engineering e o Contrato de Permissões
3	Spec-Driven Development do Zero Parte 1
4	Spec-Driven Development do Zero Parte 2
5	Guardrails, Revisor e Delegação

Aula 1 — O Agente de Código Por Dentro (GitHub Copilot)

Unidade 1 • Arquitetura de Agentes de Código (GitHub Copilot)

OBJETIVOS DE APRENDIZAGEM

- Compreender agente, skill e autonomia.
- Compreender gitHub Copilot como harness.
- Compreender os três modos de interação.

GITHUB DA UNIDADE 1

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Agente, skill e autonomia
- GitHub Copilot como harness
- Os três modos de interação
- Criando o esqueleto do projeto no modo interativo
- Permissões, diffs e revisão humana
- Iteração a partir de um erro real

Conteúdo da aula

Neste módulo, eu mudo o ponto de vista em relação ao que construímos anteriormente. Nos módulos anteriores, trabalhamos com agentes criados do zero, estruturando componentes como LangChain, loops, padrão ReAct, ferramentas e memória. Agora, em vez de concentrar o esforço na implementação do mecanismo interno do agente, vamos aprender a operar uma agente de código real, utilizando o GitHub Copilot como ambiente de trabalho para desenvolver software com método profissional.

A proposta desta unidade é prática. Ao final, teremos um repositório que pode fazer parte do portfólio e uma API construída a partir de especificações, sem a necessidade de digitar manualmente toda a implementação. Ao longo desse processo, também vamos trabalhar com revisão especializada, guardrails que não dependem apenas do comportamento esperado do modelo e delegação de funcionalidades a partir de uma issue até a entrega por pull request.

A aplicação inicial será uma API de notas. Ela é simples de propósito. O objetivo não é tornar esse primeiro projeto complexo, mas criar um cenário controlado para que eu possa concentrar a atenção na arquitetura e na operação do agente. A partir da unidade seguinte, o projeto passa a crescer. Entraremos no OpsPilot, um copiloto de plantão voltado ao gerenciamento de incidentes de produção, alertas, runbooks e war rooms, evoluindo até chegar à publicação no fluxo de CI/CD.

Embora eu utilize a interface do GitHub Copilot durante as demonstrações, a ideia não está restrita a essa ferramenta. Ambientes como Claude Code ou Cursor também oferecem modos de agente e podem ser usados com adaptações. Algumas opções, comandos ou fluxos podem variar, mas a lógica principal permanece a mesma.

A própria interface do VS Code também passa a fazer parte desse processo. Eu posso abrir a área de agentes pelo ícone correspondente ou pela paleta de comandos, criar um agente customizado e abrir uma janela dedicada às sessões. Nessa janela, escolho o projeto em que a sessão será executada e qual agente ou integração vai operar. Como outras extensões podem ser reconhecidas como opções válidas, é importante conferir se estou realmente utilizando o agente do GitHub Copilot quando esse for o objetivo da aula.

O curso pode ser acompanhado com o Copilot no plano gratuito. Para estudantes que possuem e-mail educacional elegível, existe ainda a possibilidade de acesso ao GitHub Copilot Pro, o que libera recursos adicionais. Isso não altera o fundamento que estou ensinando. O importante é entender o funcionamento do modo agente e saber adaptar pequenas diferenças de interface ou disponibilidade de recursos ao ambiente utilizado.

Agente, skill e autonomia

A diferença fundamental entre um agente e uma skill está na autonomia. Eu considero o agente de código como uma entidade capaz de receber um objetivo, planejar como atingir esse objetivo e executar uma tarefa completa. A skill, por outro lado, representa uma capacidade específica utilizada durante essa execução.

Uma skill é atômica e especializada. Ela pode ser executada diretamente por uma inteligência artificial ou disponibilizada ao agente dentro do contexto em que ele trabalha. Para tornar essa distinção mais concreta, posso comparar o agente a um programador da equipe. Esse programador recebe uma issue do Jira ou de outro sistema de gerenciamento de tarefas, analisa o problema, pensa em uma solução, altera o software, executa testes e prepara uma entrega. As skills correspondem às ferramentas de trabalho deste desenvolvedor, como terminal, editor, Git e compilador.

As ferramentas são essenciais, mas não possuem autonomia. Um terminal não decide sozinho qual comando precisa executar. Um editor não identifica por conta própria quais arquivos precisam ser alterados. O agente é responsável por raciocinar sobre o objetivo e escolher como utilizar essas capacidades.

Essa ideia pode ser resumida em um ciclo que já conhecemos: o agente percebe um ambiente, raciocina sobre um objetivo, age por meio de ferramentas e ajusta seu comportamento com base no resultado obtido. No contexto de um agente de código, o ambiente deixa de ser abstrato e passa a ser principalmente o repositório de software.

GitHub Copilot como harness

É importante separar duas experiências diferentes do Copilot. A primeira é o autocomplete tradicional, que apresenta sugestões de continuação enquanto escrevemos código. Esse recurso não constitui um agente. Ele não executa um loop de decisão, não planeja uma sequência de ações e não opera autonomamente sobre o projeto.

A segunda experiência é o Copilot Chat em Agent Mode. É a esse modo que eu me refiro quando falo do agente ao longo da unidade. Nesse caso, o repositório representa o ambiente, e as ferramentas disponíveis correspondem às ações normais de um desenvolvedor: ler arquivos, editar conteúdo, executar comandos, pesquisar informações no código e utilizar o Git.

O objetivo do agente é produzir uma mudança no software. O feedback pode vir de um teste que passa, de uma compilação bem-sucedida ou de um erro que indica que a estratégia precisa ser ajustada.

Um modelo de linguagem isolado apenas produz texto. O que transforma esse modelo em um agente é o harness, ou seja, o programa responsável por executar o loop, disponibilizar ferramentas e aplicar regras. Quando construímos agentes anteriormente, implementamos esse mecanismo explicitamente. O loop que chamava o modelo, avaliava o resultado e decidia se deveria continuar era parte do harness. O Copilot oferece esse mecanismo pronto e integrado ao ambiente de desenvolvimento.

Isso muda o nosso trabalho. Em vez de construir o loop básico, passamos a configurar e operar um harness maduro, observando como ele interpreta instruções, solicita permissões, altera arquivos, executa comandos e reage aos resultados.

Os três modos de interação

No Copilot, eu separo três modos principais. O modo Ask é voltado a perguntas e respostas. Nele, eu posso pedir explicações sobre o projeto ou sobre determinado trecho, mas não espero que o agente faça alterações.

O modo Edit é utilizado para edições dirigidas em arquivos. Ele atua de forma mais restrita, concentrado na modificação solicitada.

O modo Agent é aquele em que temos o comportamento de agente propriamente dito. Nesse modo, o Copilot pode planejar, alterar múltiplos arquivos, executar comandos no terminal e iterar quando encontra erros. Se uma ação não produz o resultado esperado, ele pode analisar o retorno, formular outra abordagem e tentar novamente.

Dentro da experiência de agentes também podemos escolher estratégias de execução, como o modo interativo e o modo de planejamento. Essa diferença fica mais clara quando observamos o agente trabalhando em um projeto real.

Criando o esqueleto do projeto no modo interativo

A primeira atividade consiste em criar apenas o setup inicial do projeto de notas. Eu não entrego ainda uma especificação funcional da API. O objetivo é observar o agente configurando a base do repositório.

Eu descrevo que quero um projeto Node.js chamado notas-api, com Git inicializado, package.json preparado para módulos, dependências instaladas e TypeScript configurado. Entre as dependências, incluo Zod e ferramentas de desenvolvimento relacionadas a TypeScript, execução com TSX e tipos do Node.

Também especifico as principais opções do tsconfig. Defino uma versão moderna do ECMAScript como alvo, resolução de módulos compatível com o ambiente Node atual, verificação estrita de tipos, interoperabilidade de módulos, verificação de bibliotecas otimizada, consistência de maiúsculas e minúsculas em nomes de arquivo, diretório de saída e diretório raiz. Indico ainda que a pasta de código-fonte deve ser incluída na compilação.

No package.json, peço scripts para desenvolvimento, execução da interface de linha de comando, testes, checagem de tipos e limpeza. Também solicito um .gitignore que exclua dependências instaladas, arquivos de ambiente e variações desses arquivos, além do diretório de distribuição.

Eu faço questão de fornecer essas definições manualmente porque quero conseguir avaliar se o agente está seguindo as especificações. Ele poderia inferir uma configuração funcional sozinho, mas, neste momento, o objetivo é observar sua aderência ao que foi solicitado.

Durante a escrita do prompt, alguns nomes e opções podem conter erros de digitação. O agente é capaz de perceber inconsistências e sugerir correções para chegar a uma configuração funcional. Isso não elimina a necessidade de revisão. Ao contrário, cria um ponto importante de controle: eu preciso observar o que foi corrigido e decidir se a alteração está de acordo com a intenção original.

Permissões, diffs e revisão humana

No modo interativo, o agente começa a executar e pede autorização à medida que precisa realizar ações. Primeiro, ele pode solicitar permissão para criar diretórios e inicializar o projeto. Depois, pede autorização para entrar no diretório criado e instalar dependências.

Esse comportamento é importante porque torna o processo observável. Eu consigo verificar o que ele pretende executar antes de permitir a ação. Quando chega o momento de modificar arquivos, também posso analisar o diff. No package.json, por exemplo, verifico se o tipo de módulo, os scripts e as demais propriedades correspondem ao que solicitei.

A recomendação é revisar tanto os comandos quanto as edições. Automatizar parte do desenvolvimento não significa entregar completamente o controle ao agente. Desenvolver software com agentes de forma responsável exige acompanhar as mudanças, conferir as decisões e validar se o resultado faz sentido para o projeto.

Na demonstração, o agente cria o .gitignore e o tsconfig com as opções solicitadas e ainda corrige alguns erros presentes no prompt. Em seguida, inicializa o Git, cria a pasta de código-fonte e tenta executar a checagem de tipos.

Alteração a partir de um erro real

Nesse momento surge uma situação útil para entender o comportamento do agente. A pasta de código-fonte ainda não contém arquivos. Ao executar a checagem de tipos, o TypeScript não encontra entradas e retorna o erro TS18003.

O ponto relevante não é o erro em si, mas o que acontece depois dele. O agente identifica que a configuração solicitada entra em conflito com o fato de não existir nenhum arquivo em src. Em vez de simplesmente encerrar a tarefa, ele interpreta o feedback e propõe uma solução: criar um arquivo mínimo como placeholder para que o compilador tenha uma entrada válida.

Eu avalio essa proposta e, nesse contexto, ela é aceitável. Autorizo a criação do placeholder e o agente executa novamente a validação. Assim, conseguimos observar o ciclo completo: objetivo, ação, feedback, análise e nova ação. Essa é uma manifestação concreta do loop de um agente.

O resultado é o setup inicial funcionando de acordo com o que foi pedido. O projeto continua simples, mas já nos permite observar o planejamento local, execução de ferramentas, tratamento de erro e validação.

Modo de planejamento

Depois de trabalhar no modo interativo, eu volto ao modo de planejamento para comparar as abordagens. O novo pedido é simples: criar um README com informações úteis sobre o projeto e instruções para inicialização e execução.

No modo de planejamento, o agente não começa imediatamente a alterar o repositório. Primeiro, ele lê os arquivos existentes, como as configurações do projeto e o .gitignore, para compreender o estado atual. Em seguida, estrutura um plano.

O agente também pode fazer perguntas de contexto antes da implementação. Na demonstração, ele pergunta em qual idioma o README deve ser escrito. Eu escolho português. A partir disso, ele registra um planejamento com foco em onboarding e execução local, incluindo visão geral, pré-requisitos, instalação, scripts disponíveis, estrutura inicial e observações sobre o estado atual do projeto.

Esse plano pode ser revisado antes da implementação. Eu posso lê-lo, editar pontos específicos ou fornecer feedback para que o agente ajuste a estratégia. Só depois de considerar o planejamento adequado é que autorizo a implementação.

Essa é a principal diferença em relação ao fluxo interativo apresentado anteriormente. No modo interativo, o agente começa a executar a tarefa e solicita permissões conforme avança. No modo de planejamento, existe uma etapa explícita anterior à implementação, na qual a proposta é registrada e pode ser refinada.

Depois da aprovação, o agente cria o README com as informações planejadas, incluindo orientações de instalação, checagem de tipos, execução do projeto, uso da interface de linha de comando, testes, estrutura atual e observações pertinentes.

A escolha entre esses modos depende do tipo de tarefa e do nível de controle desejado. Para uma alteração simples, o fluxo interativo pode ser suficiente. Quando a tarefa envolve várias decisões, maior impacto ou necessidade de alinhamento prévio, o planejamento oferece uma oportunidade de revisar a estratégia antes que o repositório seja modificado.

A partir desse setup inicial, o projeto de notas continuará evoluindo nas próximas aulas. Ele serve como laboratório para aprendermos a configurar o agente, delegar trabalho, revisar decisões e aplicar controles antes de avançarmos para o projeto maior do módulo, o OpsPilot.

NA PRÁTICA

Abra o snapshot da Unidade 1 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Agente, skill e autonomia” no fluxo apresentado?
- Que decisão de engenharia está associada a “GitHub Copilot como harness” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Agente, skill e autonomia.
- GitHub Copilot como harness.
- Os três modos de interação.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Context Engineering e o Contrato de Permissões

Unidade 1 • Arquitetura de Agentes de Código (GitHub Copilot)

OBJETIVOS DE APRENDIZAGEM

- Compreender a memória permanente do projeto.
- Compreender instruções curtas e factuais.
- Registrar a stack do projeto.

GITHUB DA UNIDADE 1

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- A memória permanente do projeto
- Instruções curtas e factuais
- Registrando a stack do projeto
- Comandos que fazem parte do fluxo
- Estrutura e responsabilidades das camadas
- Convenções que o agente precisa respeitar

Conteúdo da aula

Nesta aula, eu quero trabalhar uma habilidade que considero decisiva para quem pretende extrair mais valor de um agente de desenvolvimento: o context engineering. Quando falamos em agentes, não basta apenas escrever uma boa instrução. É preciso decidir o que efetivamente ocupa a janela de contexto do modelo e, principalmente, quais informações precisam permanecer disponíveis durante o trabalho.

Existe uma diferença importante entre prompt engineering e context engineering. No prompt engineering, minha preocupação está concentrada em escrever uma boa instrução para uma determinada interação. No context engineering, a preocupação é mais ampla. Eu estou gerenciando todo o espaço de trabalho que o agente utiliza para raciocinar e tomar decisões. Isso inclui instruções permanentes, arquivos do projeto, resultados produzidos pelas ferramentas e também o histórico acumulado da conversa.

Essa distinção é importante porque o contexto precisa ser tratado como um orçamento. Cada informação inserida nele ocupa espaço e disputa a atenção do modelo. Quando eu mantenho informações irrelevantes, redundantes ou excessivamente extensas, começo a diluir essa atenção. Esse fenômeno é conhecido como context rot.

Por isso, a regra que quero seguir é simples: eu preciso curar o contexto, não apenas acumulá-lo. O objetivo não é colocar o máximo possível de informações à disposição do agente, mas selecionar aquilo que realmente orienta seu comportamento dentro do projeto.

A memória permanente do projeto

No GitHub Copilot, eu posso criar uma memória permanente para o projeto utilizando o arquivo copilot-instructions.md, localizado dentro da pasta.GitHub. Esse arquivo é carregado no contexto das conversas associadas ao repositório.

Na prática, ele funciona como uma memória permanente daquele projeto. Sempre que eu utilizar o agente dentro do repositório, essas instruções poderão orientar a forma como ele deve trabalhar.

Essa característica também significa que projetos diferentes podem possuir contextos diferentes. Um repositório pode trabalhar com determinadas convenções arquiteturais, enquanto outro utiliza tecnologias, estruturas e regras completamente diferentes. Em vez de depender de uma configuração genérica para todos os meus projetos, eu consigo especializar o comportamento do agente por repositório.

Também existem possibilidades de definir instruções no nível do workspace ou até instruções associadas ao usuário. Essas opções são úteis quando eu possuo determinadas preferências globais. Nesta aula, porém, meu interesse está no contexto específico do repositório, justamente porque quero que o agente compreenda com precisão onde está trabalhando e quais regras deve seguir naquele projeto.

Instruções curtas e factuais

Um ponto fundamental é que esse arquivo de instruções precisa ser curto e factual. Eu quero registrar comandos, convenções, fatos importantes sobre o projeto e comportamentos que devem ser evitados.

Não é o lugar adequado para procedimentos muito longos ou documentos extensos.

Se eu transformar esse arquivo em um grande manual, começo a consumir o orçamento de contexto permanentemente. Como essas informações entram nas conversas do repositório, cada frase desnecessária passa a competir com informações que podem ser mais importantes para a tarefa atual.

Por isso, eu prefiro instruções diretas.

Em vez de explicar em vários parágrafos por que determinada tecnologia foi escolhida, basta declarar qual tecnologia está sendo utilizada e qual restrição precisa ser respeitada. O agente precisa de direcionamento operacional, não de uma longa justificativa histórica.

Registrando a stack do projeto

Uma das primeiras informações que coloco nessa memória permanente é a stack utilizada pelo projeto.

No exemplo trabalhado nesta aula, eu informo que estamos utilizando Node.js 22 LTS e TypeScript, com módulos ES e modo estrito habilitado. Também deixo claro que o Zod deve ser utilizado nas fronteiras do sistema.

Isso significa que dados provenientes de fontes externas, como requisições HTTP ou entradas recebidas pela interface de linha de comando, precisam passar por validação.

Essa é uma informação arquitetural importante para o agente. Se ela está registrada no contexto permanente, eu não preciso repetir a cada solicitação que uma nova entrada HTTP deve ser validada.

Também específico a estratégia de testes. O projeto utiliza o sistema de testes do próprio Node executado por meio do TSX, sem introduzir um framework adicional.

Para HTTP, eu quero explicitamente a biblioteca nativa `node:http`. Registrar isso evita que o agente tome a iniciativa de adicionar Express ou outro framework simplesmente porque considera essa solução mais comum.

Esse é um exemplo claro de por que o contexto precisa conter fatos objetivos. Se existe uma decisão tecnológica deliberada no projeto, eu quero que o agente conheça essa decisão antes de começar a implementar.

Comandos que fazem parte do fluxo

Outro conjunto importante de informações são os comandos utilizados no projeto.

Eu quero que o agente conheça como iniciar o ambiente de desenvolvimento, como executar a aplicação localmente, como rodar os testes e como realizar a verificação de tipos.

No exemplo da aula, a API de desenvolvimento utiliza o ambiente local na porta 3000. Também existem comandos associados a teste e type checking.

Em vez de inventar comandos, eu posso orientar o agente a extrair essas informações diretamente do `package.json`. Com isso, a memória continua curta e alinhada àquilo que realmente existe no projeto.

A ideia novamente é reduzir ambiguidades. Se o agente conhece os comandos válidos, diminui a possibilidade de ele criar uma sequência alternativa ou assumir scripts que não fazem parte do repositório.

Estrutura e responsabilidades das camadas

Também utilizo as instruções permanentes para registrar a estrutura do projeto.

No domínio, quero manter os tipos e as definições relacionadas ao modelo da aplicação. Essa camada não deve realizar operações de entrada e saída. Com isso, preservo o domínio de dependências associadas a transporte, persistência ou interfaces externas.

A persistência fica atrás de uma abstração. Durante o exemplo, utilizamos uma implementação em memória, mas essa decisão pode ser alterada posteriormente para outro mecanismo de armazenamento. O ponto importante é que a camada responsável pelas regras de negócio não deve depender diretamente dos detalhes concretos de persistência.

A camada de service concentra as regras da aplicação. Já a parte HTTP representa uma das formas de entrada no sistema, assim como a interface de linha de comando.

Também mantenho uma área específica para especificações e outros artefatos relacionados ao processo de specification-driven development. Dessa forma, quando o agente precisar gerar documentação associada às especificações, ele sabe exatamente onde esses arquivos devem ser colocados.

É importante perceber que essa estrutura não é uma regra universal. Se eu estiver trabalhando em um projeto com outra organização, posso alterar essas definições. O objetivo do context engineering é justamente registrar as decisões do projeto em que estou trabalhando.

Se eu decidir mudar a organização das camadas, posso fazer isso. Entretanto, preciso estar consciente de que essa mudança pode gerar divergência em relação a outros exemplos ou comandos utilizados durante o desenvolvimento.

Convenções que o agente precisa respeitar

Além da estrutura, eu registro convenções arquiteturais.

Uma regra importante é que as camadas não devem ser puladas. Uma entrada recebida por HTTP ou pela CLI deve chegar ao service antes de acessar a persistência.

Essa convenção evita que uma camada de transporte comece a incorporar regras que deveriam estar concentradas na aplicação ou que passe a acessar diretamente mecanismos de armazenamento.

Também determino que toda entrada externa seja validada com Zod. A validação acontece justamente na fronteira, antes que os dados sejam utilizados pelas regras internas.

Erros pertencentes ao domínio também precisam ser tratados de maneira coerente quando atravessam essas fronteiras, mantendo a separação entre os conceitos internos da aplicação e a representação utilizada na interface externa.

Outra regra importante é que lógica nova nasce acompanhada de teste. Quando uma nova regra de negócio for implementada, quero que o agente considere o teste parte daquela implementação.

Além disso, tanto os testes quanto a verificação de tipos devem permanecer passando. Essa regra transforma qualidade e consistência do projeto em parte do contrato de trabalho do agente, em vez de algo que eu preciso lembrar manualmente em cada conversa.

Também registro uma restrição relacionada a segredos. O agente não deve incluir informações sensíveis do arquivo.env em commits e também não deve tratar esse arquivo como uma fonte que pode ser lida indiscriminadamente.

Um fluxo de trabalho explícito

As instruções também podem registrar o fluxo pelo qual quero conduzir mudanças no projeto.

No exemplo da aula, eu separei o processo em fases de especificação, planejamento das tarefas e implementação.

Entre essas fases existe revisão humana.

Isso é importante porque utilizar agentes não significa eliminar o desenvolvedor do fluxo. Eu quero aproveitar a capacidade do agente para produzir especificações, organizar tarefas e implementar alterações, mas continuo mantendo pontos explícitos em que uma pessoa avalia o que foi produzido antes de avançar.

Os artefatos gerados nesse processo devem permanecer na área destinada às especificações. Também quero que as instruções sejam compactas, aproximadamente quarenta linhas, sem transformar o arquivo em um manual de estilo completo.

Essa limitação reforça o princípio inicial: contexto permanente precisa ser curado.

Instruções especializadas por parte do repositório

As regras gerais do projeto não são a única forma de trabalhar o contexto.

Eu também posso criar instruções específicas que somente entram em cena quando o agente está trabalhando em determinada parte do repositório.

Essa é uma forma de especialização sob demanda.

Dentro da estrutura de instruções do GitHub, posso definir regras com um escopo associado a determinados arquivos. No exemplo, crio instruções direcionadas à camada de service.

Essas regras podem dizer, por exemplo, que funções puras devem ser utilizadas quando possível e que arquivos da camada de service nunca devem importar diretamente elementos da camada HTTP.

A grande vantagem é que eu não preciso colocar todas essas regras especializadas no contexto de qualquer tarefa.

Quando estiver alterando arquivos do service, as instruções relevantes para aquela camada são utilizadas. Quando estiver trabalhando em outra parte do projeto, elas não precisam disputar espaço desnecessariamente.

Esse mesmo princípio pode ser aplicado em projetos que possuem controllers, repositories, adapters ou qualquer outra divisão arquitetural. Eu consigo criar regras específicas para cada contexto e orientar o agente de maneira muito mais precisa.

Depois de criar ou alterar uma instrução, é importante observar se ela está sendo reconhecida pela conversa atual. Caso uma nova regra não seja capturada pelo chat já aberto, uma possibilidade é encerrar aquela janela e iniciar uma nova conversa, permitindo que o contexto seja carregado novamente.

Também vale testar essas regras deliberadamente. Eu posso solicitar algo que violaria uma convenção e verificar se o agente efetivamente respeita a restrição definida.

Instrução não é permissão

Até aqui, estamos trabalhando com instruções que influenciam o modelo. Agora eu quero separar esse conceito de outro mecanismo importante: as permissões.

Uma instrução orienta o comportamento do agente.

Uma permissão é aplicada pelo ambiente que executa as ações do agente.

Essa diferença é essencial.

Posso instruir o modelo dizendo que ele não deve realizar determinada operação. Isso influencia sua decisão. Entretanto, quando quero garantir que uma ação sensível não seja executada automaticamente, preciso utilizar os mecanismos de permissão do ambiente.

É aqui que surge o contrato de permissões.

No modo de agente, comandos executados no terminal podem exigir aprovação do desenvolvedor. Esse comportamento padrão cria uma barreira de segurança, porque uma ação produzida pelo agente não é necessariamente executada de forma automática.

Ao mesmo tempo, pedir aprovação para absolutamente tudo pode tornar o fluxo pouco produtivo.

Se eu preciso confirmar manualmente cada execução de teste, cada consulta simples ao Git ou cada comando rotineiro, começo a introduzir atrito em tarefas que já considero seguras.

Por isso, posso trabalhar com uma lista de comandos previamente permitidos e outra lista destinada a ações que não devem acontecer automaticamente.

Allow list e deny list

Na allow list, eu coloco operações que considero seguras para execução automática.

Posso permitir, por exemplo, a execução dos scripts de teste do projeto, determinados comandos do Node e operações do Git que apenas consultam ou preparam mudanças locais, como verificar o estado do repositório, analisar diferenças, adicionar arquivos ou realizar commits quando isso fizer parte do fluxo definido.

A intenção é eliminar pop-ups de aprovação para ações rotineiras que já foram classificadas previamente.

Por outro lado, algumas operações precisam continuar exigindo intervenção humana.

Comandos que removem arquivos recursivamente são um exemplo evidente. Operações executadas com privilégios elevados também merecem proteção.

Da mesma maneira, um git push forçado representa uma ação com impacto muito maior do que uma simples consulta ao estado do repositório.

Até mesmo uma tentativa de ler diretamente o conteúdo do .env pode ser configurada como uma operação que exige aprovação.

Nesse caso, o ambiente não está apenas contando com a boa vontade do modelo para seguir uma instrução. Existe uma regra operacional que controla se aquela ação poderá ser executada automaticamente.

Configurando a aprovação automática do terminal

No VS Code, essas regras podem ser configuradas nas opções relacionadas à aprovação automática do terminal.

Eu posso utilizar a interface de configurações ou editar diretamente o settings.json. No exemplo da aula, prefiro trabalhar diretamente nesse arquivo porque consigo visualizar e controlar as regras de maneira explícita.

Depois de configurar a allow list, posso testar solicitando novamente uma execução de testes.

Antes da configuração, o agente solicita minha autorização.

Depois que esse comando passa a fazer parte da lista permitida, os testes podem ser executados sem apresentar o mesmo pop-up.

Em seguida, faço o teste inverso.

Solicito uma operação destrutiva, como a remoção recursiva de node_modules. Como esse tipo de comando está protegido, o ambiente volta a solicitar minha autorização.

Eu simplesmente não aprovo a ação.

Esse teste demonstra a diferença entre os dois mecanismos que estamos construindo nesta aula. De um lado, utilizo context engineering para ensinar ao agente como o projeto funciona, quais convenções ele deve respeitar, que tecnologias fazem parte da stack e como as camadas estão organizadas. Do outro, utilizo permissões para determinar aquilo que o ambiente pode ou não executar automaticamente.

Ao combinar contexto curado, instruções gerais, regras especializadas por escopo e um contrato explícito de permissões, consigo deixar o agente mais focado e reduzir o consumo desnecessário de contexto. Ao mesmo tempo, elimino aprovações repetitivas para comandos rotineiros sem abrir mão do controle sobre operações sensíveis.

O resultado é um fluxo no qual eu não preciso repetir constantemente as convenções do projeto nem interromper o trabalho para aprovar ações já classificadas como seguras. As decisões permanentes ficam registradas no contexto correto, enquanto as restrições de execução permanecem protegidas pelo mecanismo de permissões.

NA PRÁTICA

Abra o snapshot da Unidade 1 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “A memória permanente do projeto” no fluxo apresentado?
- Que decisão de engenharia está associada a “Instruções curtas e factuais” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- A memória permanente do projeto.
- Instruções curtas e factuais.
- Registrando a stack do projeto.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Spec-Driven Development do Zero Parte 1

Unidade 1 • Arquitetura de Agentes de Código (GitHub Copilot)

OBJETIVOS DE APRENDIZAGEM

- Compreender as cinco fases do Spec-Driven Development.
- Compreender a função da especificação.
- Compreender construindo a Constitution.

GITHUB DA UNIDADE 1

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- As cinco fases do Spec-Driven Development
- A função da especificação
- Construindo a Constitution
- Validação nas fronteiras
- Erros como conceitos de domínio
- Testes fazem parte da tarefa

Conteúdo da aula

Quando eu começo a trabalhar com um agente mais potente, o principal gargalo deixa de ser simplesmente a capacidade dele de programar. A questão passa a ser outra: eu consegui dizer com precisão o que eu queria que fosse construído?

Quando eu faço um pedido solto em uma janela de chat, o agente pode produzir algo interessante e até gerar rapidamente um protótipo. O problema aparece quando eu tento aplicar esse mesmo comportamento em uma feature real, principalmente quando ela envolve várias regras, diferentes arquivos e decisões que precisam sobreviver a mais de uma interação.

Cada conversa possui uma janela de contexto. Se eu depender exclusivamente daquilo que foi discutido no chat, uma nova sessão pode começar sem conhecer todas as decisões tomadas anteriormente. É exatamente nesse ponto que entra o Spec-Driven Development, ou SDD.

A ideia é trabalhar com uma especificação escrita, versionada e revisada antes do código. A especificação deixa de ser apenas um documento produzido no início do desenvolvimento e esquecido posteriormente. Ela passa a dirigir o trabalho do agente.

As cinco fases do Spec-Driven Development

Eu organizo esse processo em cinco fases principais, sempre com revisão humana entre elas.

A primeira fase é a Constitution. Nela, eu registro os princípios que o agente deve respeitar durante todo o desenvolvimento daquela feature.

A segunda fase é a Specify. Aqui eu descrevo o que precisa ser feito e por que aquilo precisa existir.

A terceira fase é o Plan, responsável por decidir como a solução será construída.

A quarta fase é Tasks, na qual o planejamento é dividido em tarefas menores, ordenadas e testáveis.

A quinta fase é Implement, quando o agente começa efetivamente a modificar o projeto e implementar cada uma das tarefas definidas anteriormente.

Essa separação é importante porque evita que eu pule diretamente de uma ideia genérica para o código. Cada etapa produz um artefato que pode ser revisado antes de avançarmos.

Esses arquivos ficam persistidos em disco e podem fazer parte do Git. Isso significa que o contexto não fica preso à sessão atual do agente. Ele pode sobreviver a uma nova conversa e também pode participar do processo normal de revisão por pull request.

A função da especificação

Uma especificação pode utilizar critérios de aceite escritos com uma estrutura mais formal. Um dos formatos que podemos adotar é o EARS, que permite expressar comportamentos em uma estrutura semelhante a: quando determinado evento acontecer, o sistema deverá produzir determinada resposta.

O importante não é apenas seguir uma sintaxe específica, mas retirar ambiguidades daquilo que esperamos do software.

Esse modelo pode ser utilizado tanto em uma feature relativamente pequena quanto em mudanças mais complexas envolvendo vários arquivos.

Ao mesmo tempo, eu não considero a spec como uma fonte de verdade eterna.

Em um projeto maduro, o código continua representando o estado real do sistema. Os testes são responsáveis por impor diversos comportamentos que não podem ser quebrados, enquanto a especificação preserva e ancora a intenção que originou aquela mudança.

No projeto utilizado na aula, esses artefatos ficam organizados dentro da pasta de especificações.

Construindo a Constitution

O primeiro passo prático é criar a Constitution.

Quando peço ao agente para criar esse arquivo, eu deixo uma restrição explícita: ele deve criar o novo documento sem alterar arquivos existentes.

Faço isso porque, se a instrução não for clara, existe a possibilidade de o agente interpretar que aquelas regras deveriam ser acrescentadas às instruções permanentes do Copilot, já que parte do conteúdo pode ser semelhante.

A Constitution possui outra finalidade.

Ela existe para registrar os princípios não negociáveis que deverão ser relidos durante as próximas etapas do Spec-Driven Development.

Um dos primeiros princípios que defino está relacionado às camadas do projeto.

Quero dependências explícitas e um fluxo arquitetural em que a entrada HTTP chegue ao service e, a partir dele, à store. Também reforço que o domínio não realiza operações de entrada e saída.

Essa regra pode parecer repetitiva porque uma estrutura semelhante já foi definida nas instruções do projeto. A repetição, neste caso, é intencional.

Eu quero garantir que essa regra esteja presente no contrato que acompanha todo o processo da feature.

Validação nas fronteiras

Outro princípio da Constitution é a validação de dados externos.

Toda entrada recebida de fora do domínio deve ser validada com Zod antes de ser convertida para os objetos ou tipos utilizados internamente.

Isso vale para dados vindos de HTTP e também para informações recebidas por outras interfaces externas.

Dessa maneira, eu não permito que dados não validados entrem diretamente nas regras de negócio.

Erros como conceitos de domínio

Também estabeleço regras para tratamento de erros.

Falhas previsíveis devem ser representadas como erros de domínio. Essas falhas podem depois ser traduzidas nas bordas da aplicação para a representação adequada ao protocolo utilizado.

No caso de HTTP, por exemplo, um erro de domínio pode resultar na escolha de um status apropriado.

Essa separação é importante porque eu não quero que o domínio conheça detalhes de HTTP. A regra de negócio representa o problema em termos do próprio domínio. A camada externa decide como aquele erro será apresentado ao consumidor.

Testes fazem parte da tarefa

Outro princípio não negociável é que o teste faz parte da própria implementação.

Eu não trato o teste como uma atividade opcional que será feita em algum momento posterior. Se existe uma lógica nova, essa lógica precisa nascer acompanhada de teste.

Além disso, tanto a verificação de tipos quanto os testes precisam permanecer passando.

Essa é uma regra importante para agentes porque eu não quero que o objetivo seja simplesmente modificar arquivos até que a feature aparentemente funcione. Existe um conjunto de verificações que precisa continuar verde durante todo o processo.

Segurança não depende da confiança no modelo

Também registro princípios de segurança.

Segredos não devem ser adicionados ao repositório e operações destrutivas precisam passar pelos guardrails definidos no ambiente.

Essa distinção é fundamental.

Eu não quero depender da confiança de que o modelo decidirá corretamente todas as vezes. Se uma ação possui potencial destrutivo, ela precisa ser controlada por uma barreira externa.

Esse princípio se conecta diretamente ao contrato de permissões trabalhado anteriormente. Ações sensíveis devem passar por mecanismos como deny lists, aprovações explícitas e, quando aplicável, verificações realizadas antes de commits.

O agente pode receber instruções sobre o comportamento esperado, mas uma barreira de segurança importante deve existir fora da decisão do próprio modelo.

Spec antes de código

Outro princípio da Constitution estabelece a ordem do processo.

Mudanças relevantes devem passar primeiro pela especificação, depois pelo planejamento, pela decomposição em tarefas e somente então pela implementação.

Entre essas fases eu quero revisão humana.

A revisão não aparece como um detalhe adicional. Ela faz parte do método.

Ao utilizar um agente, eu não estou entregando todo o processo de desenvolvimento de uma vez e esperando apenas o resultado final. Quero acompanhar decisões intermediárias e poder corrigir o direcionamento antes que a mudança avance.

Mudanças pequenas e reversíveis

Também estabeleço que as tarefas precisam ser pequenas e reversíveis.

Idealmente, cada tarefa deve caber em um único commit.

Isso facilita a revisão e reduz o impacto de uma decisão errada.

Se eu permitir que o agente execute uma mudança gigantesca, envolvendo muitos arquivos e diversas decisões ao mesmo tempo, a revisão se torna mais difícil. Esse problema não existe apenas para quem está utilizando o agente. Em um ambiente de equipe, outros desenvolvedores também precisarão revisar essas alterações.

Uma tarefa menor é mais fácil de compreender, testar, revisar e, se necessário, reverter.

Reforçando a stack obrigatória

Ao final da Constitution, também reforço a stack que deve ser utilizada no projeto.

Estamos trabalhando com Node.js 22, TypeScript com módulos ES e modo estrito, Zod para validação, o sistema de testes do Node executado por TSX e a biblioteca HTTP nativa do Node.

Esses princípios propositalmente possuem pontos em comum com as instruções permanentes criadas anteriormente.

A diferença está no papel de cada arquivo.

As instructions funcionam como memória da conversa e do projeto. A Constitution funciona como um contrato que cada etapa do Spec-Driven Development precisa reler.

Por isso mantenho esses elementos separados.

Criando o comando de especificação

Depois da Constitution, começo a criar os arquivos de prompt que representarão as próximas fases.

O primeiro é o prompt de especificação.

Esse arquivo é criado na área de prompts do repositório e configurado para operar em modo agente. Sua responsabilidade é produzir a especificação de uma feature sem escrever código.

Essa restrição precisa estar explícita.

Durante a fase de especificação, eu quero discutir apenas o problema, os comportamentos e os requisitos. Decisões de implementação pertencem ao planejamento e não devem aparecer antecipadamente.

O prompt também precisa determinar que a Constitution seja lida e respeitada antes da geração da nova especificação.

Estrutura da spec

Eu defino uma estrutura padronizada para os arquivos produzidos pelo comando de especificação.

As specs recebem uma numeração sequencial e um identificador textual relacionado à feature. O agente deve encontrar o próximo número livre para manter a organização.

Dentro da especificação, quero inicialmente o contexto e a descrição do problema.

Depois, quero as user stories.

Utilizo a estrutura tradicional baseada em papel, objetivo e benefício. Dessa forma, eu identifico quem precisa da funcionalidade, o que essa pessoa precisa realizar e por que esse comportamento é útil.

Em seguida, entram os requisitos funcionais numerados.

Depois deles, quero critérios de aceite utilizando EARS.

Também reservo uma seção para aquilo que está fora do escopo. Essa parte é particularmente útil porque evita que o agente expanda a implementação para comportamentos que não foram solicitados.

Por último, mantenho uma seção de questões em aberto.

Se alguma informação importante estiver ambígua, o agente deve perguntar antes de finalizar a especificação.

Menor privilégio por fase

Existe ainda uma preocupação relacionada às ferramentas disponíveis em cada etapa.

Durante a especificação, por exemplo, não existe motivo para o agente editar os arquivos de código-fonte.

Essa ideia representa um princípio de menor privilégio por fase.

Uma etapa responsável apenas por especificar não deveria possuir liberdade operacional para implementar.

A sintaxe exata utilizada para limitar ferramentas pode variar conforme a versão utilizada, por isso essa configuração precisa ser confirmada na documentação correspondente ao ambiente.

O princípio, porém, continua o mesmo: cada fase deve receber apenas as capacidades necessárias para executar sua responsabilidade.

O prompt de planejamento

Depois da especificação, crio o prompt responsável pelo planejamento.

Essa etapa recebe uma spec já existente e transforma o que foi definido em uma proposta técnica.

O agente deve ler tanto a especificação quanto a Constitution.

O plano fica armazenado junto à feature correspondente.

Nesse documento, quero a arquitetura proposta, as camadas envolvidas e os arquivos que podem precisar ser criados ou alterados.

Também quero o modelo de dados, incluindo tipos e os esquemas de validação necessários.

Outro ponto são os contratos externos, como rotas HTTP ou comandos de linha de comando, sempre deixando claras as entradas e saídas esperadas.

O planejamento também precisa registrar decisões técnicas e trade-offs.

Além disso, quero uma estratégia de testes baseada no mecanismo de testes já adotado pelo projeto.

Nesse momento, o agente ainda não deve implementar nada.

Ele deve apontar riscos e situações que dependam de uma decisão humana.

O planejamento é, portanto, um degrau abaixo da especificação. A spec explica o que precisamos. O plano começa a decidir como aquilo será realizado.

Transformando o plano em tarefas

A próxima etapa é o prompt de tarefas.

Agora eu quero pegar o planejamento e dividi-lo em unidades pequenas, ordenadas e testáveis.

O agente deve ler a spec e o plano da feature.

Cada tarefa precisa ser pequena o suficiente para resultar em um único commit sempre que possível.

Também quero que cada tarefa deixe claro como poderá ser verificada.

Dependências entre tarefas precisam ser declaradas explicitamente.

A lista deve ser organizada de modo que o agente saiba o que precisa estar pronto antes que determinada tarefa possa começar.

Utilizo ainda uma marcação de progresso para diferenciar tarefas pendentes das já concluídas.

Novamente, essa fase não implementa nada.

Ela apenas transforma o planejamento técnico em uma sequência operacional.

O prompt de implementação

A última peça é o prompt de implementação.

Agora o agente pode trabalhar nas tarefas pendentes, mas deve fazer isso uma de cada vez.

Primeiro ele identifica a próxima tarefa ainda não realizada cujas dependências já tenham sido atendidas.

Depois implementa aquela tarefa seguindo o plano, as instruções do projeto e a Constitution.

Os testes relacionados precisam ser criados ou atualizados.

Ao final, o agente executa tanto os testes quanto a verificação de tipos.

A tarefa só pode ser marcada como concluída quando essas verificações estiverem passando.

Depois de terminar uma tarefa, o processo deve parar para revisão.

Esse comportamento é importante porque eu continuo querendo uma revisão humana entre unidades de trabalho.

Também deixo uma regra especial: o agente nunca deve desativar testes ou enfraquecer verificações de tipos simplesmente para conseguir concluir uma tarefa.

Esse comportamento pode acontecer em processos automáticos quando o objetivo é interpretado apenas como fazer o pipeline passar. Por isso, eu transformo essa restrição em regra explícita.

Se durante a implementação o agente perceber que a especificação está incorreta, ele também deve parar e avisar, em vez de tentar contornar silenciosamente o problema.

Executando a primeira especificação

Depois de criar esses prompts, eu encerro a conversa atual e abro uma nova janela para garantir que os novos comandos sejam reconhecidos.

A partir daí, utilizo o comando de especificação para criar uma primeira feature de gerenciamento de tarefas.

A intenção é permitir criar uma tarefa informando seu título, listar tarefas, concluir tarefas e removê-las, oferecendo acesso por HTTP e também pela interface de linha de comando.

Durante esse processo, o agente consulta diferentes elementos do projeto. Ele lê a Constitution, as instruções, o prompt de especificação, além de documentos como README e package.json.

Ao analisar o comportamento solicitado, ele identifica uma ambiguidade.

O que deve acontecer quando alguém tentar concluir uma tarefa que já está concluída?

Em vez de inventar silenciosamente uma decisão, o agente pede uma definição.

Para essa feature, escolho um comportamento idempotente: solicitar novamente a conclusão de uma tarefa já concluída deve continuar resultando em sucesso.

Com essa decisão tomada, a ambiguidade deixa de existir e a especificação pode ser finalizada.

A primeira spec de gerenciamento de tarefas registra o contexto do problema, incluindo o fato de que o projeto ainda possui apenas um esqueleto inicial e precisa de uma capacidade mínima de gerenciamento de tarefas acessível pelas duas interfaces.

Ela também contém user stories, requisitos funcionais e critérios de aceite.

Além disso, o agente identifica explicitamente comportamentos que não fazem parte do escopo atual, como edição de tarefas, priorização, reabertura, funcionalidades adicionais de identificação e persistência.

Como as ambiguidades relevantes foram resolvidas durante o processo, a especificação pode terminar sem questões em aberto, deixando preparado o artefato que será utilizado pela próxima fase do fluxo.

NA PRÁTICA

Abra o snapshot da Unidade 1 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “As cinco fases do Spec-Driven Development” no fluxo apresentado?
- Que decisão de engenharia está associada a “A função da especificação” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- As cinco fases do Spec-Driven Development.
- A função da especificação.
- Construindo a Constitution.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Spec-Driven Development do Zero Parte 2

Unidade 1 • Arquitetura de Agentes de Código (GitHub Copilot)

OBJETIVOS DE APRENDIZAGEM

- Planejar a spec 001.
- Compreender o conteúdo do plano técnico.
- Transformar o plano em tarefas.

GITHUB DA UNIDADE 1

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Planejando a spec 001
- O conteúdo do plano técnico
- Transformando o plano em tarefas
- Implementando uma tarefa por vez
- Domínio, store e services
- Validação e erros de domínio

Conteúdo da aula

Nesta aula, eu continuo exatamente do ponto em que encerramos a etapa anterior. Já temos a nossa spec 001 criada, relacionada ao gerenciamento de tarefas, e agora quero percorrer as próximas fases do fluxo de Spec-Driven Development que montamos no projeto.

A sequência que vamos seguir é a mesma que definimos anteriormente: primeiro o planejamento, depois a decomposição em tarefas e, por fim, a implementação. O mais importante aqui é observar que cada etapa consome os artefatos produzidos anteriormente. Dessa forma, eu não dependo exclusivamente da memória da conversa para preservar as decisões tomadas durante o desenvolvimento.

Planejando a spec 001

Como já temos a especificação de gerenciamento de tarefas, posso chamar diretamente a etapa de planejamento utilizando o identificador 001.

O agente então procura a spec correspondente e cria o plano técnico associado a ela. A intenção é transformar aquilo que foi descrito em termos de comportamento e requisitos em uma visão de como a feature será construída dentro do projeto.

Ao analisar o primeiro resultado, percebo um problema de organização. O conteúdo do planejamento está adequado, mas o nome do arquivo criado não preserva a numeração utilizada anteriormente na especificação.

Esse é exatamente o tipo de detalhe que eu quero corrigir no próprio framework que estamos criando.

Se a spec foi identificada como 001, quero que os artefatos seguintes mantenham essa mesma referência. Isso facilita localizar a especificação, o plano e posteriormente as tarefas associadas à mesma feature.

Por isso, solicito uma alteração no prompt de planejamento para que ele respeite a numeração da spec utilizada como origem.

Esse ajuste é importante porque estamos construindo não apenas uma feature, mas também um processo reutilizável. Quando percebo um comportamento que pode causar inconsistência nas próximas execuções, prefiro corrigir o prompt responsável por gerar aquele comportamento.

Depois do ajuste, a organização fica coerente e consigo associar visualmente a spec e o plano técnico da mesma funcionalidade.

O conteúdo do plano técnico

Ao revisar o plano gerado, verifico que o agente entendeu corretamente a arquitetura que definimos anteriormente.

Ele prevê a criação dos elementos do domínio responsáveis por representar as tarefas e seus estados. Também identifica a necessidade de criar os esquemas de validação com Zod, respeitando a regra de que as entradas externas devem ser validadas antes de entrar no domínio.

Na camada de service, ele planeja a implementação das regras relacionadas ao gerenciamento das tarefas.

Também existe uma abstração para a store, representando a interface de persistência utilizada pela aplicação. Nesse primeiro momento, a implementação concreta será mantida em memória.

O plano inclui ainda a exposição da funcionalidade por HTTP e também por linha de comando.

Além dos arquivos relacionados diretamente às regras da feature, o planejamento prevê alterações em pontos de integração do projeto e na documentação necessária para explicar a utilização da funcionalidade.

Esse é o tipo de resultado que eu espero da etapa de planejamento.

Ainda não quero código. Quero saber quais partes do sistema serão envolvidas, quais responsabilidades serão adicionadas e como essas peças se relacionam.

Transformando o plano em tarefas

Depois de revisar o planejamento, avanço para a etapa de tarefas utilizando novamente a referência 001.

O agente lê a especificação e o plano técnico e transforma o trabalho em pequenas unidades de execução.

Neste caso, ele gera cinco tarefas.

A primeira está relacionada à modelagem do domínio.

A segunda trata da implementação do service responsável pelo gerenciamento das tarefas.

As etapas seguintes avançam para os erros de domínio, persistência, interfaces externas, integração e validação final da feature.

A divisão em tarefas menores é uma das partes mais importantes desse processo.

Se eu simplesmente entregar o planejamento completo para o agente e pedir que implemente tudo, ele pode modificar muitos arquivos simultaneamente. Além de aumentar o volume de mudanças, isso dificulta a revisão.

Com tarefas pequenas, eu consigo acompanhar o desenvolvimento em incrementos.

Cada tarefa precisa ser suficientemente pequena para que eu consiga revisar o que aconteceu antes de permitir que o agente avance.

Implementando uma tarefa por vez

Com a lista pronta, começo a fase de implementação da spec 001.

A regra definida anteriormente continua valendo: o agente deve implementar uma tarefa de cada vez.

Na primeira execução, ele trabalha apenas na primeira tarefa.

Ao verificar o resultado, confirmo que ele não avançou automaticamente para a segunda. Isso é exatamente o comportamento desejado.

Mesmo que várias etapas estejam claramente descritas e o agente tenha capacidade para executar todo o conjunto, quero preservar a divisão do trabalho.

Essa separação permite revisar código, testes e decisões antes que novas camadas sejam adicionadas.

Depois de aprovar a primeira tarefa, autorizo o avanço para a segunda.

Na sequência, continuo acompanhando a implementação das próximas tarefas.

Domínio, store e services

Durante as primeiras tarefas, o agente cria os tipos associados ao domínio das tarefas.

Também começa a estruturar a abstração utilizada para persistência.

A store representa o contrato de acesso aos dados, enquanto a implementação em memória fornece o comportamento concreto utilizado neste projeto inicial.

Essa separação segue as regras arquiteturais que já havíamos colocado tanto nas instructions quanto na Constitution.

O service utiliza essa abstração para executar as operações de negócio.

Quando analiso o resultado, encontro as operações necessárias para criar, listar, concluir e remover tarefas.

Também observo que o agente respeitou uma instrução específica que havíamos definido anteriormente: utilizar funções puras sempre que possível na camada de service.

Esse é um ponto interessante porque demonstra a interação entre os diferentes níveis de contexto que construímos.

A especificação descreve a feature.

O planejamento descreve como ela será organizada.

As tarefas dividem a implementação.

A Constitution registra princípios que devem permanecer válidos durante o fluxo.

As instructions preservam convenções específicas do projeto e até regras direcionadas a determinadas partes do repositório.

Quando todos esses elementos trabalham juntos, eu preciso repetir menos informações durante a implementação.

Validação e erros de domínio

Outro ponto implementado está relacionado ao tratamento dos dados de entrada.

As informações recebidas pelo service passam pela validação definida para o projeto. Isso mantém a fronteira entre dados externos e objetos válidos para o domínio.

O agente também cria representações específicas para erros previsíveis.

Em vez de utilizar uma falha genérica para qualquer situação, as condições conhecidas pelo negócio são representadas como erros do domínio.

Depois, esses erros podem ser traduzidos na camada HTTP para respostas adequadas.

Essa tradução é importante porque o domínio não deve conhecer os detalhes do protocolo utilizado para expor a aplicação.

O erro existe como um conceito interno da regra de negócio. A borda da aplicação decide como representá-lo externamente.

Expondo a feature por HTTP

Uma das tarefas seguintes adiciona o acesso HTTP ao gerenciamento de tarefas.

Como deliberadamente escolhemos trabalhar com a biblioteca HTTP nativa do Node, não estamos utilizando um framework como Express para resolver o roteamento.

O agente precisa, portanto, implementar a identificação das rotas diretamente dentro da estrutura da aplicação.

Durante a revisão, observo que ele utiliza uma estratégia baseada em correspondência de caminhos para localizar determinadas operações.

Essa abordagem poderia ser construída de outras maneiras e talvez não seja a solução mais sofisticada para um projeto maior. Entretanto, dentro das restrições que definimos, ela atende ao objetivo.

O importante é perceber que a solução respeita a decisão arquitetural anterior de utilizar node:http.

As operações da feature são então disponibilizadas por métodos HTTP correspondentes às ações de criação, consulta, conclusão e remoção de tarefas.

A camada HTTP recebe a requisição, identifica a operação, valida os dados externos e encaminha a chamada para o service.

Dessa forma, ela não pula a camada de aplicação e não acessa diretamente a persistência.

Expondo o gerenciamento por linha de comando

Outra tarefa prevista no planejamento é disponibilizar a mesma capacidade através de uma interface de linha de comando.

O agente cria comandos para criar uma tarefa, listar as existentes, concluir uma tarefa e removê-la.

Essa parte demonstra uma característica importante da arquitetura que estamos construindo.

A regra de negócio não está acoplada ao HTTP.

Como o comportamento principal está no service, consigo ter diferentes interfaces chamando a mesma lógica.

HTTP e CLI são apenas formas diferentes de entrada.

Isso evita duplicar regras de negócio e mantém a responsabilidade das bordas limitada à tradução das entradas e saídas.

Durante os testes dessa interface, existe uma observação importante: como a persistência utilizada no momento é em memória, cada nova execução independente da CLI começa com um novo estado.

Isso é esperado para esta etapa do projeto e faz parte das limitações atuais da implementação.

Validando a feature de ponta a ponta

A última tarefa é dedicada à integração, documentação e validação completa da feature.

Nesse momento, o agente já possui domínio, service, persistência em memória, HTTP e CLI.

Agora ele precisa verificar se todas essas partes funcionam em conjunto.

Durante esse processo, o número de testes cresce progressivamente.

O agente executa as verificações definidas para o projeto e também inicia a aplicação para testar os endpoints.

Em determinado momento, ele precisa executar uma chamada HTTP externa através do terminal. Como esse comando ainda não estava previamente autorizado, o ambiente solicita minha permissão.

Eu aprovo a execução.

A chamada é realizada e a resposta esperada é obtida.

Depois ele testa outra rota e finaliza o processo utilizado para executar a aplicação.

Ao final dessa etapa, a feature é considerada completa e a suíte possui dezenas de testes cobrindo o comportamento criado.

O mais relevante aqui não é apenas a quantidade de testes, mas o fato de que eles foram adicionados ao longo das tarefas, em vez de serem tratados como uma atividade isolada no final.

Construímos nosso próprio framework de SDD

Tudo que fizemos até esse ponto forma, na prática, um framework próprio de Spec-Driven Development.

Criamos uma Constitution e quatro comandos principais para especificar, planejar, decompor em tarefas e implementar.

Poderíamos criar ainda mais etapas se o nosso projeto precisasse.

Essa é uma das principais ideias que quero demonstrar.

Eu não sou obrigado a utilizar exatamente um framework externo para trabalhar com especificações e agentes. Posso construir um processo que represente as necessidades da minha equipe e do meu projeto.

Ao mesmo tempo, existem soluções prontas no mercado que seguem conceitos muito semelhantes.

GitHub Spec Kit, OpenSpec e outros frameworks

Um dos exemplos é o GitHub Spec Kit, que oferece suporte ao fluxo de desenvolvimento orientado a especificações e integração com agentes como o Copilot.

Ele possui etapas equivalentes às que construímos manualmente, incluindo Constitution, Specification, Planning, Tasks e Implementation.

Isso significa que, quando passarmos a utilizar esse tipo de ferramenta, as peças não serão completamente novas.

Eu já construí manualmente o equivalente conceitual de cada uma delas.

A partir da unidade seguinte, a proposta é utilizar o Spec Kit em vez de continuar evoluindo exclusivamente o pequeno framework que criamos aqui.

Também existem outras alternativas.

O OpenSpec possui uma abordagem mais leve e pode ser utilizado em cenários com diferentes agentes.

Outro exemplo é o BMAD Method, que segue uma proposta mais extensa e procura simular diferentes funções de uma equipe dentro do processo.

O objetivo desta aula não é escolher definitivamente um desses frameworks, mas mostrar que o fluxo não depende de uma ferramenta específica.

Antes de utilizar um framework pronto, eu quero entender quais problemas ele resolve e por que cada fase existe.

Contexto estruturado e consumo de tokens

Outra consequência interessante desse processo aparece no consumo de contexto.

Quando eu especifico de forma adequada o que o agente precisa fazer, ele recebe informações mais direcionadas.

Em vez de tentar descobrir toda a intenção a partir de uma conversa longa e pouco estruturada, ele pode consultar artefatos específicos para cada etapa.

Isso pode reduzir o desperdício de tokens em determinadas tarefas.

O objetivo não é simplesmente consumir menos por consumir menos. A vantagem está em utilizar o contexto com mais eficiência.

A especificação contém o que precisa ser construído.

O plano contém as decisões de arquitetura.

As tarefas dizem qual é o próximo incremento.

As instruções permanentes contêm regras do projeto.

O agente pode então concentrar sua atenção no conjunto de informações relevante para a execução atual.

Permissão não valida o resultado do trabalho

Depois de finalizar a feature, começo a entrar em outro ponto importante: guardrails, revisão e delegação.

Até aqui, já trabalhamos com permissões.

A permissão responde a uma pergunta específica: uma determinada ação pode ser executada?

Uma deny list, por exemplo, pode impedir que determinados comandos perigosos sejam executados automaticamente.

Mas existe outra pergunta.

Como eu garanto que o resultado produzido pelo agente continua respeitando as regras do projeto?

Posso escrever uma instruction dizendo que o type check deve sempre passar.

O problema é que essa instruction continua sendo uma orientação para o modelo.

Ela pode influenciar o comportamento, mas não é a mesma coisa que uma garantia determinística.

É por isso que eu quero acrescentar outras camadas de guardrail.

Guardrails determinísticos

Um guardrail determinístico é executado independentemente da decisão do modelo.

Não importa se o agente se lembrou da regra, se a conversa está longa ou se a instrução perdeu relevância dentro da janela de contexto.

O código responsável pelo guardrail será executado.

Começamos com uma primeira camada, a deny list, responsável por controlar comandos perigosos antes da execução.

Agora adicionamos uma segunda camada: um pre-commit.

Esse pre-commit deve executar duas verificações fundamentais antes que uma alteração seja registrada no Git.

A primeira é o type check.

A segunda é a suíte de testes.

Se uma dessas verificações falhar, o commit não deve prosseguir normalmente.

Essa proteção é diferente de simplesmente pedir ao agente que execute os testes.

O agente pode receber a instrução de rodar os testes, mas o hook do Git é uma condição operacional executada durante o processo de commit.

Também poderíamos adicionar uma terceira camada em CI, executando essas verificações novamente em um ambiente centralizado.

O conceito é trabalhar com várias camadas independentes de proteção.

Criando o pre-commit

Solicito ao agente a criação do hook responsável pelo pre-commit.

Ele precisa executar a verificação de tipos e, em seguida, os testes.

Além disso, o arquivo precisa estar configurado para poder ser executado.

Durante a criação, o próprio agente identifica essa necessidade e prepara a permissão adequada para o script.

Depois disso, quero provar que o mecanismo funciona sem depender do agente.

Essa é justamente a característica determinística que estou buscando.

O hook deve funcionar no terminal normal porque é parte do repositório e do processo do Git, não apenas uma instrução existente em uma conversa.

Provocando uma falha proposital

Para verificar o guardrail, solicito deliberadamente que o agente introduza uma incompatibilidade de tipos em uma parte do projeto.

A intenção não é corrigir o código, mas gerar uma situação em que o type check necessariamente falhe.

Depois tento passar pelo processo de commit.

Como o repositório ainda não possuía um histórico anterior de commits naquela demonstração, alguns detalhes iniciais do Git precisam ser configurados.

Com a quebra introduzida, executo o pre-commit e confirmo que a verificação de tipos falha.

É exatamente o comportamento que eu queria demonstrar.

Não preciso confiar que o modelo se lembrará de conferir os tipos.

O mecanismo obrigatório identifica que existe uma inconsistência.

Depois dessa validação, solicito que o agente corrija o código.

A incompatibilidade é removida e as verificações voltam a passar.

Esse fluxo começa a formar uma combinação mais segura entre especificações, revisão humana, permissões e validações determinísticas.

A spec preserva a intenção da feature.

O planejamento organiza a solução.

As tarefas limitam o tamanho das mudanças.

A implementação acontece de maneira incremental.

A deny list controla ações perigosas.

O pre-commit verifica automaticamente tipos e testes antes de aceitar determinadas mudanças no histórico do projeto.

NA PRÁTICA

Abra o snapshot da Unidade 1 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Planejando a spec 001” no fluxo apresentado?
- Que decisão de engenharia está associada a “O conteúdo do plano técnico” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Planejando a spec 001.
- O conteúdo do plano técnico.
- Transformando o plano em tarefas.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 5 — Guardrails, Revisor e Delegação

Unidade 1 • Arquitetura de Agentes de Código (GitHub Copilot)

OBJETIVOS DE APRENDIZAGEM

- Compreender a validação da aplicação pelo HTTP.
- Compreender a limitação da CLI com persistência em memória.
- Criar a spec 002.

GITHUB DA UNIDADE 1

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Validação da aplicação pelo HTTP
- A limitação da CLI com persistência em memória
- Criando a spec 002
- User stories, requisitos e critérios de aceite
- Planejando a persistência em arquivo
- Decompondo a feature em tarefas

Conteúdo da aula

Nesta aula, eu quero fechar essa primeira aplicação olhando para o que foi construído e, principalmente, observando onde o fluxo ainda pode ser melhorado. Até aqui, nós estruturamos uma aplicação simples de gerenciamento de tarefas, criamos um processo de Spec-Driven Development e adicionamos mecanismos de proteção para que o agente não dependa apenas de instruções textuais. Agora eu quero executar a aplicação, identificar uma limitação real de usabilidade e usar o próprio processo que criamos para evoluir a solução.

Validação da aplicação pelo HTTP

Eu começo saindo da janela do agente e voltando para o VS Code. No terminal, já estou dentro da pasta do projeto da API. A primeira verificação é subir o servidor de desenvolvimento. Quando executo o script de desenvolvimento, a aplicação inicia na porta 3000.

Com o servidor em execução, abro um segundo terminal para testar a API. A ideia é criar uma tarefa pela rota HTTP que já foi implementada. Faço uma requisição para a rota de tarefas, informo que o corpo está em JSON e envio um título para a nova tarefa.

A resposta indica que a tarefa foi criada corretamente. Em seguida, faço uma nova chamada para listar as tarefas e confirmo que aquela tarefa está disponível enquanto o servidor continua em execução.

Essa validação é importante porque mostra que o fluxo HTTP está funcionando como planejado. A entrada chega pela camada HTTP, passa pela validação definida para o projeto, utiliza o service e chega à store em memória.

Nesse momento, porém, eu quero comparar esse comportamento com a interface de linha de comando.

A limitação da CLI com persistência em memória

Eu encerro o servidor e passo a testar a CLI. A aplicação já possui comandos para criar, listar, concluir e remover tarefas.

Quando executo o comando de criação pela linha de comando, a tarefa é criada e a aplicação informa que a operação ocorreu corretamente. O problema aparece logo em seguida.

Se eu executar novamente a CLI utilizando o comando de listagem, a tarefa criada anteriormente não aparece.

Isso acontece porque cada execução da linha de comando inicia um novo processo. Como a implementação atual da store mantém os dados apenas em memória, o estado existe somente durante a execução daquele processo. Assim que o comando termina, aquela memória deixa de existir.

No HTTP, a situação parece um pouco diferente porque o servidor permanece ativo. Enquanto o processo HTTP está de pé, a store em memória continua existindo e as tarefas podem ser criadas e consultadas. Quando o servidor for encerrado e iniciado novamente, entretanto, esse estado também será perdido.

Durante esse teste eu encontro ainda uma saída adicional no terminal, um texto que havia sido deixado por uma impressão simples adicionada entre as aulas. Eu localizo essa instrução e a removo para que ela não interfira na observação do comportamento real da CLI.

Com isso, fica clara a limitação que quero resolver: a linha de comando precisa manter as tarefas entre diferentes execuções.

Criando a spec 002

Em vez de simplesmente pedir ao agente para alterar o código, eu volto ao processo de Spec-Driven Development que construímos.

A primeira etapa é novamente especificar.

Eu quero criar a spec 002, desta vez com o objetivo de ajustar a linha de comando para ter persistência. A solução que proponho para essa etapa é simples: utilizar um arquivo JSON para salvar as tarefas entre diferentes sessões da CLI.

O ponto mais importante aqui é o processo. Eu não digo apenas para o agente adicionar um arquivo e começar a modificar a aplicação. Primeiro peço que ele produza a especificação.

Ao iniciar a tarefa, observo os arquivos consultados pelo agente. Ele lê as especificações que já existem, consulta as instructions do projeto e também a Constitution.

Isso mostra que o pequeno framework que construímos está funcionando como eu esperava. Antes de produzir uma nova especificação, o agente está recuperando as regras e os princípios que precisam orientar aquela feature.

A especificação identifica corretamente o problema atual: a linha de comando sempre começa com o estado vazio porque a persistência em memória existe apenas durante a vida do processo.

Consequentemente, uma tarefa criada em uma execução não fica disponível na execução seguinte.

A feature precisa introduzir uma persistência baseada em arquivo JSON para a CLI.

User stories, requisitos e critérios de aceite

A spec criada pelo agente organiza o comportamento utilizando a estrutura que definimos anteriormente.

Ela cria user stories para representar as pessoas que utilizarão a funcionalidade e para descrever o benefício esperado.

Também registra requisitos funcionais e critérios de aceite.

Eu não preciso tratar essa documentação como uma formalidade. Ela existe para permitir que eu revise o que será construído antes que qualquer decisão de implementação seja executada.

Nesse caso, o agente também identifica uma questão em aberto: qual deve ser o caminho padrão do arquivo utilizado para persistência?

Essa é uma pergunta relevante.

O local escolhido para armazenar o arquivo influencia a experiência de uso e pode afetar testes, execução local e organização do projeto. Em vez de ignorar a dúvida, a spec deixa essa decisão visível.

Para a demonstração, eu decido deixar que o próprio processo sugira uma solução posteriormente, mas o importante é observar que a ambiguidade apareceu antes da implementação.

É exatamente esse comportamento que eu quero de um fluxo orientado por especificação.

Planejando a persistência em arquivo

Depois da spec, avanço para o planejamento da feature 002.

O agente lê novamente os artefatos necessários e produz o plano técnico.

Na revisão do planejamento, observo que ele pretende criar e alterar os arquivos relacionados à persistência. A nova implementação fica associada à área de store, respeitando a separação arquitetural que já havíamos definido.

O plano também apresenta o modelo de dados e os esquemas de validação que serão necessários.

Essa parte é importante porque um arquivo JSON continua sendo uma fronteira externa.

O fato de os dados terem sido gravados anteriormente pela própria aplicação não significa que eu deva confiar automaticamente em seu conteúdo. O arquivo pode ser alterado, corrompido ou conter uma estrutura incompatível com a versão esperada.

Por isso, a validação continua fazendo sentido quando os dados são carregados de volta para a aplicação.

Se eu não concordar com alguma decisão do planejamento, esse é o momento para alterá-la. Ainda estamos antes da implementação, justamente para evitar descobrir tarde demais que a solução seguiu uma direção que não queremos.

Decompondo a feature em tarefas

Depois do planejamento, executo a etapa de tarefas para a spec 002.

O agente divide novamente o trabalho em cinco unidades menores.

Entre elas estão a modelagem necessária, a implementação da persistência, a integração da nova store com a linha de comando, os testes do comportamento de persistência e, por fim, a documentação e a validação da feature de ponta a ponta.

Essa divisão repete a estratégia utilizada na primeira feature.

Eu quero tarefas pequenas porque elas são mais fáceis de entender e revisar. O agente não precisa resolver todo o problema em um único movimento.

Também consigo observar em que ponto uma falha foi introduzida e corrigir a direção sem revisar uma quantidade excessiva de mudanças.

Acompanhamento humano durante a implementação

Quando avançamos para a implementação, acontece algo que demonstra por que a revisão humana continua importante.

Durante o processo, o agente produz um comportamento inesperado relacionado a SQL e chega a encontrar um erro envolvendo chave estrangeira.

Isso não fazia parte da solução que havíamos especificado, já que a intenção era utilizar persistência em JSON.

Nesse momento, eu acompanho o que ele está fazendo em vez de assumir que toda ação automática está correta.

Na sequência, o próprio fluxo retorna ao objetivo esperado: implementar uma store em JSON e conectá-la à linha de comando, incluindo o tratamento explícito das situações relacionadas à persistência.

Esse episódio é útil porque reforça uma ideia importante. Ter uma spec, um plano e instruções melhores reduz muito o espaço para desvios, mas não elimina a necessidade de revisão.

O agente continua executando trabalho que precisa ser observado.

A função do processo estruturado é tornar esses desvios mais fáceis de detectar e corrigir.

Allow list e deny list aparecendo na prática

Durante os testes da implementação, o agente precisa executar novos comandos.

Em um dos casos, ele utiliza um comando que não estava presente na allow list configurada anteriormente. Por esse motivo, a execução exige minha autorização.

Isso mostra o contrato de permissões funcionando na prática.

O agente pode entender perfeitamente que precisa executar um determinado teste, mas a decisão de permitir a ação não depende apenas de sua intenção.

Se o comando não foi previamente classificado como seguro para execução automática, eu continuo participando da decisão.

Em outro momento, o processo precisa remover um arquivo temporário criado durante os testes.

Como essa ação se enquadra nas proteções que definimos, ela também exige aprovação.

Esse comportamento é exatamente o que eu quero.

As operações rotineiras e já conhecidas podem ser liberadas para reduzir atrito. Ações não previstas ou potencialmente destrutivas continuam chegando até mim.

Caso um novo comando de teste passe a fazer parte frequente do fluxo e eu considere que ele é seguro, posso adicioná-lo posteriormente à allow list.

A lista não precisa ser estática. Ela pode evoluir conforme o projeto e o processo de desenvolvimento amadurecem.

Validação da persistência da CLI

Depois da implementação, o agente informa que a feature está pronta e que a suíte agora possui 42 testes passando.

Eu não quero parar apenas nessa informação. Volto ao terminal para validar o comportamento que motivou a spec.

Primeiro executo a CLI para criar uma nova tarefa.

A aplicação confirma a criação.

Depois inicio uma nova execução da linha de comando, desta vez utilizando a opção de listagem.

Agora a tarefa continua disponível.

Essa diferença confirma que o estado deixou de existir apenas na memória daquele processo. A CLI passa a utilizar o arquivo de persistência criado para a feature.

O ponto central não é o fato de um JSON ser uma solução sofisticada de banco de dados. Ele não é.

Para esse projeto, o objetivo é demonstrar o processo de evolução da aplicação e permitir que a persistência sobreviva entre sessões independentes da linha de comando.

A solução é deliberadamente simples e suficiente para o exercício atual.

O próximo exercício: uma fonte compartilhada entre CLI e HTTP

Mesmo com a melhoria, ainda existe uma diferença entre as duas interfaces.

A CLI agora utiliza o arquivo JSON.

O servidor HTTP, por enquanto, continua trabalhando com sua store em memória.

Enquanto o servidor está ativo, ele consegue criar e listar tarefas normalmente. Entretanto, quando o processo termina, os dados desaparecem.

Além disso, aquilo que eu crio pela CLI não é automaticamente compartilhado com o servidor HTTP, porque as duas interfaces ainda não estão utilizando a mesma fonte de persistência.

Por isso, proponho uma nova atividade.

A ideia é criar uma spec 003 para fazer com que a linha de comando e o servidor HTTP utilizem a mesma base de dados.

No cenário mais simples, ambos podem apontar para o mesmo arquivo JSON.

Se eu tiver optado por utilizar um banco de dados durante os exercícios, posso aplicar exatamente o mesmo princípio fazendo com que as duas interfaces acessem a mesma base.

Esse exercício força a revisão de todo o fluxo que construímos.

Eu preciso especificar o comportamento, planejar as alterações, dividir o trabalho em tarefas e implementar de maneira incremental.

Também consigo verificar se a Constitution continua sendo respeitada, se as instructions estão sendo carregadas corretamente e se os agentes estão obedecendo às decisões arquiteturais.

O objetivo não é apenas adicionar uma feature. É utilizar a feature para testar o próprio sistema de trabalho que construí para o agente.

Preparação para a próxima unidade

A partir daqui, eu encerro essa primeira etapa com um projeto simples que serviu para construir e exercitar os fundamentos.

Nós criamos nosso próprio fluxo de especificação, planejamento, tarefas e implementação. Também colocamos em prática regras de contexto, permissões e verificações que limitam o comportamento automático do agente.

Na próxima unidade, eu vou reforçar alguns desses assuntos e introduzir novos elementos.

A estrutura conceitual continuará parecida, mas não vamos mais depender apenas do framework que montamos manualmente.

Passaremos a trabalhar com o framework do GitHub, aproveitando uma implementação pronta dos mesmos conceitos que já praticamos.

Também começaremos um novo projeto, que será desenvolvido ao longo das etapas seguintes do módulo.

Antes dessa mudança, a atividade proposta é justamente usar a spec 003 para integrar a persistência das duas interfaces e observar, durante esse processo, se o framework criado até aqui continua conduzindo o agente da maneira esperada.

NA PRÁTICA

Abra o snapshot da Unidade 1 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Validação da aplicação pelo HTTP” no fluxo apresentado?
- Que decisão de engenharia está associada a “A limitação da CLI com persistência em memória” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Validação da aplicação pelo HTTP.
- A limitação da CLI com persistência em memória.
- Criando a spec 002.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 1

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE					
Contexto →	Constitution / Spec →	Plan →	Tasks →	Implement →	Revisão + Guardrails

CHECKLIST DA UNIDADE
- Revisar a Aula 1: O Agente de Código Por Dentro (GitHub Copilot) - Revisar a Aula 2: Context Engineering e o Contrato de Permissões - Revisar a Aula 3: Spec-Driven Development do Zero Parte 1 - Revisar a Aula 4: Spec-Driven Development do Zero Parte 2 - Revisar a Aula 5: Guardrails, Revisor e Delegação

GITHUB DA UNIDADE 1
Abrir no GitHub • Ler UNIDADE.md

Unidade 2

Padrões de Raciocínio e Execução

Construção do núcleo de raciocínio do OpsPilot e comparação prática entre ReAct, Plan-and-Execute e Reflection.

FLUXO VISUAL DA UNIDADE

Solicitação →	Estratégia →	ReAct / Plan-and-Execute →	Reflection →	Trace →	Benchmark
---------------	--------------	----------------------------	--------------	---------	-----------

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Aula	Tema
1	Raciocínio e Tomada de Decisão em Agentes - Os Três Padrões de Raciocínio
2	Configurando o Spec Kit
3	Criando a Estrutura Inicial do Projeto com Spec Kit
4	Definindo o Padrão ReAct Para o Agente
5	Definindo o Padrão Plan-and-Execute Para o Agente
6	Críticas e Benchmark com Padrão Reflection
7	Uma API que Também é um Agente

Aula 1 — Raciocínio e Tomada de Decisão em Agentes - Os Três Padrões de Raciocínio

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Compreender o projeto OpsPilot.
- Compreender raciocínio como trace estruturado.
- Compreender o padrão ReAct.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- O projeto OpsPilot
- Raciocínio como trace estruturado
- O padrão ReAct
- ReAct em um cenário operacional
- Quando utilizar ReAct
- Limitações do ReAct

Conteúdo da aula

Nesta aula, eu quero mudar a pergunta que está guiando o nosso trabalho. Até aqui, nós aprendemos a operar um agente de código utilizando um método estruturado de desenvolvimento, principalmente com Spec-Driven Development. Agora eu quero entender como um agente pensa, quando ele decide agir, quando ele precisa planejar e em que momento deve revisar o próprio resultado.

Esses comportamentos possuem padrões bem conhecidos. Os três que vamos estudar são ReAct, Plan-and-Execute e Reflection.

A proposta é estudar esses padrões de uma forma recursiva. Vamos utilizar o nosso agente de código para construir outro agente, desta vez um agente de produto que será consumido por API. Ao longo desse processo, vamos perceber que vários desses padrões já apareceram nas aulas anteriores. Nós apenas os utilizamos sem necessariamente nomeá-los dessa forma.

O projeto OpsPilot

A partir desta aula começa o projeto que vai nos acompanhar até o final do módulo: o OpsPilot.

O OpsPilot será um copiloto de plantão. A ideia é trabalhar com um cenário em que existem métricas, alertas e possíveis incidentes operacionais. Podemos imaginar, por exemplo, um problema de timeout em checkout ou algum outro comportamento que esteja gerando alertas em uma aplicação.

O agente deverá conseguir consultar esses alertas, abrir incidentes, recuperar informações de um runbook e ajudar a indicar por onde começar uma investigação.

Antes de implementar essa solução, eu quero estabelecer como o agente vai raciocinar.

Teremos três estratégias diferentes escondidas atrás de uma interface comum. Isso significa que o restante do sistema deve conversar com um contrato único, independentemente da estratégia utilizada naquele momento ser ReAct, Plan-and-Execute ou Reflection.

Essa decisão é importante porque quero poder trocar uma estratégia pela outra sem precisar alterar todo o restante da aplicação.

Raciocínio como trace estruturado

Todo o raciocínio executado pelo agente também vai produzir um trace.

Esse trace representa o registro estruturado do passo a passo seguido durante a tomada de decisão. Em vez de tratar o raciocínio apenas como algo temporário dentro de uma execução, quero persistir informações suficientes para entender o caminho seguido pelo agente.

Esse registro será importante para auditoria.

Se o agente produzir uma resposta incorreta, eu quero conseguir analisar em qual ponto da decisão algo começou a dar errado. Talvez ele tenha tomado uma ação inadequada, interpretado incorretamente uma observação ou escolhido um caminho que não fazia sentido para o contexto disponível.

Por isso, o trace será modelado como uma estrutura tipada dentro do sistema que vamos construir.

Além das estratégias, também teremos uma arena de comparação.

A arena será executada pelo terminal e permitirá submeter a mesma pergunta a duas estratégias diferentes. Depois, conseguiremos observar os raciocínios lado a lado e comparar também o custo de cada execução.

Essa arena será útil inclusive para os cenários em que queremos verificar se determinada situação operacional deveria ou não resultar em um alerta.

Mais adiante, teremos também um bench.

O bench será uma bateria fixa de testes que permitirá medir desempenho de forma objetiva. Em vez de dizer apenas que uma estratégia parece melhor, vamos avaliar métricas como acerto, número de chamadas e latência.

Com isso, a comparação deixa de ser apenas uma opinião e passa a produzir números.

Depois, todo esse trabalho poderá ser exposto por uma API real.

O padrão ReAct

O primeiro padrão é o ReAct.

É importante não confundir esse nome com React, a biblioteca utilizada em aplicações frontend. Aqui, ReAct representa a combinação de Reason e Act, ou seja, raciocinar e agir.

Antes desse tipo de abordagem ganhar força, era comum pensar em duas alternativas mais separadas.

Em uma delas, o modelo produzia uma cadeia de raciocínio e seguia tentando responder a partir dela. O problema é que, quando faltava alguma informação externa, o modelo poderia simplesmente preencher essa lacuna de forma incorreta e continuar.

Na outra alternativa, o modelo agia diretamente, sem uma etapa relevante de raciocínio. Essa abordagem podia ser mais rápida, mas muitas vezes não possuía fundamento suficiente para decisões mais complexas.

O ReAct intercala as duas coisas.

O agente pensa, realiza uma ação, observa o resultado dessa ação e volta a pensar utilizando a nova informação.

O ciclo básico é pensamento, ação e observação.

Esse processo pode se repetir várias vezes até que o agente possua informações suficientes para produzir uma resposta.

ReAct em um cenário operacional

Podemos imaginar uma pergunta simples dentro do OpsPilot: quais alertas críticos estão acontecendo agora?

O agente primeiro precisa interpretar o objetivo. Ele entende que o plantonista quer saber quais alertas estão disparando ou quantos deles estão em condição crítica.

Depois ele executa uma ação para consultar a lista de alertas.

A ferramenta retorna uma observação com os alertas disponíveis.

A partir dessa observação, o agente pensa novamente. Ele pode identificar, por exemplo, que entre vários alertas retornados apenas dois atendem ao critério de criticidade.

Com a nova informação em mãos, ele consegue responder que existem dois alertas críticos disparando.

A característica mais importante aqui é que cada nova decisão utiliza a informação mais recente disponível.

Por isso, o ReAct é adaptativo.

Quando utilizar ReAct

O ReAct faz bastante sentido quando eu não conheço o caminho completo de antemão.

Ele é útil em ambientes dinâmicos, em tarefas exploratórias e em situações nas quais uma ação pode revelar uma informação que muda completamente o próximo passo.

Um cenário de plantão é um bom exemplo.

Durante um incidente, posso não saber inicialmente qual serviço está causando o problema. Uma consulta mostra um alerta. Esse alerta aponta para uma métrica. A métrica leva a outro serviço. O próximo passo depende da observação anterior.

Nesse tipo de situação, um agente que se adapta a cada observação pode funcionar muito bem.

Ao mesmo tempo, existem limitações importantes.

Limitações do ReAct

A primeira limitação é que o ReAct trabalha um passo por vez.

Como ele reage continuamente às observações, ele não necessariamente começa com uma visão completa de todo o caminho que será percorrido.

Isso pode fazer com que o agente ande em círculos ou repita investigações que não estão aproximando a execução de uma resposta.

A segunda limitação está relacionada ao custo.

Cada passo pode exigir uma nova chamada ao modelo e essa chamada carrega o histórico necessário para continuar a execução.

Quanto mais longa a tarefa, maior tende a ser esse histórico e maior pode ficar o custo.

A terceira limitação é o risco de loop infinito.

Se o agente continuar encontrando razões para executar uma nova ação, ele pode nunca decidir que já possui informação suficiente para parar.

Por isso, nenhuma estratégia desse tipo deve ser executada sem um limite claro de interações.

Eu preciso definir um teto.

Quando esse limite é atingido, a execução precisa parar ou seguir algum comportamento controlado.

Plan-and-Execute

O segundo padrão é o Plan-and-Execute.

Aqui, em vez de descobrir todo o caminho passo a passo durante a execução, eu começo produzindo um plano.

Primeiro o agente pensa na tarefa de forma global e define uma sequência de passos. Depois outro momento da execução segue esse plano.

Se alguma informação indicar que o planejamento não é o mais adequado, o sistema pode replanejar.

O ciclo passa a ser planejamento, execução e, quando necessário, replanejamento.

Uma vantagem importante desse padrão é a possibilidade de utilizar modelos diferentes em momentos diferentes.

Na etapa de planejamento, posso escolher um modelo mais robusto, com maior capacidade de raciocínio, porque quero uma visão melhor do problema como um todo.

Na execução dos passos, posso utilizar modelos mais baratos ou componentes mais simples, dependendo do tipo de tarefa.

Dessa forma, consigo controlar melhor o custo.

Independência e paralelismo no Plan-and-Execute

Outra característica interessante é que os passos do plano podem ser independentes.

Quando não existe dependência entre duas ações, existe a possibilidade de executá-las em paralelo.

Isso pode reduzir o tempo total de uma tarefa.

O planejamento também cria uma visão global que o ReAct puro não possui da mesma forma.

Antes de executar o primeiro passo, eu já tenho uma ideia de onde quero chegar e quais etapas devem ser necessárias.

Isso não significa, porém, que o plano possa ser considerado definitivo.

O problema do plano envelhecido

Um plano pode ficar velho.

Se o contexto mudar, se um requisito for alterado ou se uma execução mostrar que uma hipótese inicial estava errada, continuar obedecendo ao planejamento original pode levar o agente a uma direção incorreta.

Por isso, o replanejamento precisa fazer parte do padrão.

Eu não quero apenas um planner e um executor.

Também é preciso considerar um replanner capaz de observar quando o plano deixou de representar a realidade atual.

Esse cuidado é importante principalmente em tarefas que se estendem por mais tempo ou em ambientes sujeitos a mudanças.

O Plan-and-Execute oferece uma visão mais estruturada e um custo potencialmente mais previsível, mas essa organização só permanece útil se o plano puder ser atualizado.

Reflection

O terceiro padrão é o Reflection.

Aqui, o processo começa com a geração de algum resultado.

Depois entra um crítico.

Esse crítico avalia o que foi produzido utilizando critérios definidos previamente.

Se o resultado for aprovado, a execução termina.

Se não for aprovado, o crítico produz feedback e o sistema gera uma nova versão tentando corrigir os problemas apontados.

Esse ciclo pode melhorar a qualidade do resultado porque existe uma etapa explícita de avaliação antes de considerar o trabalho pronto.

O crítico, porém, precisa avaliar contra critérios objetivos.

Não quero apenas um segundo modelo dizendo genericamente que algo poderia ficar melhor.

Quero critérios específicos para determinar o que precisa ser analisado.

O limite de reflexões

Reflection também precisa de limite.

Um crítico quase sempre consegue encontrar alguma possibilidade adicional de melhoria.

Se eu deixar o ciclo aberto, ele pode continuar revisando e gerando novas versões indefinidamente.

Por isso, defino um número máximo de reflexões.

Depois de determinado número de ciclos, o sistema precisa encerrar ou encaminhar o resultado para outro tipo de decisão.

Essa necessidade de parada faz parte da própria arquitetura.

Assim como no ReAct eu preciso controlar o número de interações, no Reflection eu preciso controlar quantas rodadas de crítica e regeneração serão permitidas.

Comparando os três padrões

Quando coloco os três padrões lado a lado, cada um apresenta uma vantagem principal e um custo associado.

O ReAct é adaptativo. Ele reage à observação mais recente e funciona bem quando o caminho ainda não é conhecido. Em contrapartida, trabalha passo a passo, pode acumular histórico e possui risco de ficar preso em loops.

O Plan-and-Execute oferece uma visão global. Ele permite separar planejamento e execução, pode utilizar executores mais baratos e abre a possibilidade de paralelizar operações independentes. O problema é que o plano envelhece e, por isso, exige uma estratégia de replanejamento.

O Reflection adiciona uma camada explícita de qualidade. O resultado passa por um crítico e pode ser melhorado antes da aprovação. Em troca, existe um custo adicional de chamadas e a necessidade de definir critérios e limites de parada.

Não existe uma estratégia universalmente melhor.

A escolha depende do tipo de problema que estou tentando resolver.

Os padrões já apareceram nas aulas anteriores

O ponto interessante é que nós já utilizamos os três padrões antes desta aula.

Quando trabalhamos no Agent Mode e acompanhamos o agente raciocinando, executando ações, observando ferramentas e decidindo o próximo passo, estávamos utilizando uma dinâmica equivalente ao ReAct.

O reasoning trace que acompanhávamos durante aquela execução mostrava justamente essa sequência de decisões.

Quando estruturamos o trabalho em especificar, planejar, transformar o plano em tarefas e depois implementar, estávamos aplicando uma lógica muito próxima de Plan-and-Execute.

Primeiro criamos uma visão do que precisava ser feito, depois um plano técnico e finalmente executamos as unidades de trabalho.

Quando adicionamos revisão de código e pre-commit ao processo, criamos comportamentos relacionados a Reflection.

Um resultado era produzido, passava por uma avaliação e precisava atender a critérios antes de ser aceito.

Então, esses três padrões não surgem agora como elementos completamente separados do que já fizemos.

O que muda é que agora vamos tratá-los explicitamente como estratégias de raciocínio do agente.

No OpsPilot, eles serão componentes que poderemos implementar, medir e comparar.

A partir das próximas etapas, vamos construir essas estratégias atrás de um contrato comum, registrar os traces de forma estruturada, executar perguntas em uma arena comparativa e medir resultados por meio de uma bateria fixa de testes, observando não apenas se o agente respondeu, mas como ele chegou até aquela resposta, quantas chamadas utilizou e qual foi a latência envolvida.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O projeto OpsPilot” no fluxo apresentado?
- Que decisão de engenharia está associada à “Raciocínio como trace estruturado” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- O projeto OpsPilot.
- Raciocínio como trace estruturado.
- O padrão ReAct.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Configurando o Spec Kit

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Compreender partindo do método que já conhecemos.
- Criar o projeto OpsPilot.
- Compreender variáveis de ambiente e configuração nativa do Node.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Partindo do método que já conhecemos
- Criando o projeto OpsPilot
- Variáveis de ambiente e configuração nativa do Node
- Scripts do projeto
- Preparando o .gitignore
- Copilot Instructions do OpsPilot

Conteúdo da aula

Nesta aula, eu continuo utilizando o mesmo método de trabalho que adotamos na unidade anterior: primeiro eu especifico, depois o agente executa e, em seguida, eu reviso o resultado. O que muda agora é a ferramenta que vai sustentar esse fluxo. Na unidade 1, nós construímos manualmente nosso próprio processo de Spec-Driven Development para entender como cada parte funcionava. Criamos prompts, arquivos e convenções na mão justamente para aprender o mecanismo. A partir daqui, eu não preciso mais manter esse framework artesanal.

Vamos utilizar o GitHub Spec Kit, um framework pronto e mantido pelo próprio GitHub. Ele instala no ambiente os comandos necessários para seguir o mesmo fluxo que já conhecemos, com etapas equivalentes a especificar, planejar, dividir em tarefas e implementar. A vantagem é que agora essas peças deixam de ser algo que precisamos construir manualmente e passam a fazer parte de uma estrutura pronta para uso.

Partindo do método que já conhecemos

A mudança de ferramenta não significa abandonar o processo que construímos. Pelo contrário, ela faz sentido justamente porque já entendemos o papel de cada etapa.

Na prática, eu continuo começando por uma especificação. Depois produzo o plano técnico. Em seguida, transformo esse plano em tarefas menores e só então avanço para a implementação. A revisão continua sendo humana entre as etapas.

O que o Spec Kit faz é nos entregar essa estrutura pronta. Em vez de criar cada prompt manualmente, inicializo o framework no repositório e passo a utilizar os comandos oferecidos por ele.

Essa transição é importante porque quero que, a partir desta unidade, o foco esteja menos na construção da infraestrutura do agente de código e mais no agente que vamos desenvolver como produto.

Criando o projeto OpsPilot

Antes de configurar o Spec Kit, eu preparo o novo projeto que vai acompanhar o restante do módulo.

Crio um novo diretório chamado OpsPilot e inicio nele um projeto Node.js com Git e package.json. A ideia é estabelecer desde o começo um repositório que já tenha a estrutura mínima para versionamento, instalação de dependências e execução dos scripts que vamos utilizar ao longo das próximas aulas.

A stack começa com Zod para validação. Também adiciono os pacotes relacionados a LangChain, integração com OpenAI e LangGraph, porque o agente que vamos construir vai utilizar essa base para organizar o fluxo e conversar com modelos através do OpenRouter.

Diferentemente do projeto anterior, desta vez quero utilizar Express na camada HTTP.

Também introduzo um banco de dados. Durante a preparação, considero alternativas como SQLite e Postgres, mas para este exercício decido trabalhar com MySQL. Essa escolha não é um requisito conceitual do módulo. Se eu estiver estudando para uma vaga específica ou já trabalhar com outro banco, posso substituir essa decisão e utilizar a tecnologia que fizer mais sentido para o meu contexto.

Para o ambiente de desenvolvimento, adiciono TypeScript, TSX e os tipos necessários para Node e Express. A intenção é continuar trabalhando com tipagem e validações desde o começo do projeto.

Variáveis de ambiente e configuração nativa do Node

Outro detalhe do setup é o carregamento das variáveis de ambiente.

Eu não quero instalar uma dependência adicional apenas para isso. Como o Node já oferece suporte nativo para carregar esse tipo de configuração, aproveito a capacidade disponível na própria plataforma.

O projeto terá um arquivo de exemplo de ambiente contendo a variável necessária para a chave de acesso ao serviço que utilizaremos, mas os arquivos reais de ambiente não devem ser versionados.

O arquivo de exemplo mostra quais configurações são esperadas para executar o projeto. Já os arquivos com valores reais podem conter credenciais e, por isso, devem permanecer fora do Git.

Scripts do projeto

No package.json, eu preparo os scripts que serão utilizados durante o desenvolvimento.

O primeiro é o script de desenvolvimento, responsável por iniciar a aplicação TypeScript a partir do arquivo principal.

Também teremos um script chamado arena. A arena será utilizada mais adiante para executar cenários em que queremos comparar estratégias de raciocínio do agente, observar como cada uma responde e analisar os resultados.

Outro script será o bench. Ele servirá para executar a bateria de avaliações que vamos construir para medir o comportamento das estratégias de forma mais objetiva.

Também configuro o script de testes utilizando o mecanismo de testes do Node com suporte do TSX para os arquivos TypeScript.

Por fim, mantenho o type check como uma verificação explícita do projeto. Quero conseguir executar a checagem de tipos separadamente e garantir que essa validação faça parte do fluxo normal de desenvolvimento.

Nesse primeiro momento, não quero começar criando toda a aplicação dentro do src. A intenção é preparar somente o necessário para que o projeto e o type check consigam funcionar. A implementação das funcionalidades deve nascer depois, orientada pelas especificações.

Preparando o.gitignore

O.gitignore também faz parte desse setup inicial.

Eu ignoro node_modules, os arquivos reais de ambiente e suas variações para desenvolvimento e produção. Ao mesmo tempo, mantenho um arquivo de exemplo versionado para documentar quais variáveis precisam existir.

A pasta de build também deve permanecer fora do controle de versão, porque representa um artefato gerado a partir do código-fonte.

Quero estabelecer essas regras antes de começar a delegar trabalho ao agente. Quanto mais clara estiver a estrutura inicial, menor é a possibilidade de ele tomar decisões incompatíveis com o fluxo que estou tentando construir.

Copilot Instructions do OpsPilot

Depois do esqueleto inicial, volto ao mesmo princípio que utilizamos anteriormente: o agente precisa conhecer o projeto no qual está trabalhando.

Por isso, crio o arquivo de Copilot Instructions dentro da pasta.GitHub.

Esse arquivo deve continuar curto e factual.

Não quero transformar as instructions em documentação extensa. Quero registrar os fatos que o agente precisa ter disponíveis durante o trabalho: qual é o projeto, qual é a stack, quais convenções precisam ser respeitadas e qual é o fluxo de desenvolvimento.

Defino o OpsPilot como um copiloto de plantão responsável por lidar com alertas e incidentes de produção.

Também registro que a API será construída como um agente baseado em LangChain e LangGraph, utilizando modelos acessados pelo OpenRouter.

A stack mantém vários princípios que já utilizamos no projeto anterior, mas agora inclui Express e a tecnologia de banco de dados escolhida para esta aplicação.

Comandos e convenções do projeto

Em vez de copiar manualmente todos os comandos para as instructions, posso orientar o agente a consultar o package.json. Isso evita duplicar informações que já possuem uma fonte clara dentro do repositório.

As convenções, por outro lado, precisam estar explícitas.

Toda entrada externa deve ser validada. Erros previsíveis devem ser tratados de forma coerente dentro da aplicação.

A organização das camadas deve seguir o padrão adotado para este projeto, baseado em uma estrutura MVC com service para concentrar a lógica de aplicação.

Também quero que a nova lógica nasça acompanhada de testes.

Outra convenção que reforço é a preferência por funções puras sempre que isso for possível. Essa regra continua sendo útil para manter determinadas partes da aplicação mais previsíveis e fáceis de testar.

Segredos não devem ser adicionados ao repositório e o agente não deve utilizar o conteúdo dos arquivos reais de ambiente como uma fonte comum de leitura.

Também registro que as especificações devem ser versionadas.

O fluxo agora pertence ao Spec Kit

Nas instructions, deixo claro que o projeto deve seguir o fluxo do Spec Kit.

Esse detalhe precisa ser revisado com cuidado.

Quando inicialmente peço ao agente para registrar apenas que deve seguir o fluxo do GitHub Spec Kit, ele coloca praticamente essa mesma frase no arquivo. Tecnicamente, ele executou o que pedi, mas não produziu o nível de precisão que eu queria.

Eu não devo assumir que um arquivo está correto apenas porque foi criado. Preciso abrir, revisar e verificar se aquilo representa realmente a intenção do projeto.

Nesse caso, atualizo a seção de fluxo para deixar as etapas explícitas: primeiro a especificação, depois o planejamento, depois as tarefas e, por fim, a implementação.

Se o agente não transformar uma orientação genérica em uma regra operacional adequada, eu corrijo o contexto.

Settings do VS Code por projeto

Outra mudança importante é que não quero depender apenas das minhas configurações globais do VS Code.

Crio um settings.json dentro da pasta.vscode do próprio projeto.

A vantagem é que as regras passam a acompanhar especificamente o OpsPilot. Isso evita que uma configuração adequada para este projeto interfira em outro repositório com necessidades diferentes.

Nesse arquivo, volto a trabalhar com allow list e deny list.

Allow list para operações rotineiras

Na allow list, quero colocar comandos que considero seguros e recorrentes.

A execução dos testes deve acontecer sem pedir autorização toda vez. A verificação de tipos também pode ser executada automaticamente.

Operações básicas do Node e determinados comandos do Git, como consultar o status, visualizar diferenças, adicionar alterações e criar commits, também podem fazer parte desse conjunto.

O objetivo é reduzir atrito. Se uma operação faz parte da rotina e eu já determinei que ela é segura naquele projeto, não faz sentido interromper a execução a cada chamada para solicitar a mesma confirmação.

Deny list e aprovação manual

A deny list representa o outro lado deste contrato.

Operações potencialmente destrutivas continuam exigindo atenção.

Remoções recursivas, uso de privilégios elevados, leitura de arquivos de ambiente e operações de Git capazes de sobrescrever histórico precisam permanecer protegidas.

Também quero que o git push fique fora da automação comum e passe por aprovação manual. Principalmente, não quero permitir push forçado.

A intenção é automatizar aquilo que é previsível e de baixo risco, mas continuar colocando uma pessoa no caminho das ações que podem causar perda de dados, exposição de segredos ou mudanças irreversíveis no histórico remoto.

Inicializando o GitHub Spec Kit

Com o projeto preparado, posso inicializar o Spec Kit.

O processo é executado dentro do repositório existente.

Como a pasta já contém arquivos, a ferramenta avisa que o diretório não está vazio e que os templates serão incorporados ao conteúdo atual. Eu confirmo porque essa é justamente a intenção.

Durante a configuração, seleciono o agente que estou utilizando. No nosso caso, escolho GitHub Copilot.

Também escolho o tipo de shell correspondente ao ambiente em que estou trabalhando.

A partir daí, o Spec Kit adiciona uma quantidade considerável de arquivos e estruturas ao projeto.

Essa quantidade faz sentido porque agora estamos utilizando um framework pronto, diferente do pequeno conjunto de arquivos que montamos manualmente na unidade anterior.

Dentro da estrutura criada, encontramos os elementos necessários para os comandos e para a memória do processo.

Na área de prompts do GitHub aparecem comandos adicionais, incluindo recursos de análise e checklist além das etapas principais do fluxo.

Também existe a estrutura interna do próprio Spec Kit, que inclui a memória onde fica a Constitution utilizada pelo framework.

Revisando o que o Spec Kit instalou

Depois da inicialização, eu não considero o trabalho encerrado.

Primeiro reviso as instructions do projeto para verificar se os comandos e convenções continuam coerentes com o que defini.

Foi justamente nessa revisão que percebi que o fluxo do Spec Kit havia ficado genérico demais e precisei atualizar explicitamente as etapas.

Esse comportamento reforça a prática que quero manter durante todo o módulo: eu específico, o agente executa e eu reviso.

O framework não elimina a revisão. Ele organiza o processo.

Adaptando a Constitution ao novo projeto

Ainda falta uma peça importante: a Constitution.

No projeto anterior, nós criamos manualmente uma Constitution com os princípios que deveriam orientar todas as fases do SDD.

Para acelerar o setup, posso utilizar aquela estrutura como referência, mas não posso simplesmente copiá-la e assumir que serve para o OpsPilot.

O novo projeto possui decisões diferentes.

A camada HTTP mudou, agora existe Express, a organização segue uma estrutura MVC com service e a stack inclui componentes relacionados ao agente e ao banco de dados.

Por isso, peço ao agente para atualizar a Constitution com base no package.json e nas novas Copilot Instructions.

Na primeira tentativa, o resultado ainda carrega ideias de camadas e fluxos do projeto anterior.

Esse é novamente um ponto em que a revisão encontra um problema real.

Eu aviso que as camadas descritas não correspondem ao MVC comum que estamos adotando e solicito o ajuste.

Atualizando a Constitution nativa do Spec Kit

Como agora estamos usando o Spec Kit, não quero manter apenas uma Constitution paralela criada por nós.

O próprio framework possui o arquivo de Constitution dentro de sua estrutura de memória.

Por isso, peço que as regras atualizadas sejam replicadas também para a Constitution nativa do Spec Kit.

A ideia é evitar duas fontes divergentes descrevendo princípios diferentes para o mesmo projeto.

Depois de confirmar que a Constitution correta está no local utilizado pelo framework, posso remover o arquivo duplicado que havia criado durante a adaptação.

Isso deixa o projeto mais limpo e reduz o risco de o agente consultar a fonte errada.

Durante esse processo, também consigo observar novamente o reasoning trace do agente enquanto ele analisa os arquivos e decide quais alterações precisa realizar. Esse rastro continua sendo útil para entender o caminho seguido durante uma tarefa de configuração.

Com o Spec Kit inicializado, as Copilot Instructions revisadas, as permissões específicas do projeto definidas e a Constitution alinhada à arquitetura do OpsPilot, o setup fica preparado para a próxima etapa, em que vamos configurar o OpenRouter e começar a trabalhar efetivamente com o agente externo que será consumido pela aplicação.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Partindo do método que já conhecemos” no fluxo apresentado?
- Que decisão de engenharia está associada a “Criando o projeto OpsPilot” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Partindo do método que já conhecemos.
- Criando o projeto OpsPilot.
- Variáveis de ambiente e configuração nativa do Node.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Criando a Estrutura Inicial do Projeto com Spec Kit

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Configurar o acesso ao OpenRouter.
- Especificar o núcleo de raciocínio.
- Compreender trace como parte do contrato.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Configurando o acesso ao OpenRouter
- Especificando o núcleo de raciocínio
- Trace como parte do contrato
- Métricas desde o início
- Uma fábrica única para o modelo
- Ferramentas mock e store em memória

Conteúdo da aula

Antes de implementar as estratégias de raciocínio do OpsPilot, eu preciso preparar a estrutura que vai sustentar o agente. Até aqui, nós já definimos que o projeto terá diferentes formas de raciocínio e que essas estratégias deverão poder ser comparadas entre si. Agora eu quero transformar essa ideia em uma especificação concreta utilizando o Spec Kit.

O primeiro ponto é que o agente precisa de um modelo. Para isso, vamos utilizar o OpenRouter. Ele funciona como um gateway compatível com o formato da OpenAI e disponibiliza acesso a diversos modelos, incluindo opções pagas e gratuitas. Para este curso, essa escolha ajuda principalmente em três aspectos: custo, flexibilidade e possibilidade de fallback.

A questão de custo é relevante porque existem modelos gratuitos identificados com a indicação de uso free. A proposta é conseguir executar toda esta unidade sem depender obrigatoriamente de um modelo pago. A flexibilidade aparece porque eu consigo trocar o modelo utilizado alterando uma configuração, sem precisar reestruturar toda a aplicação. Mais adiante, também poderemos trabalhar com fallback automático entre modelos.

Configurando o acesso ao OpenRouter

Antes de começar a escrever a especificação, eu preparo o acesso ao OpenRouter.

No site do serviço, crio uma chave de API específica para o OpsPilot. Posso configurar uma expiração para essa chave e, dependendo do cenário, também definir limites. No exemplo da aula, a chave é criada com validade de sete dias.

Depois de gerar a credencial, eu a salvo no arquivo de ambiente do projeto.

Essa chave não deve ser enviada para o GitHub. Ela é uma credencial real e precisa permanecer fora do repositório. Esse cuidado se conecta diretamente às regras que já definimos nas instructions e no .gitignore. O projeto pode versionar um arquivo de exemplo com o nome das variáveis esperadas, mas não deve versionar os valores reais.

Também posso acessar a área de modelos do OpenRouter e escolher explicitamente um modelo. O serviço oferece identificadores que podem ser utilizados diretamente na configuração da aplicação.

Entretanto, nesta aula eu não quero fixar um modelo gratuito específico.

Eu já fiz testes prévios e percebi que alguns modelos gratuitos podem apresentar uma latência muito alta. Para evitar que a experiência dependa de um modelo específico que esteja lento naquele momento, vou utilizar a opção automática de modelos gratuitos do OpenRouter.

A configuração utiliza a seleção free do próprio OpenRouter, permitindo que o gateway escolha entre as opções gratuitas disponíveis. O objetivo é ter uma resposta mais prática para os exercícios sem amarrar o projeto a um único modelo.

Especificando o núcleo de raciocínio

Com o acesso ao modelo preparado, volto ao fluxo do Spec Kit.

Agora eu uso a etapa de especificação para descrever o núcleo de raciocínio do OpsPilot.

Esse é um ponto importante porque estou especificando o agente externo que será consumido pela aplicação. O agente de código, que está me ajudando a construir o projeto, já está orientado pelas instructions, pela Constitution e pelo fluxo do Spec Kit. O que estou definindo agora é a arquitetura do agente de produto.

A primeira decisão é ter uma interface comum para todas as estratégias.

Quero poder trocar ReAct por Plan-and-Execute e, futuramente, comparar outras estratégias sem modificar o restante da aplicação. Para isso, vou utilizar uma abstração de estratégia de raciocínio.

Essa estratégia terá uma identificação e uma operação principal de execução. Ela recebe uma entrada e devolve um resultado estruturado.

O retorno precisa conter a resposta final, o trace produzido durante a execução e as métricas associadas ao processamento.

Trace como parte do contrato

O trace não será apenas um texto solto.

Eu quero que ele seja composto por eventos tipados. Esses eventos representam os diferentes momentos da tomada de decisão.

Entre os tipos previstos estão registros gerais, pensamentos, ações, observações, planejamento, crítica e resposta.

A ação precisa carregar qual ferramenta ou operação foi escolhida e quais argumentos foram utilizados.

Essa estrutura é importante porque quero conseguir analisar posteriormente o caminho percorrido pelo agente. Se a execução der errado, não quero ter apenas uma resposta final incorreta. Quero um histórico estruturado que mostre quais decisões foram registradas e quais ações ocorreram.

O trace também vai alimentar a arena que criaremos mais adiante.

Métricas desde o início

O resultado de cada estratégia também precisa incluir métricas.

Eu quero contar quantas chamadas ao LLM foram realizadas e registrar a latência da execução em milissegundos.

Essas informações são necessárias porque a comparação entre estratégias não pode depender apenas de qual resposta parece melhor.

Uma estratégia pode acertar mais, mas consumir muito mais chamadas. Outra pode ser mais rápida, mas perder qualidade. Ao registrar essas métricas desde o começo, preparo o projeto para fazer comparações objetivas no bench e na arena.

Uma fábrica única para o modelo

Outro requisito da especificação é concentrar a criação do modelo em uma fábrica única.

Não quero que cada estratégia instancie seu próprio cliente de forma diferente.

A configuração do modelo deve ler a chave do OpenRouter, o nome do modelo e a base URL correspondente. Também quero uma temperatura padrão igual a zero.

Essa fábrica centraliza a configuração e facilita trocar o modelo depois.

Se eu decidir sair da seleção automática gratuita e fixar outro modelo, a alteração precisa acontecer em um único ponto. O mesmo vale para mudanças de endpoint ou outros parâmetros relacionados ao cliente.

Ferramentas mock e store em memória

Antes de conectar o OpsPilot a sistemas reais, eu quero trabalhar com ferramentas mock.

Essas ferramentas ficam disponíveis para as estratégias de raciocínio e operam sobre uma store em memória previamente populada.

O detalhe de estar pré-populada é importante. Não quero iniciar a aplicação com um ambiente vazio e depender de criação manual de dados antes de cada teste.

A especificação determina um conjunto inicial com cinco serviços e seis alertas variados.

Dos seis alertas, três devem estar disparando e três devem estar resolvidos.

Essa distribuição cria um cenário mínimo para testar consultas, decisões e abertura de incidentes.

Também quero um script específico para executar o seed.

Assim, o estado necessário para os exercícios pode ser preparado de forma repetível.

Operações disponíveis para o agente

As ferramentas mock precisam oferecer algumas capacidades principais.

A primeira é listar alertas. Essa consulta deve aceitar um status como parâmetro para permitir que a estratégia filtre aquilo que está disparando ou aquilo que já foi resolvido.

Outra operação é abrir um incidente.

Para abrir um incidente, o agente precisa fornecer informações como título, serviço e severidade.

Também haverá uma operação para resolver um incidente existente a partir do seu identificador.

Esses dados devem ser validados com Zod.

A especificação também prevê o uso de MySQL com Sequelize na estrutura do projeto, embora nessa fase inicial parte dos testes e do comportamento possa funcionar com dados em memória. O objetivo é começar com um ambiente controlado e depois avançar para a persistência real.

Primeira estratégia: ReAct

A primeira estratégia concreta será o ReAct.

Ela ficará isolada em sua própria implementação dentro da área de agentes.

A intenção é utilizar o suporte de agente ReAct disponível no ecossistema de LangChain.

Essa estratégia seguirá o comportamento que estudamos anteriormente: o agente raciocina, executa uma ação, observa o resultado e então decide o próximo passo.

Como o trace é parte do contrato comum, cada uma dessas etapas precisa resultar em eventos estruturados que possam ser registrados.

Eu também já considero que a primeira implementação talvez não saia exatamente no formato esperado. Por isso, antes da etapa final de implementação, pretendo fornecer um exemplo mais concreto para orientar o agente de código.

Esse cuidado faz parte do mesmo processo de revisão que já estamos utilizando. O Spec Kit organiza o fluxo, mas eu continuo responsável por verificar se o resultado está coerente com o desenho que quero.

Segunda estratégia: Plan-and-Execute

A segunda estratégia será o Plan-and-Execute.

Nesse caso, eu quero estruturar o comportamento como um grafo.

O processo começa com um planner responsável por gerar uma lista estruturada de passos.

Depois entra um executor, que trabalha em um passo por vez.

Após cada passo, um replanner deve revisar aquilo que ainda resta do plano. Isso permite ajustar a execução quando alguma observação se torna parte do planejamento original inadequada.

O processo termina quando não houver mais passos ou quando o limite máximo for alcançado.

Na especificação, defino um teto de oito passos.

Essa limitação é importante porque já discutimos o risco de loops e execuções sem controle. Nenhuma das estratégias deve poder crescer indefinidamente.

Limites e contagem de chamadas

O limite de iterações não vale apenas para o Plan-and-Execute.

Toda estratégia precisa respeitar um teto de execução.

Também quero contabilizar as chamadas feitas ao LLM.

Esses dois requisitos fazem parte da observabilidade mínima do agente.

Se uma estratégia começar a consumir chamadas demais, quero conseguir enxergar isso nas métricas. Se atingir o máximo de iterações, o sistema precisa ter um comportamento previsível em vez de continuar executando indefinidamente.

Essa preocupação precisa estar no contrato desde o começo, e não ser adicionada apenas depois que surgir um problema de custo ou loop.

A arena de comparação

Outro elemento da especificação é a arena.

A arena será um comando executado no terminal capaz de rodar uma ou mais estratégias sobre a mesma entrada.

Depois da execução, ela deve imprimir os traces e as métricas.

Isso permitirá comparar estratégias em condições equivalentes.

Se eu enviar a mesma pergunta para ReAct e Plan-and-Execute, quero observar não apenas as respostas finais, mas também a quantidade de passos, as chamadas de modelo e a latência.

O comando também precisa aceitar parâmetros relacionados às estratégias que serão executadas e ao máximo de iterações.

Com isso, a arena se torna a primeira ferramenta prática para experimentar o comportamento do núcleo de raciocínio.

Testes determinísticos e sem rede

Mesmo trabalhando com agentes e LLMs, parte dos testes precisa permanecer determinística.

Quero testes para a store e para a formatação dos traces sem depender da rede.

Essa decisão é importante porque testes básicos de domínio e infraestrutura não devem ficar instáveis por causa de disponibilidade de API, variação do modelo ou latência externa.

A integração com o modelo será testada em outros níveis.

Para os componentes que podem ser avaliados localmente, a preferência é manter testes previsíveis e reproduzíveis.

O que o Spec Kit gera a partir da especificação

Depois de enviar esse conjunto de requisitos para a etapa de specify, o Spec Kit cria a estrutura correspondente dentro da pasta de specs.

Ele também gera uma checklist associada à especificação.

Nesse momento, é útil revisar o papel de cada agente envolvido.

O nosso processo de SDD já está montado porque o agente de código está operando com Spec Kit. Ele já conhece as instructions, a Constitution e os artefatos que deve consultar.

O que estamos pedindo agora é que esse agente construa o escopo das estratégias que serão executadas pelo agente externo acessado pelo OpenRouter.

Essa separação evita confundir o raciocínio do Copilot durante o desenvolvimento com as estratégias de raciocínio que farão parte do produto OpsPilot.

Revisando a spec gerada

Quando a geração termina, eu reviso a spec.

Ela contém os casos de uso e os requisitos derivados do grande conjunto de informações que forneci.

Nesse caso, o Spec Kit informa que a especificação foi concluída sem ambiguidades abertas.

A próxima etapa sugerida pelo próprio fluxo é o planejamento.

Eu avanço então para a etapa de plan.

O framework executa os comandos necessários para analisar a Constitution, as instructions e a especificação. Durante esse processo, algumas execuções podem solicitar aprovação no terminal.

Se esses comandos passarem a ser recorrentes e eu considerar que são seguros, posso colocá-los na allow list do projeto para evitar confirmações repetidas nas próximas features.

Planejamento gerado pelo Spec Kit

Depois de algum tempo, o planejamento é concluído.

O Spec Kit gera diferentes artefatos de apoio, incluindo o plano técnico, pesquisa e modelo de dados.

Eu posso revisar cada um deles e alterar decisões antes de seguir.

Essa possibilidade continua sendo importante. O fato de o framework gerar automaticamente uma estrutura mais rica não significa que eu precise aceitar tudo sem leitura.

O planejamento é mais uma etapa de revisão antes da implementação.

Também posso aproveitar esse momento para ajustar a decisão sobre armazenamento em memória ou persistência real, caso perceba que o plano seguiu uma direção diferente daquela que quero utilizar no exercício.

Transformando o plano em tarefas

Com o planejamento pronto, avanço para a etapa de tasks.

O agente lê a especificação, a Constitution, o planejamento e os demais arquivos relevantes antes de montar a lista de trabalho.

O reasoning trace apresentado durante essa execução permite acompanhar quais informações ele consultou antes de chegar às tarefas.

Depois da análise, o Spec Kit produz a lista das atividades necessárias para construir a feature.

Essa é a mesma lógica que praticamos na unidade anterior, agora executada por um framework pronto.

Ainda não quero avançar automaticamente para toda a implementação.

Na próxima etapa, pretendo fornecer diretrizes adicionais e um exemplo do padrão de código esperado antes de liberar a implementação completa das estratégias. Isso é especialmente importante para o ReAct, porque quero garantir que a integração com LangChain e a estrutura do trace sigam o contrato definido na especificação.

Também sei que, quando começarmos a chamar o modelo de verdade, precisaremos lidar com respostas e erros vindos da API. O tratamento desses dados fará parte das próximas etapas do projeto, quando passaremos do desenho inicial para a execução efetiva das estratégias.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Configurando o acesso ao OpenRouter” no fluxo apresentado?
- Que decisão de engenharia está associada a “Especificando o núcleo de raciocínio” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Configurando o acesso ao OpenRouter.
- Especificando o núcleo de raciocínio.
- Trace como parte do contrato.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Definindo o Padrão ReAct Para o Agente

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Compreender referência para a Factory do modelo.
- Compreender referência para as tools do agente.
- Compreender a descrição da ferramenta também orienta o modelo.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Referência para a Factory do modelo
- Referência para as tools do agente
- A descrição da ferramenta também orienta o modelo
- Tools como parte do ambiente do ReAct
- Referência para a estratégia ReAct
- Um limite de recursão como guardrail

Conteúdo da aula

Agora que a especificação, o planejamento e as tarefas já foram gerados pelo Spec Kit, falta executar a etapa de implementação. Antes de simplesmente liberar o agente para escrever todo o código, eu quero estabelecer algumas referências para garantir que a implementação siga o padrão de software que espero para o OpsPilot.

Esse cuidado é importante porque o Spec Kit organiza muito bem o fluxo de desenvolvimento, mas ele não substitui decisões de design que eu quero preservar no código. Se eu apenas executar a etapa de implementação sem fornecer nenhuma referência adicional, o agente pode produzir uma solução tecnicamente funcional, mas estruturada de uma maneira diferente daquela que quero ensinar e utilizar no restante do módulo.

Por isso, nesta aula eu preparo três referências principais: a Factory responsável por criar o modelo, a definição das tools utilizadas pelo agente e a estratégia ReAct propriamente dita. Esses exemplos não representam a implementação final. Eles funcionam como parâmetros para o agente de código inferir a estrutura desejada.

Referência para a Factory do modelo

A primeira referência está relacionada à criação do modelo.

Eu quero centralizar a inicialização do cliente utilizado para conversar com o OpenRouter. Em vez de deixar cada estratégia montar seu próprio cliente, mantenho uma função de criação única dentro da área de agentes.

Essa função será responsável por instanciar o cliente de chat utilizado pela aplicação.

Os dados de configuração devem vir das variáveis de ambiente já preparadas no projeto. Entre elas estão o nome do modelo e a chave de API do OpenRouter.

Esse detalhe precisa ser conferido com cuidado. Durante a escrita da referência, eu não confio automaticamente nos nomes sugeridos pelo autocomplete. Volto ao arquivo de ambiente e verifico quais nomes de variáveis foram realmente definidos.

Esse hábito é importante porque uma diferença pequena no nome de uma variável pode gerar um erro difícil de perceber durante a execução. O agente também pode inferir um nome plausível, mas diferente daquele estabelecido no projeto.

A configuração do cliente também precisa apontar para a base URL da API do OpenRouter.

Dessa forma, continuo utilizando uma interface compatível com o formato esperado pelo cliente, mas direciono as chamadas para o gateway que escolhemos na aula anterior.

Outro parâmetro relevante é a temperatura.

Durante a construção da referência, considero valores diferentes, mas o importante é deixar essa configuração explícita. A temperatura influencia o nível de variação das respostas do modelo e, em um agente que será comparado por meio de métricas e testes, esse comportamento precisa ser controlado de maneira consciente.

A referência da Factory serve, portanto, para mostrar ao agente de código que eu quero uma única origem para a configuração do modelo, lendo as variáveis já existentes e retornando um cliente pronto para ser utilizado pelas estratégias.

Referência para as tools do agente

A segunda referência define as ferramentas que o agente poderá utilizar.

Esse ponto é central para o padrão ReAct porque o agente não deve apenas produzir texto. Ele precisa ser capaz de escolher ações, executar ferramentas e utilizar o resultado dessas ações para decidir o próximo passo.

As tools são justamente a ponte entre o raciocínio do modelo e as operações disponibilizadas pelo sistema.

A primeira ferramenta que eu modelo como referência é a listagem de alertas.

Ela recebe um status e consulta a store da aplicação.

Os estados previstos para essa consulta incluem alertas disparando, alertas resolvidos e a possibilidade de consultar todos.

Também estabeleço um valor padrão para o status. Isso ajuda a tornar a ferramenta previsível quando o agente não informa explicitamente um filtro.

A descrição da ferramenta também orienta o modelo

Uma tool não é composta apenas por nome e implementação.

A descrição tem um papel muito importante porque será lida pelo modelo durante a decisão sobre qual ferramenta utilizar.

Por isso, eu quero uma descrição suficientemente clara para explicar em que situações a listagem de alertas deve ser escolhida.

Se o plantonista perguntar o que está disparando, como estão os serviços ou até mesmo como está o plantão, essa ferramenta é uma candidata natural.

Quanto melhor eu descrevo a finalidade da ferramenta, menor é a ambiguidade para o modelo durante a escolha da ação.

Isso é diferente de escrever uma descrição extensa sem necessidade. A ideia continua sendo factual e precisa, mas agora essa precisão ajuda o próprio agente a decidir qual operação executará.

Os parâmetros da tool são validados com Zod.

Com isso, a entrada que o modelo fornece para a ferramenta também passa por um contrato explícito. O agente pode gerar argumentos dinamicamente, mas esses argumentos precisam respeitar a estrutura aceita pela aplicação.

Essa combinação de nome, descrição e schema transforma a ferramenta em uma interface muito mais controlada.

Tools como parte do ambiente do ReAct

No OpsPilot, as tools permitirão consultar o estado do plantão e também atuar sobre incidentes.

A listagem de alertas é apenas uma das operações.

A estrutura preparada anteriormente também prevê abertura e resolução de incidentes. Quando o agente recebe uma pergunta, ele pode precisar primeiro consultar informações e, dependendo da situação, decidir se uma operação adicional faz sentido.

Esse comportamento é exatamente o tipo de ciclo que estudamos no ReAct.

O modelo raciocina sobre a entrada, escolhe uma tool, recebe uma observação e então decide o que fazer em seguida.

Por isso, a qualidade das tools influencia diretamente a qualidade do agente.

Uma ferramenta mal descrita, com parâmetros ambíguos ou retorno inconsistente, pode fazer o agente escolher ações inadequadas mesmo que o modelo seja bom.

Referência para a estratégia ReAct

A terceira referência é a implementação da própria estratégia ReAct.

Ela precisa seguir o contrato comum de estratégia de raciocínio definido na especificação.

A estratégia possui um nome e uma operação de execução.

Quando recebe uma entrada, a primeira coisa que quero registrar é o momento de início da execução. Esse timestamp será utilizado depois para calcular a latência total.

Em seguida, a estratégia cria o agente ReAct utilizando o modelo fornecido pela Factory e o conjunto de tools do OpsPilot.

A ideia é aproveitar a implementação já disponível no ecossistema de LangChain e LangGraph, em vez de reconstruir manualmente todo o mecanismo de chamada de ferramentas e controle da conversa.

A entrada do usuário é então enviada ao agente como uma mensagem.

A partir daí, o agente pode raciocinar, selecionar ferramentas, observar seus resultados e continuar até produzir uma resposta.

Um limite de recursão como guardrail

Existe um detalhe dessa referência que considero especialmente importante: o limite de recursão.

Eu já discuti anteriormente que um agente ReAct pode entrar em um ciclo.

Como cada observação pode gerar uma nova ação, nada garante que o modelo decidirá parar no momento que eu considero adequado.

Por isso, a execução precisa de um teto.

Na referência da aula, estabeleço um limite de doze ciclos.

O número específico pode ser alterado conforme o cenário, mas o conceito precisa permanecer.

Isso funciona como um guardrail para impedir um loop infinito.

Em vez de confiar apenas no modelo para decidir quando encerrar, coloco uma restrição técnica na execução.

Esse é o mesmo princípio que já utilizamos em outros pontos do módulo: comportamentos críticos não devem depender somente de uma instrução textual. Sempre que possível, eu transformo a restrição em algo determinístico no sistema.

Extraindo a resposta do resultado

Depois que o agente termina a execução, eu preciso converter o resultado para o contrato comum da estratégia.

A primeira parte é a resposta final.

O agente produz uma sequência de mensagens durante a execução, e eu quero extrair o conteúdo textual correspondente à resposta mais recente.

Essa resposta é o que eventualmente será apresentada ao usuário.

No entanto, ela não é o único dado que me interessa.

Também preciso converter o histórico de mensagens em um trace estruturado.

O objetivo é transformar os elementos produzidos pelo agente em eventos compatíveis com os tipos definidos anteriormente.

Assim, ações, observações e respostas deixam de existir apenas como mensagens internas da biblioteca e passam a fazer parte do modelo de auditoria do OpsPilot.

Construindo as métricas da execução

A estratégia também devolve métricas.

Uma delas é a quantidade de chamadas ao LLM.

Em vez de estimar esse número externamente, eu posso analisar as mensagens produzidas durante a execução e contar aquelas associadas às respostas do modelo.

Essa informação será importante quando começarmos a comparar estratégias na arena.

A outra métrica é a latência.

Como registrei o instante inicial antes da execução, consigo calcular o tempo total quando a estratégia termina.

Com isso, cada chamada do ReAct já produz os três componentes que definimos no contrato: resposta, trace e métricas.

Essa padronização é fundamental porque o restante da aplicação não precisa conhecer os detalhes internos da estratégia.

Quando implementarmos o Plan-and-Execute, a lógica será diferente, mas o resultado poderá seguir a mesma forma.

Utilizando os exemplos na etapa de implementação

Depois de preparar as três referências, eu volto para o agente de código.

Agora executo a etapa de implementação do Spec Kit e informo explicitamente que os exemplos fornecidos devem ser utilizados como referência para seus respectivos arquivos.

Esse é um ponto importante do meu fluxo.

Eu não estou substituindo o Spec Kit por código manual.

A especificação, o planejamento e as tarefas continuam sendo a base do processo.

O que faço é complementar a etapa de implementação com um padrão concreto para reduzir a chance de o agente interpretar de forma diferente alguns detalhes arquiteturais.

A partir daí, deixo o agente executar as tarefas necessárias.

Esse processo leva alguns minutos porque ele precisa consultar os artefatos do Spec Kit, criar os arquivos, utilizar as dependências existentes, ajustar tipos e executar as validações previstas pelo projeto.

Depois da execução, volto para a revisão.

Revisando a interface da estratégia

Um dos primeiros arquivos criados representa os tipos e contratos relacionados às estratégias.

Essa estrutura é importante porque é o ponto comum entre os diferentes padrões de raciocínio.

A aplicação não deve depender diretamente dos detalhes do ReAct.

Ela deve depender do contrato de estratégia.

Isso permite que eu troque a implementação posteriormente e mantenha arena, métricas e outros componentes funcionando sobre a mesma interface.

Na revisão, verifico se essa abstração foi criada de forma compatível com aquilo que havíamos especificado.

Revisando a Factory gerada

Também reviso a Factory responsável pelo modelo.

O resultado fica bastante próximo do formato recomendado para utilização do cliente com OpenRouter.

A Factory retorna uma instância configurada com os valores necessários para a comunicação com o modelo.

O principal aspecto que quero preservar é a centralização.

Quando outras estratégias precisarem utilizar o LLM, elas não deverão repetir a configuração.

Isso também facilita futuras mudanças de fallback, modelo ou parâmetros globais.

Revisando as tools

Em seguida, observo o arquivo de tools.

O agente utiliza a referência que forneci para estruturar as operações disponíveis ao modelo.

É nesse ponto que ficarão as funções que permitem consultar alertas e atuar sobre incidentes.

Inicialmente, essa interação será explorada principalmente pela linha de comando e pela arena.

Mais adiante, podemos pensar em outras interfaces, mas neste momento quero garantir primeiro que o núcleo do agente funcione.

O importante é que as ferramentas permaneçam tipadas, validadas e descritas de maneira adequada para o LLM.

Seed e dados em memória

A implementação também cria o seed em memória que havíamos solicitado na especificação.

Ao revisar os dados, verifico se o cenário inicial possui a distribuição esperada.

Existem alertas associados aos serviços e temos três alertas em estado de firing e três em estado resolved.

Esse conjunto é suficiente para começar a testar perguntas sobre o estado do plantão.

Ter dados conhecidos também será útil para os testes e para a arena, porque consigo prever determinadas respostas esperadas em cenários controlados.

O agente também prepara testes para a store em memória e para componentes relacionados à construção ou formatação dos dados.

Isso preserva a regra de que lógica nova deve nascer acompanhada de teste.

Arena e arquivos ainda pendentes

Durante a revisão, percebo que o agente também criou a estrutura inicial da arena.

É nela que mais adiante vamos executar entradas e comparar estratégias.

Por outro lado, observo que o Plan-and-Execute ainda não foi efetivamente implementado naquele momento, mesmo que tenha aparecido entre os artefatos esperados.

Esse tipo de verificação é justamente a razão pela qual eu não considero a saída do agente automaticamente pronta.

Eu recarrego a janela do projeto e confirmo quais arquivos realmente existem.

A aula atual está concentrada no ReAct, então não preciso corrigir imediatamente toda a estratégia seguinte.

O Plan-and-Execute será tratado de forma específica na próxima etapa.

Também considero ajustar a tecnologia usada para persistência ao longo do projeto. Mesmo que uma escolha inicial tenha sido feita durante a especificação, eu posso trocar a abordagem para algo mais simples, como SQLite, se isso fizer mais sentido para o andamento do exercício.

O importante é que essa mudança seja feita conscientemente e refletida nos artefatos do projeto, em vez de deixar a implementação divergir silenciosamente das decisões atuais.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Referência para a Factory do modelo” no fluxo apresentado?
- Que decisão de engenharia está associada a “Referência para as tools do agente” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Referência para a Factory do modelo.
- Referência para as tools do agente.
- A descrição da ferramenta também orienta o modelo.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 5 — Definindo o Padrão Plan-and-Execute Para o Agente

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Compreender um estado compartilhado entre os nós.
- Compreender o planner e o plano estruturado.
- Compreender executor e replanner.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Um estado compartilhado entre os nós
- O planner e o plano estruturado
- Executor e replanner
- Um teto de oito passos
- Orientando o Spec Kit para implementar
- Revisando a implementação gerada

Conteúdo da aula

Nesta aula, eu continuo a implementação das estratégias de raciocínio do OpsPilot. Na etapa anterior, usei alguns códigos de referência para orientar o agente na criação da Factory do modelo, das tools e principalmente da estratégia ReAct. O Spec Kit chegou a produzir artefatos relacionados a Plan-and-Execute, mas, ao revisar o projeto, percebi que essa estratégia ainda não estava implementada da forma que eu queria.

Por isso, antes de simplesmente aceitar o resultado gerado, eu volto ao editor e preparo uma referência específica para o Plan-and-Execute. Aqui existe uma diferença importante em relação ao ReAct. Para o ReAct, consigo aproveitar uma construção mais pronta do ecossistema que estamos utilizando. Para Plan-and-Execute, quero montar o grafo explicitamente, deixando visíveis os componentes que estudamos na parte teórica.

A ideia é fazer com que cada nó do grafo corresponda a uma responsabilidade clara. Teremos um estado compartilhado, um planner responsável por criar o plano, um executor que realiza um passo por vez e um replanner que decide se o plano continua válido, precisa ser ajustado ou pode ser encerrado.

Um estado compartilhado entre os nós

O primeiro elemento que preparo como referência é o estado do Plan-and-Execute. Eu preciso de uma estrutura que funcione como um documento compartilhado entre os diferentes nós do grafo. Cada etapa lê esse estado e pode acrescentar novas informações para a próxima decisão.

Esse estado começa com a entrada original do usuário. Também mantém o plano de execução, o histórico dos passos que já foram concluídos e a resposta produzida ao final. Durante a implementação gerada pelo agente, essa estrutura é ampliada com informações como trace, número de iterações e quantidade de chamadas ao LLM, porque essas métricas já fazem parte do contrato geral das estratégias do OpsPilot.

Uma parte importante do estado é a lista de passos concluídos. Eu não quero que um nó simplesmente sobrescreva o que aconteceu anteriormente. Por isso, a referência utiliza um comportamento de acumulação. A cada passo executado, uma nova evidência é acrescentada ao histórico.

Esse acumulador é relevante para o replanner. Quando ele precisar decidir se deve alterar o plano, continuar ou finalizar, poderá consultar não apenas o pedido original, mas também aquilo que já foi efetivamente executado. O estado, portanto, representa a memória operacional daquela execução.

O planner e o plano estruturado

Depois do estado, preparo a referência do planner. A responsabilidade dele é receber a entrada e transformar o objetivo em uma sequência de passos curtos, ordenados e executáveis com as ferramentas disponíveis no sistema.

Eu não quero que o planner devolva um texto livre que depois precise ser interpretado manualmente pela aplicação. Em vez disso, defino um schema com Zod para representar o plano. Esse schema exige uma lista de passos em formato conhecido e validado.

A descrição associada ao campo dos passos também orienta o modelo. Ela informa que os passos precisam ser curtos, ordenados e executáveis com as tools disponíveis. Assim, o próprio contrato de saída ajuda a reduzir planos vagos ou impossíveis de executar.

Para gerar esse plano, utilizo a capacidade de saída estruturada do modelo. A chamada recebe uma mensagem de sistema com as instruções do planner e a entrada do usuário. O resultado já precisa obedecer ao schema definido.

Esse ponto é importante porque aplicamos novamente a mesma lógica de validação de fronteira que já usamos em outras partes do projeto. Antes, utilizávamos Zod quando dados externos entravam pelo HTTP, pela CLI ou por algum arquivo. Agora a fronteira é o próprio modelo.

O LLM continua sendo um componente externo do ponto de vista da aplicação. Eu não quero assumir que qualquer texto retornado pode ser utilizado como um plano válido. A saída estruturada transforma a resposta do modelo em um dado que a aplicação consegue validar e tratar de forma previsível.

Executor e replanner

Depois que o plano existe, entra o executor. A regra aqui é simples: ele trabalha em um passo por vez. Em vez de receber a lista inteira e executar tudo de uma vez, ele pega a próxima unidade de trabalho, utiliza as ferramentas disponíveis e registra o resultado no estado compartilhado.

Quando um passo é concluído, essa evidência é adicionada ao histórico de itens realizados. A partir daí, o replanner consegue olhar para o plano restante e para o que já aconteceu.

O replanner é a peça que evita que o Plan-and-Execute vire apenas uma sequência rígida produzida no começo da execução. Depois de cada passo, ele pode decidir entre três comportamentos principais: ajustar o plano, continuar com aquilo que já está definido ou encerrar.

Essa revisão constante é necessária porque o plano inicial pode deixar de representar a realidade. Uma ferramenta pode retornar uma informação diferente da esperada, uma ação pode revelar uma restrição nova ou parte do objetivo pode já ter sido atendida.

Sem o replanner, o agente correria o risco de continuar obedecendo a um plano envelhecido apenas porque ele foi criado no início. Com esse nó adicional, o fluxo mantém a visão global do Plan-and-Execute, mas continua capaz de reagir ao que acontece durante a execução.

Um teto de oito passos

Assim como fizemos com o ReAct, eu não quero que essa estratégia tenha liberdade para executar indefinidamente. O Plan-and-Execute também precisa de um guardrail de parada.

Na referência desta aula, estabeleço um teto total de oito passos. Esse limite faz parte do comportamento esperado da estratégia. Se o fluxo continuar encontrando motivos para replanejar, existe um ponto em que a aplicação precisa interromper o ciclo.

Esse tipo de limite é ainda mais importante quando o planner e o replanner utilizam chamadas de LLM. Cada revisão do plano representa custo, latência e mais contexto sendo processado. Portanto, limitar o número de passos não é apenas uma proteção contra loop infinito. Também é uma forma de controlar o consumo da estratégia.

Orientando o Spec Kit para implementar

Com as referências preparadas, volto ao agente de código e executo novamente a etapa de implementação do Spec Kit. Informo que quero o Plan-and-Execute voltado para o modelo acessado pelo OpenRouter e forneço os exemplos como orientação.

Também deixo explícito o comportamento esperado do executor e do replanner. O executor utiliza as tools e envia os resultados para o conjunto de passos concluídos. O replanner observa esse progresso e decide se precisa ajustar, seguir ou terminar.

Além disso, reforço o teto de oito passos e peço a execução do script de seed. Para esta fase do projeto, decido trabalhar apenas com o arquivo JSON e os dados de seed como base para o model layer.

Essa decisão simplifica a execução enquanto validamos as estratégias. Embora o projeto tenha discutido uma persistência relacional nas etapas anteriores, aqui o objetivo principal é observar o comportamento do agente. Posso evoluir a persistência depois sem misturar esse problema com a validação do raciocínio.

Revisando a implementação gerada

Depois da execução do Spec Kit, volto aos arquivos para revisar o que foi produzido. O agente utiliza os componentes de mensagens e ferramentas do ecossistema de LangChain e LangGraph e cria a estrutura necessária para o Plan-and-Execute.

O schema do plano aparece de acordo com a referência que forneci. Também é criado um schema específico para o replanejamento, representando decisões como ajustar, continuar ou finalizar.

O estado gerado contém a entrada, o plano, os itens concluídos e a resposta, além de informações adicionais importantes para observabilidade, como trace, número de iterações e chamadas de LLM.

Também aparecem funções auxiliares para formatar o plano e transformar os passos concluídos em texto quando necessário. A estratégia é exportada seguindo o mesmo contrato comum utilizado pelo restante do OpsPilot.

Esse ponto confirma uma decisão arquitetural importante. ReAct e Plan-and-Execute possuem implementações internas muito diferentes, mas o consumidor não precisa conhecer essa diferença. As duas estratégias podem devolver resposta, trace e métricas utilizando a mesma interface.

Validando primeiro o ReAct na arena

Antes de testar o novo Plan-and-Execute, eu aproveito para executar a estratégia ReAct na arena, algo que ainda não tínhamos validado diretamente.

A pergunta utilizada é sobre quantos alertas críticos estão disparando. Ao iniciar a execução, encontro primeiro um erro de configuração: uma das variáveis de ambiente obrigatórias não foi carregada.

A variável existe no projeto e está correta. O problema é que estamos utilizando o suporte nativo do Node para carregar o arquivo de ambiente, e o script da arena não estava configurado para fazer esse carregamento.

Peço ao próprio agente de código para analisar o comando e corrigir a configuração. O ajuste é feito no script correspondente, indicando ao Node qual arquivo de ambiente deve ser carregado durante a execução da arena.

Depois dessa correção, executo exatamente a mesma pergunta novamente. O ReAct começa a produzir o trace esperado. Ele decide listar os alertas, executa a ação e observa o retorno da store.

A observação mostra três alertas em estado de firing, exatamente como havíamos definido nos dados mock. A partir desses alertas, a estratégia identifica que dois são críticos.

A resposta final informa corretamente que existem dois alertas críticos disparando. Também consigo observar as métricas: foram feitas duas chamadas ao LLM e a latência ficou em torno de sete segundos, aproximadamente sete mil milissegundos naquele teste.

Esse resultado é importante porque agora consigo enxergar no OpsPilot o mesmo conceito de trace que observamos no agente de código. Só que, desta vez, o registro pertence ao agente que estamos construindo e faz parte do nosso próprio sistema.

Executando o Plan-and-Execute

Com o ReAct validado, parto para um cenário mais adequado ao Plan-and-Execute. Peço que o agente abra três incidentes de severidade dois para alertas relacionados a checkout, payments e catálogo, nessa ordem, e depois resolva o primeiro incidente.

Na primeira tentativa, digito o identificador da estratégia de forma incorreta. A arena identifica que aquele nome não existe e apresenta as estratégias disponíveis. Esse comportamento também é útil porque evita que uma simples digitação incorreta gere uma falha pouco explicativa.

Corrijo o nome e executo novamente.

Como estou utilizando a seleção gratuita do OpenRouter, a resposta pode demorar mais. A latência depende bastante dos modelos que estão disponíveis no gateway naquele momento. Uma opção automática paga poderia responder mais rápido, mas também introduziria custo. Para os exercícios, continuo utilizando a configuração gratuita.

O trace do Plan-and-Execute é significativamente maior do que o trace observado no ReAct. Primeiro, o planner cria uma sequência de passos. Entre eles estão listar os alertas necessários e abrir os incidentes solicitados para os serviços correspondentes.

Depois aparece a crítica do plano. Em um dos momentos, o agente percebe que a operação de listagem precisa de um status de alerta definido. Essa observação entra no processo e o fluxo utiliza o estado de firing para continuar.

A estratégia consulta os alertas, encontra os três que estão disparando e registra essas evidências. Em seguida, o replanner avalia o resultado e conclui que o plano continua válido.

Com isso, a execução prossegue para a abertura dos incidentes. Após cada etapa, novas informações são registradas e o plano pode ser novamente revisado.

Esse comportamento deixa clara a diferença para uma execução totalmente linear. O plano existe desde o início, mas não é tratado como uma ordem imutável. O agente revisita a estratégia ao longo da execução e pode decidir se precisa de alguma alteração.

Custo e latência do planejamento

Ao final do cenário, o Plan-and-Execute chega ao teto de oito passos definido para a execução. O trace mostra o progresso realizado e as decisões tomadas durante o planejamento e o replanejamento.

O resultado também evidencia uma característica importante desse padrão: ele pode custar bastante mais.

Enquanto a consulta simples com ReAct utilizou duas chamadas ao LLM, o cenário de Plan-and-Execute chegou a aproximadamente vinte chamadas.

A latência também foi muito maior. Em vez de alguns segundos, a execução passou de um minuto naquele teste.

Isso não significa que o Plan-and-Execute seja uma estratégia ruim. O cenário executado também é mais complexo e envolve planejamento, múltiplas ações e revisões sucessivas. O que essa medição mostra é que a visão global e o replanejamento possuem um preço.

É exatamente por isso que quero manter as métricas dentro do contrato das estratégias. Se eu analisasse apenas a resposta final, poderia dizer que as duas estratégias funcionaram. Quando coloco chamadas e latência ao lado da qualidade do resultado, começo a enxergar qual abordagem faz mais sentido para cada tipo de problema.

A próxima comparação poderá colocar ReAct e Plan-and-Execute diante do mesmo objetivo para medir eficiência em condições equivalentes. A arena passa a ser o lugar em que conseguimos observar essas diferenças de forma concreta, olhando para resposta, trace, número de chamadas e tempo de execução em vez de depender apenas de uma impressão subjetiva.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Um estado compartilhado entre os nós” no fluxo apresentado?
- Que decisão de engenharia está associada a “O planner e o plano estruturado” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Um estado compartilhado entre os nós.
- O planner e o plano estruturado.
- Executor e replanner.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 6 — Críticas e Benchmark com Padrão Reflection

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Especificar a camada de Reflection.
- Compreender critérios de parada da reflexão.
- Compreender reflection disponível na arena.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Especificando a camada de Reflection
- Critérios de parada da reflexão
- Reflection disponível na arena
- Executando Specify, Plan e Tasks
- Trocando a janela de agentes sem perder o processo
- Referência para o crítico

Conteúdo da aula

Nesta aula, eu fecho os três padrões de raciocínio que estamos estudando no OpsPilot. Já implementamos ReAct e Plan-and-Execute, e agora falta trabalhar com Reflection. A diferença é que Reflection não vai entrar no sistema como uma terceira estratégia independente. Em vez disso, eu quero criar uma camada que possa ser aplicada sobre qualquer estratégia já existente.

Essa decisão muda a forma como eu modelo o problema. ReAct e Plan-and-Execute implementam o mesmo contrato de estratégia de raciocínio. Reflection, por outro lado, recebe uma dessas estratégias, executa o comportamento original e acrescenta uma etapa de crítica antes de devolver o resultado. Se a crítica reprovar a resposta, a camada pode pedir uma nova geração utilizando o feedback produzido pelo crítico.

Em termos de design de software, isso se aproxima do padrão Decorator. Eu preservo o mesmo contrato externo, mas adiciono comportamento no meio do caminho. Dessa forma, posso ter ReAct puro, Plan-and-Execute puro, ReAct com Reflection ou Plan-and-Execute com Reflection sem alterar os consumidores das estratégias.

Especificando a camada de Reflection

Eu começo novamente pelo fluxo do Spec Kit. Em vez de partir diretamente para a implementação, crio uma nova especificação para a camada de Reflection.

A ideia central é ter uma função que recebe qualquer Reasoning Strategy e devolve outra estratégia compatível com o mesmo contrato. Essa nova estratégia primeiro executa a base e, depois, chama um crítico.

O crítico utiliza o próprio modelo e devolve uma saída estruturada. Eu quero duas informações principais: se a resposta foi aprovada e qual feedback deve ser utilizado quando houver algo para corrigir.

Essa avaliação não deve olhar apenas para o texto final. O crítico precisa considerar a resposta em relação às observações registradas no trace. Assim, ele consegue verificar se aquilo que está sendo afirmado é compatível com as evidências obtidas durante a execução.

Se a crítica aprovar o resultado, a execução pode terminar. Se reprovar, eu quero regenerar utilizando o feedback, o contexto disponível e as informações que já foram produzidas pela estratégia base.

Critérios de parada da reflexão

Assim como ReAct e Plan-and-Execute possuem limites, Reflection também precisa de um teto. Um crítico sempre consegue encontrar alguma possibilidade de melhoria. Se eu permitir que ele revise indefinidamente, posso criar um ciclo caro e desnecessário.

Por isso, a camada recebe uma configuração de número máximo de reflexões. O valor padrão definido para esta etapa é dois.

A execução deve parar em uma de duas condições: quando o crítico aprovar a resposta ou quando o número máximo de reflexões for atingido.

Também quero registrar a crítica no próprio trace. Isso significa que o histórico da execução passa a mostrar não apenas pensamentos, ações e observações da estratégia base, mas também a avaliação realizada pelo Reflection.

As métricas precisam incluir as chamadas extras de LLM. Se eu adicionar uma etapa de crítica, não quero esconder esse custo. A comparação entre uma estratégia pura e a mesma estratégia decorada só faz sentido se o número total de chamadas e a latência refletirem todo o processamento realizado.

Reflection disponível na arena

A arena também precisa entender as estratégias decoradas. Quero conseguir executar, por exemplo, ReAct puro e ReAct com Reflection na mesma ferramenta.

O mesmo vale para Plan-and-Execute. A identificação utilizada na arena precisa deixar claro quando estou rodando apenas a estratégia base e quando estou adicionando a camada de crítica.

Com isso, a arena deixa de ser apenas um lugar para comparar dois padrões principais. Ela também passa a permitir que eu meça o impacto de adicionar revisão ao resultado.

Executando Specify, Plan e Tasks

Depois de descrever os requisitos, executo as fases de specify, plan e tasks do Spec Kit.

O fluxo continua igual ao que já construímos. A especificação registra o comportamento esperado, o planejamento decide como a camada será estruturada e as tarefas dividem o trabalho em unidades que podem ser implementadas e revisadas.

Antes de executar o implement, eu paro novamente para preparar uma referência de código. Essa prática já apareceu nas estratégias anteriores. O objetivo não é escrever a solução inteira manualmente, mas fornecer uma forma concreta do padrão que eu quero ver no código final.

Trocando a janela de agentes sem perder o processo

Durante essa etapa acontece uma situação prática interessante. Eu esgoto os tokens disponíveis na janela do Copilot e decido continuar utilizando o Cursor.

Essa troca não impede o andamento do projeto. O Cursor possui uma experiência de agente bastante semelhante e consegue trabalhar sobre os mesmos arquivos que já foram produzidos pelo Spec Kit.

Essa é uma demonstração importante do motivo pelo qual colocamos especificações, planejamento, tarefas, Constitution e instructions no repositório. O contexto principal do desenvolvimento não está preso apenas a uma conversa específica.

Ao abrir outra janela de agente, consigo continuar a partir da task de Reflection porque os artefatos do SDD já estão persistidos. O novo agente pode ler aquilo que foi produzido anteriormente e prosseguir.

Na prática, isso mostra que o nosso método de trabalho é mais importante do que a interface específica utilizada para conversar com o modelo.

Referência para o crítico

Antes da implementação, preparo uma referência para o crítico.

Quero um schema de veredito validado com Zod. Esse schema contém um indicador booleano de aprovação e uma string de feedback.

O feedback precisa ser específico e acionável. Se a resposta estiver correta, a crítica deve deixar isso claro. Se houver problema, quero saber o que exatamente precisa ser corrigido.

Também preparo uma função de crítica que recebe a entrada original e o resultado da estratégia.

Essa função utiliza saída estruturada para garantir que o retorno do modelo respeite o schema de veredito. Mais uma vez, a saída do LLM é tratada como uma fronteira que precisa de validação.

O prompt do crítico orienta que a avaliação considere as observações presentes no trace e o pedido original. Na mensagem enviada ao modelo, são reunidos o pedido, as observações relevantes e a resposta que está sendo avaliada.

Com essa referência pronta, volto à janela do agente e executo a etapa de implementação do Spec Kit, pedindo que o exemplo seja levado em conta.

Testando ReAct e Reflect-ReAct

Depois da implementação, volto à arena para verificar o comportamento da camada.

A pergunta utilizada pede um resumo do estado do plantão com uma recomendação do que deve ser atacado primeiro.

Executo duas variações: ReAct e ReAct com Reflection.

No ReAct puro, a estratégia consulta os alertas e identifica corretamente os problemas mais relevantes. Entre os alertas encontrados, a API de pagamentos aparece como prioridade principal por estar associada a uma condição crítica e a impacto direto em transações.

A resposta também organiza outras prioridades e sugere próximos passos. Naquela execução, o ReAct utilizou duas chamadas ao LLM e levou aproximadamente trinta e cinco segundos.

Em seguida, observo a execução com Reflection. A estratégia base produz seu resultado e a camada de crítica avalia se a resposta está adequada às observações.

Nesse caso, o resultado já estava suficientemente bom na primeira avaliação. Por isso, não foi necessário passar por várias rodadas de regeneração.

Ainda assim, existe uma chamada adicional ao LLM para o crítico. A execução decorada leva mais tempo, aproximadamente quarenta e cinco segundos naquele teste.

Esse comportamento é exatamente o que eu esperava. Reflection melhora o controle de qualidade, mas adiciona custo. Se a primeira resposta já estiver correta, o ganho pode ser pequeno enquanto a latência e o número de chamadas aumentam.

Criando o benchmark

Depois de validar Reflection na arena, eu quero sair da comparação pontual e criar uma bateria fixa de avaliação.

Desde as primeiras aulas do OpsPilot eu mencionei a ideia de um bench. Agora chegou o momento de implementá-lo.

Peço ao agente para criar o script de benchmark com três cenários e duas estratégias principais: ReAct e Plan-and-Execute.

O resultado de cada execução deve ser comparado com o estado da store depois que a estratégia terminar. Dessa forma, o critério de acerto não depende apenas do texto produzido. Para cenários que modificam o sistema, eu quero verificar se o estado final realmente corresponde ao pedido.

A saída precisa ser consolidada em uma tabela com cenário, estratégia, acerto, chamadas de LLM e latência em milissegundos.

Também prevejo flags para selecionar um cenário e controlar aspectos relacionados ao replanner, permitindo executar avaliações mais específicas quando necessário.

Os três cenários do bench

O primeiro cenário é direto. A pergunta é quantos alertas críticos estão disparando.

Esse caso exige basicamente consulta e interpretação. Ele se aproxima do tipo de tarefa em que ReAct já mostrou boa eficiência.

O segundo cenário é estruturado. O agente deve abrir três incidentes de severidade dois para checkout, payments e catálogo, respeitando essa ordem, e resolver o primeiro incidente.

Aqui existe uma sequência de ações e dependências que tornam o problema mais interessante para Plan-and-Execute.

O terceiro cenário é dinâmico. A estratégia precisa utilizar os alertas que estão disparando, abrir um incidente para o mais antigo e informar quantos restaram.

Esse caso combina observação do estado com uma ação que altera o próprio ambiente, criando uma situação em que o agente precisa se adaptar ao resultado das operações.

Primeira execução e correção de erros

Depois que o script é criado, executo o benchmark completo.

Na primeira tentativa, encontro erros associados ao Plan-and-Execute. Dependendo de como o agente de código implementou essa estratégia, o mesmo problema pode ou não aparecer em outra execução do curso.

Mesmo com a falha, já consigo observar um dado interessante no cenário que roda corretamente para as duas estratégias. O ReAct resolve o primeiro caso com duas chamadas ao LLM, enquanto o Plan-and-Execute precisa de sete.

Como quero comparar as estratégias em condições válidas, corrijo os problemas antes de aceitar o benchmark como referência. A orientação prática é usar o próprio erro junto ao comando executado e pedir ao agente de código que identifique e ajuste a implementação.

Depois das correções, executo novamente a bateria.

Resultados do benchmark

Na segunda execução, a diferença entre as estratégias fica mais clara.

O ReAct utiliza menos chamadas ao LLM em todos os cenários avaliados. Em alguns casos, o Plan-and-Execute chega a consumir perto do dobro ou mais das chamadas necessárias pelo ReAct.

A latência também favorece o ReAct na bateria executada nesta aula.

O ponto mais importante, porém, está no critério de acerto. O ReAct atende corretamente os cenários utilizados no bench. O Plan-and-Execute apresenta problemas nos cenários dois e três porque o estado final da store não corresponde ao comportamento esperado.

Isso mostra por que eu não quero avaliar agentes apenas observando se a resposta textual parece convincente. Uma estratégia pode explicar muito bem o que pretendia fazer e ainda assim deixar o sistema em um estado incorreto.

Ao validar o estado real da store, o benchmark consegue detectar esse tipo de falha.

Para essa bateria específica, o ReAct apresenta o melhor resultado: faz menos chamadas, responde mais rápido e atende aos critérios de acerto definidos.

Isso não transforma ReAct em uma estratégia universalmente superior. O resultado pertence aos cenários, à implementação e às ferramentas que construímos neste projeto. O valor do benchmark está justamente em substituir uma preferência abstrata por evidências mensuráveis.

A partir daqui, existe um exercício natural: ajustar o Plan-and-Execute para que ele consiga produzir o estado correto nos cenários em que falhou e rodar o benchmark novamente. Assim, consigo observar se a melhoria de acerto vem acompanhada de mudanças no número de chamadas e na latência.

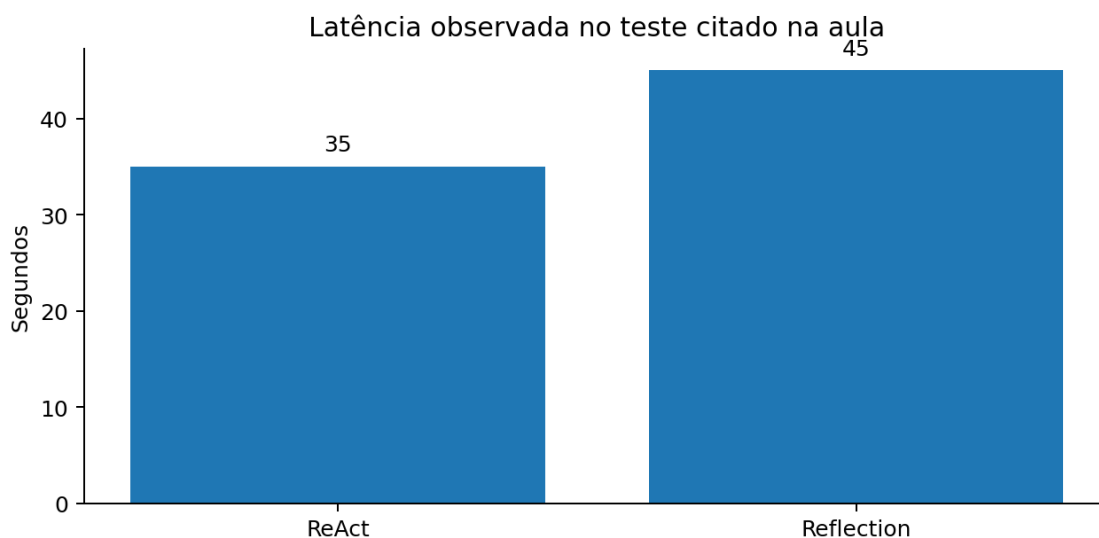
Com essa base de comparação pronta, o próximo passo do projeto será expor o comportamento do OpsPilot por uma API, mantendo os critérios de trace, métricas e avaliação que já estruturamos nesta unidade.

O que o benchmark revela sobre o projeto

Esse benchmark também começa a transformar o OpsPilot em um sistema observável do ponto de vista do raciocínio. Eu não estou medindo apenas o resultado final. Passo a relacionar qualidade, quantidade de chamadas, latência e efeito real sobre o estado da aplicação. Esses quatro elementos precisam ser analisados em conjunto. Uma resposta pode parecer correta e ainda assim não ter executado as ações necessárias. Da mesma forma, uma estratégia pode acertar, mas consumir chamadas demais para uma tarefa simples.

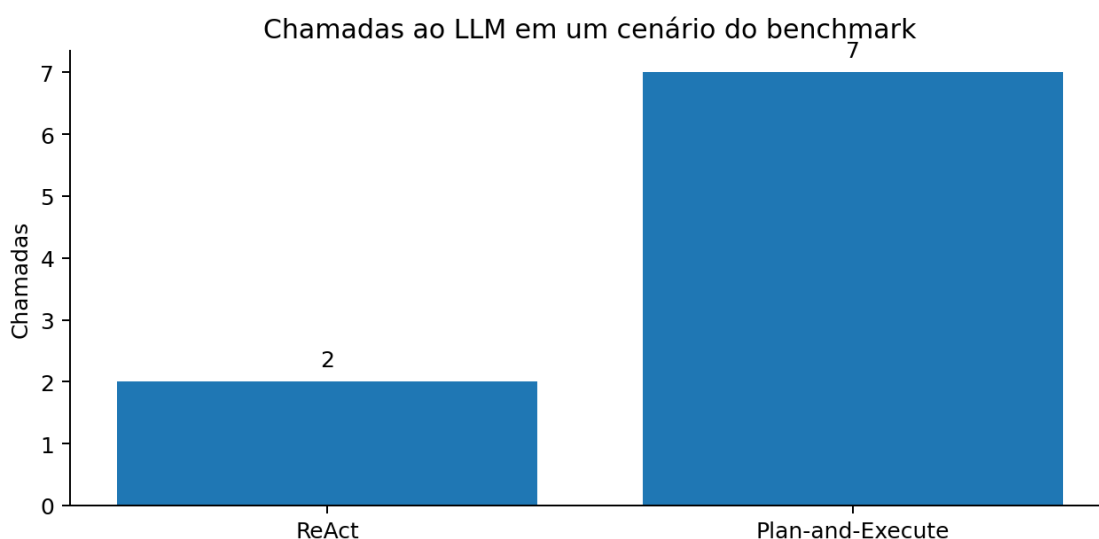
Essa leitura também ajuda a decidir quando vale a pena adicionar Reflection. Se uma estratégia já apresenta alta taxa de acerto em um cenário direto, uma crítica adicional pode aumentar custo e latência sem trazer benefício proporcional. Em tarefas mais sensíveis, porém, essa chamada extra pode ser justificável porque introduz uma verificação explícita antes de aceitar o resultado. O objetivo não é escolher um padrão para todas as situações, mas construir evidência suficiente para decidir qual composição de estratégia, crítica e limites de execução é adequada para cada classe de problema.

Leitura visual dos dados citados na aula



Valores registrados no material original para aquela execução específica.

ReAct foi citado com cerca de 35 s e a execução com Reflection com cerca de 45 s naquela demonstração. O valor é ilustrativo daquele teste, não um benchmark universal.



Exemplo pontual citado na primeira execução do benchmark, antes das correções.

Em um cenário da primeira execução do benchmark, o material registra 2 chamadas para ReAct e 7 para Plan-and-Execute. A implementação foi corrigida depois; o gráfico preserva o contexto da medição.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Especificando a camada de Reflection” no fluxo apresentado?
- Que decisão de engenharia está associada a “Critérios de parada da reflexão” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Especificando a camada de Reflection.
- Critérios de parada da reflexão.
- Reflection disponível na arena.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 7 — Uma API que Também é um Agente

Unidade 2 • Padrões de Raciocínio e Execução

OBJETIVOS DE APRENDIZAGEM

- Compreender o que já existe no OpsPilot.
- Compreender a API precisa representar o agente.
- Especificar o endpoint de chat.

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- O que já existe no OpsPilot
- A API precisa representar o agente
- Especificando o endpoint de chat
- Contrato de resposta e erros HTTP
- Registry de estratégias
- Teste de integração sem rede

Conteúdo da aula

Nesta aula, eu quero fechar a unidade transformando o que já construímos em uma API que também funciona como um agente. Antes disso, faço um alerta importante sobre o benchmark. Como estamos utilizando o OpenRouter na modalidade gratuita, existe um limite de requisições que pode ser atingido se eu executar a bateria muitas vezes. Mesmo com limites de interação configurados nas estratégias, o benchmark compara cenários e padrões diferentes, o que naturalmente aumenta bastante o número de chamadas ao modelo.

Por isso, eu evito rodar o benchmark repetidamente sem necessidade. A ferramenta existe para medir comportamento, custo e latência, mas precisa ser utilizada com consciência, principalmente quando estamos trabalhando com modelos gratuitos ou planos com limite diário de requisições.

Também vale recapitular o caminho até aqui. Na unidade 1, nós construímos nosso próprio fluxo de Spec-Driven Development e aprendemos a trabalhar com especificação, planejamento, tarefas e implementação. Depois percebemos que vários comportamentos do agente de código já correspondiam aos padrões ReAct, Plan-and-Execute e Reflection. Na unidade 2, passamos a implementar esses mesmos conceitos sobre uma LLM externa, acessada pelo OpenRouter.

O que já existe no OpsPilot

Neste ponto, o OpsPilot já possui as estratégias principais e seus guardrails. Temos ReAct, Plan-and-Execute e a camada de Reflection. Também construímos a arena para observar traces e métricas e o benchmark para comparar as estratégias em cenários fixos.

Essas implementações já estão materializadas nos arquivos do projeto. Posso abrir a estratégia Plan-and-Execute e revisar como ela extrai ações, mantém o estado, executa passos e utiliza o replanner para decidir o que acontece depois. Toda a lógica de planejamento que fomos discutindo está ali, organizada dentro do fluxo que o agente de código gerou a partir da nossa especificação.

O ReAct segue uma abordagem diferente. Neste caso, aproveitamos o suporte de LangChain e LangGraph para criar um agente ReAct pré-construído. Nós fornecemos o papel, o contexto, a mensagem de entrada, as tools disponíveis e o limite máximo de recursões. A biblioteca cuida de boa parte da mecânica necessária para intercalar raciocínio, ação e observação.

Isso não significa que eu deixe de revisar o que foi gerado. Em alguns pontos do Plan-and-Execute, por exemplo, é possível perceber ligações que poderiam ser ajustadas. Esse tipo de diferença pode variar conforme o agente de código e o modelo utilizados. A estrutura produzida pelo Copilot pode não ser idêntica àquela

produzida pelo Cursor ou por outro agente. O importante é que eu consiga entender o desenho e fazer correções quando necessário.

A API precisa representar o agente

O que falta agora é uma interface HTTP para o OpsPilot. Eu não quero criar apenas uma API tradicional que execute operações estáticas. Quero que a própria API represente o agente.

A ideia é ter um endpoint de chat. O plantonista envia uma mensagem, informa qual estratégia deseja utilizar e pode decidir se a execução deve ou não passar por Reflection. A API escolhe a estratégia adequada, executa o agente e devolve não apenas a resposta, mas também o trace e as métricas.

Esse desenho conecta tudo o que fizemos anteriormente. A mensagem recebida pela API vira a entrada do Reasoning Strategy. O registry escolhe ReAct ou Plan-and-Execute. Se Reflection estiver habilitado, aplicamos o decorator sobre a estratégia escolhida. O resultado final mantém o mesmo contrato de resposta que já utilizamos na arena.

Com isso, a lógica de raciocínio não fica acoplada ao terminal. A arena continua útil para experimentação e comparação, mas agora qualquer cliente HTTP pode conversar com o agente.

Especificando o endpoint de chat

Como já fizemos ao longo de toda a unidade, eu começo essa etapa pelo Spec Kit. Primeiro executo a especificação da nova feature. O objetivo é criar uma rota POST para /chat dentro da camada HTTP do projeto, utilizando Express.

Se eu quisesse trabalhar apenas com as APIs nativas do Node, poderia orientar a especificação para usar node:http. Neste projeto, porém, a stack já foi definida com Express, então mantenho essa decisão.

O corpo da requisição precisa conter a mensagem do usuário. Também pode informar a estratégia e se Reflection deverá ser aplicada.

Esses dados de entrada devem ser validados com Zod. Eu continuo tratando HTTP como uma fronteira externa. Antes de transformar qualquer valor recebido em decisão de aplicação, quero garantir que ele esteja dentro do contrato previsto.

A estratégia padrão será ReAct. Isso significa que um cliente pode enviar apenas a mensagem e ainda assim obter uma execução válida. Se quiser, ele pode selecionar explicitamente Plan-and-Execute ou habilitar Reflection.

Contrato de resposta e erros HTTP

Para uma execução bem-sucedida, quero devolver uma resposta HTTP 200 contendo o answer, o trace e as métricas.

Esse contrato é importante porque a API não deve esconder como a estratégia se comportou. O answer representa o texto final destinado ao usuário. O trace permite auditoria do fluxo. As métricas mostram custo operacional, quantidade de chamadas e latência.

Também especifico respostas de erro claras. Se o body for inválido, a API deve retornar status 400 com as issues produzidas pela validação do Zod. Assim, o consumidor consegue saber exatamente qual parte da entrada está incorreta.

Se a estratégia solicitada não existir, quero uma resposta 422. Esse erro representa um pedido semanticamente compreensível, mas que não pode ser executado porque o identificador da estratégia não faz parte do registry.

Também preciso de um timeout. Inicialmente penso em um limite de sessenta segundos, mas como estamos utilizando o OpenRouter gratuito e já observamos chamadas que podem demorar bastante, aumento esse teto para cento e oitenta segundos. Se a execução ultrapassar esse tempo, a API deverá responder com status 504.

Essa decisão é prática. Um timeout curto demais poderia gerar falhas artificiais apenas porque o modelo gratuito demorou para responder. Ao mesmo tempo, não quero deixar uma requisição pendurada indefinidamente.

Registry de estratégias

Para resolver a seleção dinâmica, quero um registry centralizado na área de agents.

Esse registry associa nomes às estratégias disponíveis. A rota HTTP não precisa conhecer todos os detalhes internos de ReAct ou Plan-and-Execute. Ela apenas consulta o registry e obtém a estratégia correspondente ao identificador recebido.

Quando Reflection estiver habilitado, aplico a camada withReflection sobre a estratégia selecionada. Como Reflection preserva o mesmo contrato de Reasoning Strategy, o restante do fluxo continua funcionando sem precisar de tratamento especial.

Esse é exatamente o benefício do desenho que construímos anteriormente. Strategy nos permite trocar o mecanismo de raciocínio. Decorator nos permite acrescentar Reflection. A API trabalha apenas com a abstração comum.

Teste de integração sem rede

Outro requisito importante é o teste de integração da API.

Eu não quero que esse teste dependa de uma chamada real ao OpenRouter. Se toda execução de CI precisar chamar um LLM externo, os testes ficam sujeitos a latência, limite de requisições, indisponibilidade e respostas não determinísticas.

Por isso, especifico uma estratégia fake e determinística para os testes da camada HTTP. Ela implementa o mesmo contrato das estratégias reais, mas responde de forma previsível e sem rede.

Com isso, consigo testar validação de body, seleção de estratégia, formato de resposta, status HTTP e integração entre rota e serviço sem depender do comportamento de um modelo.

As chamadas reais ao LLM continuam pertencendo à arena e ao benchmark. É nesses ambientes que quero avaliar raciocínio, custo e qualidade do modelo. No CI, priorizo testes rápidos, repetíveis e determinísticos.

Executando Plan, Tasks e Implement

Depois de gerar a especificação, avanço para as etapas de plan e tasks. Essa parte segue exatamente o fluxo que já estamos utilizando, então posso deixá-la rodando enquanto reviso os artefatos produzidos.

Na implementação não existe nenhuma mudança conceitual tão diferente quanto tivemos ao criar ReAct ou Plan-and-Execute. Por isso, não preciso preparar uma referência extensa de código desta vez. O agente já possui a stack, as conventions, a Constitution e os requisitos HTTP suficientemente claros.

Mesmo assim, depois da implementação eu continuo revisando o que foi criado. O fato de a feature ser mais convencional não elimina a necessidade de verificar rotas, validações, registry e tratamento de erros.

Carregando corretamente as variáveis de ambiente

Ao executar a aplicação, encontro novamente um detalhe relacionado às variáveis de ambiente. O script de desenvolvimento não estava carregando o arquivo de ambiente da forma esperada.

Esse é o mesmo tipo de problema que apareceu anteriormente na arena. Como optamos por utilizar o suporte nativo do Node, o script precisa indicar corretamente o arquivo de ambiente durante a inicialização.

Faço esse ajuste no script de desenvolvimento e tento iniciar o servidor novamente.

Nesse momento aparece outro problema bastante comum: a porta 3000 já está em uso. Isso acontece porque algum processo anterior continuou executando. Eu identifico qual processo está ocupando a porta e encerro essa execução antes de subir o servidor de novo.

Depois disso, a aplicação inicia normalmente e passa a escutar na porta 3000.

Primeira validação da API

Com o servidor em execução, abro outro terminal para testar as rotas.

Primeiro faço uma chamada simples para verificar os alertas ativos. A API retorna os alertas que estavam em estado ativo no conjunto de dados que já utilizamos nas aulas anteriores.

Essa chamada confirma que a aplicação está de pé e que a camada HTTP consegue acessar as informações esperadas.

O tempo de resposta pode variar conforme o modelo escolhido pelo OpenRouter. Como já discutimos, a seleção gratuita pode utilizar modelos com latências diferentes, então é normal que uma mesma operação demore mais ou menos em execuções distintas.

Chamando o agente pelo endpoint /chat

Depois parto para a chamada mais importante: o POST de chat.

A mensagem solicita que seja aberto um incidente de severidade dois para o catálogo. Também informo que quero utilizar a estratégia ReAct e habilito Reflection.

Escolho ReAct nesse teste porque ele tende a consumir menos chamadas em cenários como os que avaliamos anteriormente. Poderia utilizar Plan-and-Execute, mas não existe motivo para pagar o custo adicional se a tarefa não exige um planejamento mais elaborado.

Na primeira tentativa, a chamada apresenta problema porque a requisição foi montada incorretamente. Corrijo os detalhes do cabeçalho e do JSON enviado e executo novamente.

A API processa a mensagem, aciona o agente e responde que o incidente de severidade dois para o catálogo foi aberto com sucesso.

A resposta inclui o identificador do incidente, o título e o status aberto. Também retorna as métricas da execução. Naquele teste, foram necessárias três chamadas ao LLM.

Isso confirma que a API realmente está funcionando como porta de entrada para o agente. Não estamos apenas chamando uma função de CRUD. Estamos enviando intenção em linguagem natural, deixando uma estratégia de raciocínio para interpretar essa intenção, utilizar ferramentas e produzir o resultado.

Uma única lógica acessível por várias interfaces

Com o endpoint funcionando, o OpsPilot começa a ganhar a forma que queríamos desde o início da unidade.

Eu consigo interagir pelo terminal, utilizar a arena para observar e comparar raciocínios, executar o benchmark para medir as estratégias e agora também consumir o agente por HTTP.

Se eu preferir, não preciso fazer essas chamadas manualmente pelo terminal. Posso configurar Postman, Insomnia ou qualquer outro cliente HTTP. O protocolo é o mesmo.

O ponto principal é que todas essas interfaces convergem para o mesmo núcleo de agentes. As estratégias continuam centralizadas, as tools continuam sendo reutilizadas e a Reflection continua podendo ser aplicada sem duplicar lógica na camada HTTP.

Revisão técnica da unidade

Ao chegar aqui, eu consigo enxergar com mais clareza onde LangChain e LangGraph entraram no projeto. Em alguns momentos, como boa parte do código foi criada pelo agente a partir das especificações, pode parecer que não utilizamos diretamente essas bibliotecas.

Na prática, elas estão dentro das implementações geradas para ajudar a construir os agentes, organizar tools e modelar os fluxos de raciocínio. No ReAct, por exemplo, aproveitamos uma construção pronta para criar o agente. No Plan-and-Execute, usamos a estrutura de grafo para organizar planner, executor e replanner.

Tudo isso foi conduzido pelo nosso agente de código, que já estava configurado com SDD. Nós descrevemos o comportamento desejado, geramos especificação, planejamento e tarefas e fornecemos referências quando queríamos um padrão de implementação mais específico.

Esse processo também explica por que erros diferentes podem aparecer para pessoas diferentes. Eu posso utilizar Copilot em uma execução, Cursor em outra ou até modelos diferentes dentro da mesma ferramenta. Cada combinação pode tomar decisões de implementação ligeiramente diferentes.

Até mesmo o projeto preparado previamente para a aula pode apresentar erros diferentes do projeto construído durante a gravação. Isso faz parte do desenvolvimento com agentes. O objetivo não é esperar que todos produzam exatamente os mesmos arquivos, mas entender o conceito e conseguir revisar e corrigir o resultado.

Por isso, continuo reforçando a revisão humana. Mesmo quando não conseguimos revisar linha por linha durante a aula por causa do tamanho da aplicação, essa revisão deve fazer parte do trabalho real.

O que eu preciso levar desta unidade é o entendimento de quando ReAct faz sentido, quando Plan-and-Execute pode ser útil, porque Reflection acrescenta uma camada de crítica, por que preciso limitar interações e como traces, métricas, guardrails e testes determinísticos ajudam a tornar o agente observável e controlável.

NA PRÁTICA

Abra o snapshot da Unidade 2 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O que já existe no OpsPilot” no fluxo apresentado?
- Que decisão de engenharia está associada a “A API precisa representar o agente” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- O que já existe no OpsPilot.
- A API precisa representar o agente.
- Especificando o endpoint de chat.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 2

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE

Solicitação →	Estratégia →	ReAct / Plan-and-Execute →	Reflection →	Trace →	Benchmark
---------------	--------------	----------------------------	--------------	---------	-----------

CHECKLIST DA UNIDADE

- Revisar a Aula 1: Raciocínio e Tomada de Decisão em Agentes - Os Três Padrões de Raciocínio
- Revisar a Aula 2: Configurando o Spec Kit
- Revisar a Aula 3: Criando a Estrutura Inicial do Projeto com Spec Kit
- Revisar a Aula 4: Definindo o Padrão ReAct Para o Agente
- Revisar a Aula 5: Definindo o Padrão Plan-and-Execute Para o Agente
- Revisar a Aula 6: Críticas e Benchmark com Padrão Reflection
- Revisar a Aula 7: Uma API que Também é um Agente

GITHUB DA UNIDADE 2

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Unidade 3

Function Calling e Tool Use

Conexão do agente com persistência e ferramentas reais, incluindo SQLite, APIs externas e interoperabilidade via MCP.

FLUXO VISUAL DA UNIDADE

Agente →

Tools →

Store / SQLite →

APIs externas →

HTTP / MCP →

Validação

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Aula	Tema
1	Especificando a Integração com Banco de Dados
2	Criando e Consultando Incidentes no Banco com Nosso Agente
3	Validando Disponibilidade de Provedores Externos
4	Disponibilizando Nosso Agente Via MCP

Aula 1 — Especificando a Integração com Banco de Dados

Unidade 3 • Function Calling e Tool Use

OBJETIVOS DE APRENDIZAGEM

- Explicar por que trocar para SQLite.
- Compreender atualizando a Constitution antes da nova spec.
- Especificar uma persistência real para as operações.

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Por que trocar para SQLite
- Atualizando a Constitution antes da nova spec
- Especificando uma persistência real para as operações
- Arquivo de banco e uso de:memory:
- As tabelas que representam o domínio atual
- Checks e restrições do domínio no banco

Conteúdo da aula

Agora que a aplicação já consegue conversar pela API e executar as estratégias que construímos, eu preciso resolver uma limitação importante do projeto: até aqui, praticamente tudo está sendo mantido em memória. Isso foi suficiente para aprender os padrões de raciocínio, testar as tools, comparar estratégias e validar a API. A partir deste ponto, porém, quero começar a persistir as operações de verdade.

Essa mudança é relevante porque o estado passa a sobreviver às execuções do processo. Alertas, incidentes, serviços e runbooks deixam de depender apenas da memória da aplicação. Ao mesmo tempo, eu quero fazer essa evolução sem tirar o foco principal do módulo, que continua sendo o desenvolvimento de agentes e a forma como eles operam dentro de uma aplicação.

Por esse motivo, vou alterar uma decisão anterior relacionada ao banco de dados. Em versões anteriores da especificação, tínhamos considerado uma solução relacional tradicional associada a bibliotecas como Sequelize e a bancos como MySQL ou Postgres. Agora eu quero simplificar essa parte e utilizar SQLite.

Por que trocar para SQLite

A decisão por SQLite é principalmente uma decisão de compatibilidade e praticidade para o curso. Se eu mantivesse um banco externo, teria que preparar a infraestrutura necessária para executá-lo, possivelmente utilizando Docker, além de configurar corretamente o serviço, as conexões e todo o ambiente local.

Isso não seria errado. Em uma aplicação real, eu poderia perfeitamente optar por MySQL, Postgres ou outra tecnologia adequada ao contexto. O problema é que essa infraestrutura começaria a consumir tempo de aula em um ponto que não é o nosso objetivo principal.

Com o SQLite, eu consigo manter a ideia de persistência real e continuar estudando as decisões importantes de integração entre agentes, tools e estado da aplicação sem transformar a unidade em uma aula sobre administração de banco de dados.

Outro fator é que quero utilizar o suporte nativo disponível no Node para SQLite, evitando adicionar mais uma biblioteca apenas para essa camada. Assim, a estrutura fica menor e conseguimos enxergar melhor o que está acontecendo entre a aplicação e o banco.

Mais adiante, a migração para outro banco pode inclusive virar um exercício. Se eu já trabalho com outra tecnologia ou quero praticar algo específico para meu contexto profissional, posso trocar a implementação da store sem alterar o raciocínio principal que estamos construindo aqui.

Atualizando a Constitution antes da nova spec

Como essa mudança afeta uma decisão importante de arquitetura, eu não quero simplesmente pedir para o agente alterar alguns arquivos. O primeiro passo é atualizar a Constitution do projeto.

A Constitution precisa refletir o projeto atual. Se ela continuar dizendo que a persistência utiliza outra tecnologia, as próximas fases do Spec Kit podem ler uma regra antiga e tomar decisões incompatíveis com aquilo que eu realmente quero.

Por isso, antes da nova especificação, peço ao agente que atualize a Constitution para registrar o uso de SQLite no lugar do banco tradicional considerado anteriormente.

Essa sequência reforça o papel que demos aos artefatos do SDD. A Constitution não é um arquivo decorativo. Ela orienta as decisões das fases seguintes. Quando uma decisão arquitetural muda, esse contrato também precisa mudar.

Especificando uma persistência real para as operações

Com a Constitution atualizada, começo uma nova especificação no Spec Kit voltada para persistência real das operações.

Quero uma implementação específica da OpsStore baseada em SQLite e compatível com a interface que já existe no projeto. Isso é importante porque eu não quero reescrever os agentes para que eles passem a conhecer detalhes do banco.

As estratégias continuam trabalhando com o contrato da store. A nova implementação apenas troca o mecanismo concreto utilizado para persistir e consultar os dados.

Essa separação também preserva a possibilidade de manter a store em memória em cenários nos quais ela ainda é mais adequada, como testes e benchmark. A aplicação pode usar SQLite em execução normal, enquanto os testes continuam utilizando uma implementação descartável e previsível.

Arquivo de banco e uso de:memory:

Na especificação, defino um caminho padrão para o arquivo do banco de dados dentro de uma pasta de dados do projeto. Esse arquivo será utilizado nas execuções normais do OpsPilot.

Para os testes, porém, quero utilizar o modo:memory: do SQLite.

A motivação é simples. Eu não quero que uma suíte de testes escreva no banco utilizado pelo desenvolvedor e deixe dados residuais depois da execução. Quando uso um banco em memória, consigo criar a estrutura, executar o cenário e descartar tudo ao final sem sujar o estado local.

Isso também facilita a repetição dos testes. Cada execução pode começar de um estado conhecido, aplicar o seed e então verificar o comportamento esperado.

Existem diferentes filosofias para testes de banco de dados, mas, para esta etapa, o SQLite em memória nos dá uma solução simples e adequada ao tipo de validação que queremos fazer.

As tabelas que representam o domínio atual

A nova store terá quatro tabelas principais: services, alerts, incidents e runbooks.

Essas tabelas precisam espelhar os tipos de domínio que já existem na aplicação. Eu não quero que a persistência invente uma representação completamente diferente. O objetivo é armazenar o estado que os agentes e as tools já conhecem.

Na parte de incidents, acrescento também informações necessárias para acompanhar melhor o ciclo de vida. Um incidente deve conseguir registrar quando foi resolvido e também possuir um summary que pode ser nulo quando essa informação ainda não existe.

Essa modelagem começa a preparar o OpsPilot para manter histórico de operações de uma forma mais realista. Abrir e resolver um incidente deixa de ser apenas uma mudança temporária no objeto mantido em memória.

Checks e restrições do domínio no banco

Além da validação realizada pela aplicação, quero que a estrutura do banco também respeite valores aceitos pelo domínio.

Na especificação, peço checks para campos fechados, como tier, severidade e status. A ideia é não permitir que qualquer string seja gravada nesses campos apenas porque chegou até a camada de persistência.

Isso cria mais uma camada de proteção. Zod continua validando os dados no lado da aplicação, mas o banco também conhece algumas restrições fundamentais do modelo que está armazenando.

Essa combinação é coerente com a filosofia que estamos usando ao longo do projeto. Em vez de depender de uma única barreira, eu distribuo garantias nos pontos em que elas fazem sentido.

DDL e seed idempotentes

Outro requisito é que tanto a criação da estrutura quanto o seed sejam idempotentes.

Quero poder inicializar a store mais de uma vez sem destruir o ambiente nem criar duplicações descontroladas. A criação das tabelas deve funcionar mesmo quando a estrutura já existe.

O seed segue a mesma ideia. O cenário inicial precisa poder ser aplicado de forma previsível para que o projeto comece com os dados que já estamos utilizando nos exercícios.

O conjunto continua representando cinco serviços e seis alertas variados, com três em estado de firing e três resolvidos. Também quero usar runbooks associados aos contextos de checkout, payments e outros serviços utilizados nos cenários do OpsPilot.

Esses dados são importantes porque as estratégias dependem deles para tomar decisões. Quando pergunto o que está acontecendo no plantão, o agente precisa encontrar um cenário coerente e reproduzível.

Prepared statements como regra de segurança

Um dos requisitos que faço questão de deixar explícito é o uso de prepared statements em todas as queries.

Não quero SQL montado pela concatenação direta de valores recebidos da aplicação.

Um prepared statement mantém o comando SQL separado dos dados que serão inseridos nele. Os valores entram nos pontos reservados para parâmetros, em vez de serem colados diretamente no texto da consulta.

Essa separação é particularmente importante porque alguns desses valores podem ter origem em entradas do usuário. O título de um incidente, por exemplo, pode nascer de uma mensagem enviada ao agente e depois chegar até a persistência.

Se eu simplesmente concatenar essa entrada dentro de um comando SQL, crio o tipo de condição associada a ataques de SQL Injection.

Aqui, Zod e prepared statements cumprem responsabilidades diferentes. Zod ajuda a validar a estrutura e os tipos dos dados que entram na aplicação. O prepared statement protege a interação com o banco ao impedir que o valor seja tratado como parte do comando SQL.

Eu gosto de relacionar essa escolha com a mesma filosofia que utilizamos nas deny lists. Em vez de confiar que todos os dados estarão sempre seguros, construo uma barreira técnica para reduzir a possibilidade de determinadas classes de falha.

Novas operações e evolução das tools

Com a persistência real, também quero evoluir as operações que ficam disponíveis para o agente.

Uma delas é a listagem de incidentes. Essa consulta deve aceitar um status e permitir trabalhar com incidentes abertos, resolvidos ou com todos. O comportamento padrão será listar os abertos.

Também mantenho as operações de abertura e resolução de incidentes e acrescento a capacidade de consultar runbooks associados a serviços.

A integração com runbooks é importante para o objetivo do OpsPilot. O agente não precisa apenas identificar que existe um incidente. Ele também deve conseguir recuperar material que ajude o plantonista a decidir por onde começar a atuação.

Além de criar ou ajustar essas tools, peço uma revisão das descrições existentes. Como já vimos no ReAct, a descrição da ferramenta é parte do contexto que ajuda o modelo a decidir quando utilizá-la.

Se uma tool possui uma descrição ruim ou incompleta, o modelo pode escolher a operação errada mesmo que toda a implementação esteja correta. Por isso, quero revisar especialmente as regras que orientam quando abrir um incidente e deixar os campos descritos de forma clara.

Composição da aplicação e stores diferentes por cenário

Na composição principal da aplicação, a implementação normal passa a utilizar a SQLiteOpsStore.

Nos testes e no benchmark, porém, quero continuar podendo injetar a store em memória.

Essa decisão preserva a separação por contrato. A estratégia não deve saber se os dados estão vindo de SQLite ou de uma estrutura temporária. Ela apenas utiliza as operações oferecidas pela OpsStore.

Para o benchmark, isso também é útil porque preciso de cenários reproduzíveis. Quando comparo ReAct e Plan-and-Execute, não quero que uma execução anterior deixe dados persistidos e altere o resultado da próxima estratégia.

A store em memória continua sendo, portanto, uma ferramenta importante de teste e avaliação, mesmo depois que introduzimos persistência real na aplicação.

Protegendo o arquivo local de dados

Como agora teremos um arquivo de banco local, a pasta que contém esses dados precisa entrar no .gitignore.

Eu não quero versionar o banco de desenvolvimento. O repositório deve conter a estrutura necessária para criar o banco e os mecanismos de seed, não o estado particular produzido na máquina de cada pessoa.

Esse cuidado mantém o Git limpo e evita que operações locais do agente ou dos testes gerem alterações constantes em um arquivo de dados versionado.

Testes previstos para a nova store

A especificação também precisa deixar claro como a nova persistência será testada.

Quero validar o ciclo de criação, leitura, atualização e resolução utilizando SQLite em memória. Também quero testar filtros e as restrições que colocamos na estrutura.

Os testes já existentes devem continuar podendo rodar sobre a store em memória quando esse for o cenário apropriado.

Essa separação permite validar a nova integração sem tornar toda a suíte dependente do arquivo físico usado pela aplicação em desenvolvimento.

Novamente, o objetivo é preservar determinismo. Um teste precisa começar com um estado conhecido e terminar sem deixar efeitos colaterais para a próxima execução.

Revisando a spec gerada pelo Spec Kit

Depois de enviar todos esses requisitos, deixo o Spec Kit gerar a nova especificação.

A nova spec aparece na estrutura do projeto com as user stories, os requisitos funcionais e as premissas assumidas pelo framework.

Eu reviso esse material antes de seguir porque essa feature altera uma parte importante da arquitetura. Não quero chegar à implementação e só então descobrir que o agente interpretou a persistência de outra forma.

Com a especificação validada, avanço para plan e tasks. Essas duas etapas podem demorar, então deixo o agente processar os artefatos e depois volto para revisar principalmente a lista de tarefas.

O que aparece no planejamento e nas tarefas

As tarefas começam com ajustes de setup para alinhar o projeto à nova escolha de persistência. Isso inclui rever dependências que ficaram associadas à configuração anterior e adequar o package.json ao uso de SQLite nativo.

Também aparecem mudanças relacionadas à OpsStore e aos tipos principais da aplicação.

O planejamento prevê a adição de erros específicos, como o caso em que um runbook solicitado não é encontrado. Esse tipo de erro precisa continuar seguindo a lógica de domínio que já utilizamos em outras partes do projeto.

Outro ponto importante é a atualização do seed. Agora ele deixa de existir apenas como um recurso para dados temporários e passa a participar da inicialização da nova persistência.

A lista de tarefas também cobre consultas, integração das tools, testes e os ajustes necessários na composição da aplicação.

Ao revisar o tamanho das tarefas, observo que elas continuam pequenas o suficiente para caber em commits individuais. Mesmo que o agente não esteja criando um commit automaticamente para cada item, essa granularidade continua sendo útil para revisão e reversão.

Se eu quiser tornar essa regra ainda mais rígida, posso acrescentá-la às instructions do projeto e exigir explicitamente um commit por tarefa.

Preparando a implementação para a próxima etapa

Neste momento eu paro antes do implement.

A especificação, o planejamento e as tarefas estão prontos, mas quero fornecer uma referência para a implementação da SQLiteOpsStore antes de liberar o agente.

Não quero que ele invente completamente essa parte porque aqui existe valor didático em enxergar como a aplicação conversa diretamente com o banco.

Se o agente sugerir um ORM ou outra camada de abstração, isso pode ser uma alternativa válida posteriormente. Entretanto, para esta primeira implementação, prefiro trabalhar mais próximo do SQLite e dos prepared statements para entender claramente o que está acontecendo.

Depois que esse mecanismo estiver funcionando e os conceitos estiverem claros, podemos considerar uma migração para uma biblioteca ou ORM sem perder a compreensão das garantias que queremos preservar na camada de persistência.

NA PRÁTICA

Abra o snapshot da Unidade 3 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que trocar para SQLite” no fluxo apresentado?
- Que decisão de engenharia está associada a “Atualizando a Constitution antes da nova spec” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Por que trocar para SQLite.
- Atualizando a Constitution antes da nova spec.
- Especificando uma persistência real para as operações.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Criando e Consultando Incidentes no Banco com Nosso Agente

Unidade 3 • Function Calling e Tool Use

OBJETIVOS DE APRENDIZAGEM

- Criar uma referência para a SQLiteOpsStore.
- Definir a tabela de incidentes.
- Compreender severidade, status e revisão do código gerado.

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Criando uma referência para a SQLiteOpsStore
- Definindo a tabela de incidentes
- Severidade, status e revisão do código gerado
- Prepared statements nas consultas
- Listando incidentes com filtro
- Executando o implement do Spec Kit

Conteúdo da aula

Nesta aula, eu começo a implementação da persistência que especificamos anteriormente. Até aqui, já definimos que a aplicação passaria a utilizar SQLite, que a OpsStore continuaria sendo o contrato central de persistência e que os testes poderiam usar um banco em memória. Agora eu quero sair da especificação e transformar essas decisões em uma implementação concreta.

Como fizemos em outras etapas do projeto, eu não vou simplesmente executar o implement do Spec Kit e aceitar qualquer código que o agente decidir produzir. Primeiro preparo uma referência com o formato que quero para a SQLiteOpsStore. Essa referência não é o arquivo final. Ela serve como orientação para que o agente de código entenda como desejo trabalhar com a API nativa do SQLite, quais garantias quero preservar e qual padrão deve ser seguido nas queries.

Criando uma referência para a SQLiteOpsStore

Eu começo definindo a classe que vai implementar a interface OpsStore já existente no projeto. Essa decisão é importante porque a nova persistência não deve alterar o contrato utilizado pelas estratégias, pelas tools ou pelo restante da aplicação. Eu quero trocar a implementação concreta, não o comportamento esperado pelos consumidores.

A classe recebe uma instância do banco SQLite e abre o banco a partir de um caminho configurável. Em execução normal, esse caminho aponta para o arquivo local da aplicação. Quando não houver uma configuração específica, utilizo um caminho padrão dentro da pasta de dados do OpsPilot.

Nos testes, essa mesma estrutura poderá ser inicializada com o modo em memória. Dessa forma, preservo exatamente o mesmo contrato de store, mas troco o destino físico das operações.

No construtor, a classe abre a conexão e executa a preparação inicial da estrutura do banco. É nesse momento que as tabelas necessárias devem existir para que as demais operações funcionem.

Definindo a tabela de incidentes

Na referência, eu escrevo explicitamente a criação da tabela de incidents. Quero mostrar ao agente o nível de controle que espero sobre a estrutura e, ao mesmo tempo, deixar que ele complete as outras tabelas seguindo o mesmo padrão.

A tabela possui um identificador que funciona como chave primária, um título obrigatório, o identificador do serviço relacionado ao incidente, a severidade, o status, a data de criação, a data de resolução e um resumo opcional.

Alguns desses campos possuem regras adicionais. Severidade e status não devem aceitar qualquer texto arbitrário. Eu quero checks no próprio SQLite para que somente os valores previstos pelo domínio sejam aceitos.

Essa validação no banco não substitui TypeScript ou Zod. Ela adiciona mais uma camada. Se alguma inconsistência escapar do compilador ou da validação da aplicação, o banco ainda pode impedir a gravação de um valor que não pertence ao conjunto aceito.

Essa é a mesma filosofia que seguimos quando criamos deny lists, pre-commit e outros guardrails. Eu não quero depender de uma única proteção. Sempre que uma camada consegue reforçar uma regra importante, vale avaliar se essa garantia deve existir ali também.

Severidade, status e revisão do código gerado

Na referência, eu quero trabalhar com os níveis de severidade que já fazem parte do projeto. A ideia é que a estrutura do banco reflita os tipos que definimos anteriormente, sem inventar uma classificação diferente no meio da implementação.

Esse detalhe se torna importante mais tarde, durante a revisão. O agente gera uma lista de severidades diferente daquela que eu havia solicitado, utilizando termos como critical, high, medium e low. Isso não corresponde exatamente ao conjunto que eu havia descrito.

Eu não considero esse tipo de divergência um problema insolúvel. O ponto importante é perceber que ela existe porque eu abri e li o arquivo. Se eu apenas aceitasse a implementação por ter sido gerada com sucesso, uma decisão de domínio teria sido alterada silenciosamente.

Posso corrigir os checks para utilizar os valores definidos na especificação ou, se fizer sentido para o projeto, criar uma normalização explícita entre os dois formatos. O que não quero é deixar essa diferença existir sem uma decisão consciente.

Prepared statements nas consultas

Depois da criação da estrutura, preparo uma referência para a consulta de incidentes. A regra principal continua sendo o uso de prepared statements.

Eu monto a query com parâmetros, em vez de concatenar diretamente o valor do filtro dentro do SQL. Se o usuário pedir apenas incidentes com determinado status, o valor é enviado separadamente para o statement preparado.

Esse padrão é especialmente importante porque vários dados chegam até a store a partir de entradas controladas por usuários ou por respostas do agente. Um título de incidente pode conter aspas, apóstrofes ou qualquer outro conteúdo. Nada disso deve virar parte do comando SQL.

A separação entre comando e valor reduz o risco de SQL Injection e torna a persistência mais previsível. Zod valida a estrutura do dado, enquanto o prepared statement garante que aquele valor seja tratado como dado pelo banco.

Quero que esse mesmo padrão seja repetido nas demais operações da store: listar alertas, abrir incidentes, resolver incidentes e consultar runbooks. Todas devem utilizar statements preparados e devolver dados tipados de acordo com os contratos existentes.

Listando incidentes com filtro

A operação de listagem recebe um filtro. Quando o filtro solicita todos os incidentes, a consulta não precisa aplicar uma condição de status. Nos demais casos, a query utiliza o parâmetro informado.

Também quero ordenar os resultados pela data de criação para que a listagem tenha um comportamento consistente.

O retorno precisa ser convertido para os tipos de domínio esperados pela aplicação. Eu não quero que detalhes da representação interna do SQLite vazem para as estratégias ou para as tools. A store é responsável por fazer essa adaptação.

Ao fornecer esse exemplo ao agente, deixo claro que as outras funções devem seguir a mesma ideia: query preparada, leitura ou escrita controlada e retorno tipado.

Executando o implement do Spec Kit

Com essa referência pronta, volto para a janela do agente e executo a etapa de implementação do Spec Kit.

O agente já possui a especificação, o planejamento e a lista de tarefas criadas na aula anterior. Agora acrescento a referência de SQLite para orientar especificamente a construção da OpsStore.

Essa combinação é importante. O Spec Kit continua sendo responsável por conduzir o desenvolvimento orientado a especificações. A referência de código apenas reduz a ambiguidade sobre uma parte em que eu quero um formato mais específico.

Depois disso, deixo a implementação rodar e volto para revisar o resultado quando a execução termina.

Revisando a store gerada

Na revisão, o primeiro arquivo que observo é a SQLiteOpsStore.

O agente cria tipos internos para representar linhas de incidentes, alertas e runbooks. Também implementa a classe seguindo a interface OpsStore, como já havíamos pedido.

Embora eu tenha escrito apenas uma tabela como referência, ele completa as demais estruturas necessárias para services, alerts, incidents e runbooks. Isso mostra exatamente o comportamento que eu queria provocar: eu forneço o padrão e deixo o agente expandir a implementação com base na spec.

A preparação do banco também é criada e o seed é integrado ao fluxo. As funções de consulta e alteração aparecem para listar alertas, buscar incidentes, criar incidentes, resolver incidentes e recuperar runbooks.

O agente também organiza consultas e índices necessários para a utilização do banco. A implementação final é maior do que a referência, mas preserva a direção que estabeleci.

O contrato como primeira camada de proteção

Ao revisar a classe, eu volto ao significado do implements OpsStore.

A interface funciona como contrato. O compilador verifica se a implementação fornece as operações exigidas e se os tipos são compatíveis com aquilo que o restante da aplicação espera.

Se a SQLiteOpsStore deixar de implementar uma operação obrigatória ou devolver algo incompatível, o TypeScript consegue detectar essa divergência antes da execução normal.

Esse contrato vem sendo construído desde as etapas anteriores do OpsPilot. Agora ele mostra seu valor porque conseguimos trocar a store em memória por SQLite sem precisar redefinir todos os consumidores.

Checks do SQLite como outra barreira

A segunda camada de proteção está nas restrições do próprio banco.

Os checks de severidade e status impedem a gravação de valores fora dos conjuntos permitidos. Mesmo que um bug acesse TypeScript ou Zod, o SQLite ainda pode rejeitar um registro inválido.

Esse tipo de redundância é intencional. Não é uma repetição sem propósito. Cada camada protege um ponto diferente do caminho.

O TypeScript protege o contrato durante o desenvolvimento. Zod valida os dados em runtime nas fronteiras. Os checks do SQLite protegem a persistência. Os prepared statements protegem a interpretação do SQL.

Quando olhamos o sistema dessa forma, a persistência também passa a fazer parte dos guardrails da aplicação.

Executando type check e testes

Depois da revisão inicial, eu executo novamente o type check e a suíte de testes.

Os testes utilizam o SQLite em modo:memory:, conforme havíamos especificado.

Isso faz com que a criação do banco, o seed e as operações aconteçam em uma instância descartável. Ao terminar o teste, aquele estado deixa de existir. O banco local utilizado durante o desenvolvimento não fica contaminado por cenários artificiais.

A execução também é rápida porque não depende de um serviço externo de banco de dados. Não precisamos iniciar container, abrir conexão de rede ou limpar uma base compartilhada.

Com type check e testes passando, tenho uma indicação de que a store nova está compatível com os contratos e com os cenários já automatizados. Ainda assim, quero fazer uma validação de ponta a ponta.

Criando um incidente através do agente

Para verificar a persistência real, reinicio o servidor da aplicação e começo uma nova execução.

Em seguida, envio uma mensagem pela API pedindo a abertura de um incidente relacionado ao checkout, informando uma condição de latência elevada e uma severidade intermediária.

O agente recebe a mensagem, utiliza a estratégia configurada, escolhe a tool correspondente e cria o incidente na nova store SQLite.

Na primeira tentativa de observar a resposta, encontro pequenos problemas de formatação e de normalização da severidade. A aplicação chega a abrir o incidente, mas os valores ainda não estão apresentados da forma ideal.

Esse ponto se conecta à divergência que já havíamos observado nos checks de severidade. Se uma parte do projeto utiliza uma nomenclatura e outra trabalha com um formato diferente, eu preciso normalizar explicitamente essa tradução.

Consultando os incidentes abertos

Depois de criar o incidente, faço uma nova pergunta ao agente: quais incidentes estão abertos.

Agora a tool de listagem consulta a SQLiteOpsStore e recupera os registros persistidos no banco.

Esse teste é mais interessante do que simplesmente consultar o arquivo diretamente, porque ele valida o caminho completo: API, estratégia de raciocínio, tool, store e banco.

Em uma das execuções, o modelo retorna um erro que não está relacionado à lógica de persistência. É um problema vindo da integração com o modelo utilizado pelo OpenRouter.

Nesse caso, copio o erro e peço ao agente de código para analisar e corrigir o comportamento. Essa é outra característica do desenvolvimento com agentes: alguns problemas podem variar conforme o modelo que está sendo utilizado naquele momento.

O projeto piloto utilizado na preparação da aula não apresentou exatamente o mesmo erro, enquanto o projeto executado durante a gravação apresentou. Isso é esperado quando diferentes agentes e modelos tomam decisões diferentes durante a geração e a execução.

Ajustando a resposta e normalizando severidade

Durante os ajustes, também aproveito para melhorar a forma como a resposta da API aparece no terminal. Quero receber de forma mais direta os dados relevantes para a consulta, sem precisar ficar extraindo manualmente informações de uma estrutura pouco conveniente.

Outra correção é normalizar os níveis de severidade. O agente havia produzido uma combinação entre valores numéricos conceituais e nomes como low, medium, high e critical.

Eu peço uma tradução consistente entre os níveis aceitos pelo domínio e a forma utilizada nas respostas. O importante é que a aplicação possua um único significado para cada severidade e que a persistência, as tools e a apresentação estejam alinhadas.

Provando que os dados sobrevivem ao reinício

Depois dos ajustes, faço o teste que realmente demonstra a persistência.

Primeiro inicio o servidor e peço para abrir um novo incidente. O agente cria o registro no banco.

Em seguida, consulto os incidentes abertos e confirmo que existem dois registros, incluindo o que já havia sido criado anteriormente e o novo incidente.

Depois eu encerro completamente o servidor e inicio a aplicação novamente.

Se ainda estivéssemos utilizando apenas a store em memória, o estado criado na execução anterior desapareceria com o processo. Ao consultar os incidentes abertos depois do reinício, eu esperaria encontrar um conjunto diferente ou vazio, dependendo do seed.

Com SQLite, os registros permanecem. A nova consulta retorna novamente os incidentes que estavam abertos antes do restart.

Essa é a validação prática de que a aplicação deixou de depender apenas da memória do processo. O agente agora consegue abrir um incidente em uma execução, o servidor pode ser encerrado e, na próxima inicialização, a informação continua disponível para novas consultas.

Agentes diferentes podem produzir erros diferentes

Ao longo dessa implementação, alguns ajustes foram necessários porque o código gerado e o comportamento do modelo não ficaram idênticos ao projeto usado como referência antes da aula.

Isso pode acontecer também com quem estiver acompanhando o projeto. Copilot, Cursor e outros agentes podem interpretar determinados detalhes de formas diferentes. Mesmo dentro da mesma ferramenta, a troca do modelo pode produzir decisões diferentes.

Por isso, eu não quero que o objetivo seja reproduzir exatamente o mesmo erro ou exatamente o mesmo arquivo que aparece na gravação. O mais importante é conseguir identificar a origem do problema, revisar os contratos e utilizar o próprio agente para ajudar a corrigir aquilo que está divergindo.

Nesta etapa, a referência da SQLiteOpsStore, os checks do banco, os prepared statements, os testes em memória e a verificação após reiniciar o servidor nos dão critérios concretos para saber se a persistência está funcionando da maneira esperada.

NA PRÁTICA

Abra o snapshot da Unidade 3 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Criando uma referência para a SQLiteOpsStore” no fluxo apresentado?
- Que decisão de engenharia está associada a “Definindo a tabela de incidentes” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Criando uma referência para a SQLiteOpsStore.
- Definindo a tabela de incidentes.
- Severidade, status e revisão do código gerado.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Validando Disponibilidade de Provedores Externos

Unidade 3 • Function Calling e Tool Use

OBJETIVOS DE APRENDIZAGEM

- Compreender erro como observação para o agente.
- Compreender defesas básicas para toda ferramenta externa.
- Criar uma tool de status de provedores.

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Erro como observação para o agente
- Defesas básicas para toda ferramenta externa
- Criando uma tool de status de provedores
- Especificando a resiliência da chamada
- Falha final sem lançar exceção para fora da tool
- Testes sem depender da internet

Conteúdo da aula

Até aqui, todas as ferramentas que construí para o OpsPilot estavam sob meu controle direto. Elas consultavam dados locais, operavam sobre a store da aplicação e respondiam de forma rápida e previsível. A partir desta aula, esse cenário começa a mudar porque quero integrar o agente com provedores externos e, mais adiante, expor parte desse comportamento em ambientes como GitHub Pages e também através de MCP.

Quando começo a depender da rede, preciso assumir que algumas coisas vão falhar. Uma chamada pode demorar mais do que o esperado, o serviço pode ficar indisponível, a conexão pode cair ou a resposta pode chegar em um formato diferente daquele que a aplicação conhece. Nada disso é excepcional em sistemas distribuídos. É comportamento normal de integrações com APIs externas.

Por isso, eu não quero tratar uma falha externa simplesmente como uma exceção que quebra o processo inteiro. Se o plantonista receber apenas um erro 500, ele pode não entender o que aconteceu e o agente perde a chance de raciocinar sobre a limitação. A regra que quero adotar aqui é diferente: falhas externas passam a ser observações que entram no fluxo do agente.

Erro como observação para o agente

Quando uma tool externa não consegue consultar um provedor, ela deve transformar essa falha em um resultado legível. Em vez de deixar uma exceção escapar e interromper toda a execução, quero devolver algo como a informação de que não foi possível consultar o status daquele serviço.

Esse detalhe muda bastante o comportamento do sistema. Uma exceção encerra o fluxo. Uma observação pode ser lida pelo modelo.

No ReAct, essa diferença é especialmente importante. O agente pensa, executa uma ação, observa o resultado e decide o próximo passo. Se a observação informa que uma consulta externa falhou, o modelo pode decidir tentar novamente, seguir por outro caminho ou responder ao plantonista explicando a limitação.

A resiliência, portanto, não fica apenas na infraestrutura da aplicação. Ela também passa a fazer parte do raciocínio do agente. A tool precisa devolver um resultado útil mesmo quando o provedor não responde, e a estratégia pode usar essa informação para continuar a execução de forma controlada.

Defesas básicas para toda ferramenta externa

A partir daqui, eu quero que toda ferramenta que dependa de rede carregue algumas defesas básicas de fábrica.

A primeira é timeout. Nenhuma chamada deve ficar esperando indefinidamente. Se um provedor não responde dentro do tempo que considero aceitável, a requisição precisa ser interrompida e transformada em uma observação.

A segunda é retry. Algumas falhas são transitórias. Uma conexão pode cair momentaneamente ou um serviço pode retornar um erro temporário. Nesses casos, vale tentar novamente um número limitado de vezes antes de assumir a falha definitiva.

A terceira é validação da resposta. Mesmo quando o servidor responde com sucesso, eu não quero confiar automaticamente no JSON recebido. O formato pode mudar, algum campo pode desaparecer ou um valor pode vir com um tipo diferente. Por isso, continuo utilizando Zod para validar as respostas externas.

Essas três camadas formam um conjunto simples de resiliência: limitar o tempo, repetir algumas falhas recuperáveis e validar o que voltou da rede antes de utilizar o conteúdo no raciocínio.

Criando uma tool de status de provedores

A primeira integração externa será uma tool de status de provedores. A ideia é permitir que o OpsPilot consulte páginas públicas de status para tentar diferenciar um problema interno de uma indisponibilidade externa.

Se existe um alerta no checkout, por exemplo, pode ser útil descobrir se algum provedor importante também está apresentando problemas. Isso não resolve automaticamente o incidente, mas dá contexto para o plantonista e para o próprio agente.

A tool será responsável por consultar páginas públicas que não exigem chave de API. Para o exercício, começo com GitHub e Cloudflare.

O parâmetro da ferramenta representa qual provedor deve ser consultado. Esse parâmetro fica restrito aos valores que conhecemos e o GitHub funciona como opção padrão.

A descrição da tool também precisa orientar quando ela deve ser usada. Quero que o modelo pense nessa consulta quando existir suspeita de problema externo, dependência indisponível ou quando for necessário verificar se a falha observada na aplicação pode estar relacionada a um serviço de terceiros.

Especificando a resiliência da chamada

Na especificação do Spec Kit, defino um timeout de cinco segundos para cada tentativa. Esse limite será aplicado utilizando o mecanismo de cancelamento disponível no próprio ambiente de execução.

Também defino uma nova tentativa quando houver falha de rede ou erro de servidor na classe 5xx. Isso significa que a ferramenta pode fazer até duas tentativas no total antes de considerar que a consulta realmente falhou.

Não quero um retry infinito. Assim como limitamos recursões no ReAct, passos no Plan-and-Execute e reflexões no Reflection, também preciso limitar tentativas de rede. Toda repetição tem custo e aumenta a latência.

Depois de receber uma resposta, o conteúdo precisa passar por validação com Zod. O formato esperado para a página de status contém pelo menos um indicador e uma descrição.

O retorno para o agente deve ser compacto. Quero uma linha que combine indicador e descrição, porque esse texto entra no contexto da estratégia. Se eu devolver estruturas enormes para cada tool, o histórico cresce sem necessidade e o custo do raciocínio aumenta.

Falha final sem lançar exceção para fora da tool

Outro requisito importante é que a falha final não deve escapar da ferramenta como exceção.

Depois que as tentativas terminarem, a tool retorna uma string de erro legível. Essa string passa a ser a observação daquela ação.

Com isso, o agente consegue continuar. Ele pode dizer ao plantonista que não conseguiu consultar o provedor, mas ainda pode usar alertas internos, incidentes já persistidos, runbooks e outras informações disponíveis.

Essa decisão evita transformar a indisponibilidade de um serviço externo em indisponibilidade completa do OpsPilot.

O princípio é degradar de forma controlada. Se uma integração falha, perco aquela informação específica, mas preservo o restante da capacidade do agente.

Testes sem depender da internet

A especificação também exige que a função de acesso à rede seja injetável.

Eu não quero que os testes unitários dependam de GitHub, Cloudflare ou qualquer outro provedor real. Se o teste precisar da internet, ele fica sujeito à instabilidade externa, latência e mudanças que não controlamos.

Por isso, a função de fetch pode ser substituída por uma implementação fake durante os testes.

Quero cobrir pelo menos três cenários: uma resposta válida, um timeout e uma resposta com formato inválido.

Todos eles devem funcionar sem rede. Dessa forma, consigo verificar o comportamento de resiliência no CI de forma determinística. A integração real continua sendo testada em execuções específicas, mas a lógica da ferramenta não precisa de um serviço externo para ser validada.

Executando Specify, Plan e Tasks

Com esses requisitos descritos, executo novamente o fluxo do Spec Kit.

Primeiro gero a especificação da nova tool. Depois avanço para o planejamento e para a geração das tarefas.

Eu mantenho a mesma disciplina usada nas features anteriores. A existência de uma integração externa não muda o método. Continuo registrando o comportamento esperado antes de implementar.

Depois que plan e tasks estão prontos, paro antes do implement para fornecer uma referência de código. Quero mostrar ao agente como imagino a estrutura de validação, URLs dos provedores, tentativa de chamada e tratamento de falhas.

Referência para o schema da página de status

Na referência, começo pelo schema da resposta externa.

Quero um objeto com indicator e description, ambos textuais. Esse contrato representa apenas os dados de que realmente preciso para o raciocínio.

Não tenho interesse em carregar toda a resposta do provedor se o agente só precisa saber qual é o estado e qual descrição acompanha aquele estado.

Também centralizo as URLs dos provedores em uma estrutura conhecida. Assim, a função recebe apenas o nome do provider e localiza o endpoint correspondente sem espalhar URLs pela aplicação.

Tentativas, timeout e respostas HTTP

A função de consulta recebe o provedor e a implementação de fetch. Em execução normal, utilizo o fetch real. Nos testes, posso passar a versão fake.

O fluxo percorre no máximo duas tentativas.

Em cada tentativa, faço a requisição com timeout de cinco segundos. Se o servidor retornar um status 5xx, trato essa situação como uma falha recuperável e permito que a repetição aconteça.

Também diferencio respostas HTTP que não representam sucesso. Quero transformar esse estado em uma mensagem legível em vez de deixar um erro genérico chegar ao restante da aplicação.

Quando a resposta é válida, converto o JSON e submeto o conteúdo ao schema do Zod. Só depois disso considero que possuo uma observação confiável para devolver ao agente.

O resultado final fica compacto, informando o provedor, o indicador e a descrição.

Tratando a falha definitiva

Dentro do tratamento de erro, verifico se já estou na última tentativa.

Se ainda existe uma chance de retry, a execução volta para a próxima tentativa. Se o limite foi alcançado, devolvo uma mensagem informando que não foi possível consultar o status do provedor e incluo a causa conhecida da falha.

Essa mensagem também orienta o raciocínio seguinte. Mesmo sem o status externo, o agente deve continuar com base nos alertas internos e avisar o plantonista de que aquela dependência não pôde ser verificada.

Esse comportamento mostra de forma concreta o que significa transformar erro em observação. A falha não desaparece. Ela fica explícita no contexto e pode influenciar a resposta do modelo.

Implementando a feature com o Spec Kit

Depois de preparar a referência, volto ao agente e executo o implement do Spec Kit.

O agente utiliza a especificação, o plano, as tarefas e o exemplo que forneci para completar a feature.

Como nas aulas anteriores, eu deixo o processo terminar e depois volto para uma validação prática. O ponto principal agora não é revisar cada linha da implementação, mas verificar se a ferramenta externa se comporta como definimos quando existe uma falha real.

Provocando uma falha de propósito

Para testar o comportamento, mantenho o servidor do OpsPilot em execução e envio uma nova mensagem utilizando ReAct.

Durante o exercício, uma das URLs configuradas para o provedor externo é propositalmente inválida. Isso cria uma falha controlada e permite observar como a tool e o agente reagem.

A consulta ao GitHub pode funcionar normalmente, enquanto a integração é configurada de forma incorreta falha.

Em vez de derrubar a execução inteira, essa falha volta para o agente como observação. O ReAct consegue ler o resultado, combinar essa limitação com os dados internos e continuar tomando uma decisão.

O agente continua operando mesmo com provedor indisponível

Na execução demonstrada, mesmo com uma das consultas externas falhando, o agente consegue abrir o incidente solicitado.

A resposta informa o incidente criado, sua severidade e o status, enquanto a falha externa permanece disponível como parte do contexto utilizado durante o raciocínio.

Esse resultado é o comportamento que eu queria validar. O OpsPilot não precisa considerar que tudo está perdido apenas porque uma dependência externa não respondeu.

Ele consegue trabalhar com aquilo que ainda está disponível e comunicar a limitação ao plantonista.

Essa característica é especialmente importante em operações. Justamente durante incidentes, serviços externos podem estar instáveis. Um copiloto de plantão que quebra por completo quando uma única página de status fica indisponível não ajuda no momento em que mais precisamos dele.

Resiliência como parte do desenho do agente

A partir desta integração, eu começo a tratar resiliência como parte do desenho das tools e não como uma correção posterior.

Toda nova ferramenta externa deve considerar tempo máximo de espera, número controlado de tentativas, validação do retorno e uma forma segura de representar falhas para o agente.

Esse padrão também melhora a observabilidade. Quando a consulta falha, o trace pode registrar a observação e eu consigo entender que determinada decisão foi tomada sem aquela informação externa.

Mais adiante, essa mesma filosofia será útil quando o OpsPilot passar a conversar com outros serviços e quando começarmos a expor capacidades através de MCP. Quanto mais integrações externas existirem, mais importante se torna diferenciar uma falha de ferramenta de uma falha completa do agente.

NA PRÁTICA

Abra o snapshot da Unidade 3 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Erro como observação para o agente” no fluxo apresentado?
- Que decisão de engenharia está associada a “Defesas básicas para toda ferramenta externa” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Erro como observação para o agente.
- Defesas básicas para toda ferramenta externa.
- Criando uma tool de status de provedores.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Disponibilizando Nosso Agente Via MCP

Unidade 3 • Function Calling e Tool Use

OBJETIVOS DE APRENDIZAGEM

- Compreender mCP como protocolo entre agentes e ferramentas.
- Compreender o MCP não é um novo backend.
- Especificar o servidor MCP.

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- MCP como protocolo entre agentes e ferramentas
- O MCP não é um novo backend
- Especificando o servidor MCP
- A regra crítica do transporte stdio
- Testando o contrato do servidor
- Revisando a especificação gerada

Conteúdo da aula

Neste ponto do projeto, eu já construí uma quantidade relevante de capacidades para o OpsPilot. Tenho tools para consultar alertas, abrir e resolver incidentes, consultar status de provedores externos e trabalhar com dados persistidos em SQLite. Também tenho estratégias de raciocínio, API HTTP, arena, benchmark e mecanismos de resiliência. O próximo problema é de compatibilidade.

Até agora, essas capacidades foram utilizadas principalmente dentro das ferramentas e agentes que acompanharam o desenvolvimento, como Copilot e Cursor. A pergunta passa a ser outra: se eu quiser utilizar essas mesmas operações em outro agente, em outra IDE ou até em uma aplicação escrita em outra linguagem, preciso reescrever todas as integrações?

Esse é o tipo de situação que pode se transformar em um problema N por M. Eu posso ter N agentes diferentes e M fontes de ferramentas. Se cada combinação exigir uma integração específica, começo a manter várias pontes artesanais para oferecer as mesmas capacidades. A resposta que vamos adotar para evitar esse acoplamento é o MCP, Model Context Protocol.

MCP como protocolo entre agentes e ferramentas

O MCP funciona como um protocolo aberto entre agentes e provedores de capacidades. Em vez de implementar uma integração específica para cada agente, eu escrevo um servidor de ferramentas uma única vez e deixo clientes compatíveis consumirem esse contrato.

A proposta é justamente permitir que diferentes ambientes utilizem as mesmas capacidades. Copilot, Cursor e outros agentes compatíveis podem se conectar ao mesmo servidor MCP sem que eu precise duplicar toda a lógica de negócio.

Um servidor MCP pode expor três categorias principais de elementos. A primeira são as tools, que representam funções que o agente pode chamar. A segunda são resources, que representam dados que podem ser consultados. A terceira são prompts, que funcionam como templates de interação prontos para determinados cenários.

Nesta etapa, meu foco está nas tools porque o OpsPilot já possui operações de negócio bem definidas que quero disponibilizar para outros agentes.

O MCP não é um novo backend

Existe uma regra arquitetural que considero essencial antes de escrever a implementação: o MCP não deve virar um segundo backend independente.

Eu já tenho um domínio, uma OpsStore, schemas de validação e tools que são utilizados pela API e pelas estratégias do OpsPilot. Não quero recriar versões paralelas dessas regras apenas porque apareceu uma nova forma de acesso.

O MCP será uma segunda porta para o mesmo sistema. A API HTTP é uma porta. O MCP será outra. As duas precisam chegar à mesma lógica e ao mesmo estado persistido.

Se a versão HTTP abrir um incidente de uma forma e a versão MCP abrir de outra, eu passo a manter dois produtos diferentes dentro do mesmo repositório. Isso cria divergência de regras, validações e comportamento.

Por isso, um dos requisitos mais importantes da especificação é reutilizar a mesma OpsStore e os mesmos schemas Zod que já sustentam as tools existentes. Eu quero uma única fonte de verdade.

Especificando o servidor MCP

Volto novamente ao Spec Kit e crio uma nova especificação para o servidor MCP do OpsPilot.

O servidor ficará em um arquivo dedicado e utilizará o SDK do Model Context Protocol com transporte stdio. A primeira versão precisa expor as operações que considero centrais para o plantão: listar alertas, abrir incidentes e resolver incidentes.

Essas operações já existem no projeto. O trabalho aqui não é inventar uma nova regra para cada uma delas, mas criar a ponte MCP que permita chamá-las.

Também defino um nome único para o servidor, OpsPilot, e preparo um script do projeto para inicializá-lo. Se a execução depender de variáveis de ambiente, esse script também precisa carregá-las corretamente, seguindo o mesmo cuidado que tivemos com arena, benchmark e servidor HTTP.

A regra crítica do transporte stdio

O transporte stdio introduz uma regra importante: eu não posso usar console.log livremente no servidor.

Nesse tipo de transporte, o stdout é o próprio canal de comunicação do protocolo. O cliente e o servidor utilizam esse fluxo para trocar mensagens. Se eu escrever logs arbitrários no mesmo canal, posso corromper a conversa.

Isso significa que um log aparentemente inofensivo pode fazer o cliente interpretar texto de diagnóstico como se fosse parte de uma mensagem MCP.

Por isso, a especificação registra explicitamente que não deve existir console.log no servidor. Quando eu precisar de diagnóstico, a saída deve ir para stderr, mantendo stdout reservado ao protocolo.

Essa regra parece pequena, mas representa bem a diferença entre um programa comum de terminal e um processo que utiliza stdio como transporte estruturado.

Testando o contrato do servidor

Também quero testes para o servidor MCP.

Um dos cenários principais é validar a listagem das tools expostas. Isso me permite confirmar que o cliente conseguirá descobrir as operações previstas e que o servidor está registrando os contratos corretamente.

Da mesma forma que fizemos em outras partes do projeto, a preferência é testar a lógica sem depender desnecessariamente de serviços externos. O MCP é uma nova interface para capacidades já existentes, então os testes precisam se concentrar no contrato e na integração com os componentes do OpsPilot.

Revisando a especificação gerada

Depois de executar a etapa de specify, abro a nova spec e reviso os requisitos.

Verifico se as três operações principais realmente aparecem, se a mesma OpsStore está sendo reutilizada e se os mesmos schemas de validação foram mantidos. Esses são os pontos que evitam a duplicação de backend.

Também reviso os casos de borda e as regras relacionadas ao transporte. Só depois disso avanço para plan e tasks.

Novamente, eu não trato o Spec Kit como uma caixa-preta. O framework organiza o trabalho, mas continuo responsável por verificar se a especificação gerada representa o desenho que quero manter.

Planejamento e tarefas antes da implementação

Com a spec validada, executo planejamento e geração de tarefas.

O fluxo permanece o mesmo que utilizamos em todas as features anteriores. O agente lê Constitution, instructions e especificação, produz o plano técnico e transforma esse plano em unidades de implementação.

Antes do implement, eu paro para fornecer uma referência. Quero mostrar como imagino o registro de uma tool no servidor MCP, como a store deve ser criada e como o transporte deve ser conectado.

Essa referência funciona novamente como um parâmetro de qualidade. Eu não preciso escrever todas as tools manualmente, mas ofereço um exemplo suficientemente claro para que o agente complete as demais seguindo o mesmo padrão.

Registrando as tools do OpsPilot

Na referência, começo criando o servidor com o nome OpsPilot e uma versão inicial.

Também instancio a SQLiteOpsStore, porque quero que as operações feitas por MCP cheguem à mesma persistência já utilizada pela aplicação.

A primeira tool usada como exemplo é a abertura de incidentes.

A descrição da tool precisa ser clara para o cliente e para o modelo. Ela deve indicar que essa operação deve ser utilizada quando o usuário relatar um problema em produção ou pedir explicitamente o registro de um incidente.

Os campos de entrada também precisam possuir descrições úteis. Título, serviço afetado e severidade não são apenas parâmetros técnicos. Esses metadados ajudam o agente a montar a chamada de maneira correta.

A severidade deve utilizar os valores normalizados que definimos no projeto, preservando o domínio que já foi corrigido nas etapas anteriores.

Quando a tool é executada, ela chama a mesma operação da store e transforma o retorno em conteúdo compatível com o protocolo MCP.

As operações de listagem de alertas e resolução de incidentes seguem o mesmo padrão. O importante é que eu não replique a regra de negócio. Apenas adapto entrada e saída para o protocolo.

Conectando o servidor ao transporte

Depois de registrar as tools, o servidor é conectado ao transporte stdio.

A partir daí, ele fica pronto para ser iniciado por um cliente compatível. O processo não precisa abrir uma porta HTTP específica para essa integração, porque a comunicação ocorre pelos fluxos de entrada e saída do processo.

Para diagnóstico, mantenho qualquer informação de inicialização fora do stdout. Isso preserva o canal do protocolo e evita uma classe de erros que seria difícil de diagnosticar olhando apenas para o cliente.

Implementando com o Spec Kit

Com a referência pronta, volto para a janela do agente, executo o implement do Spec Kit e forneço o exemplo como orientação.

O agente utiliza os artefatos que já produzimos para criar o servidor MCP, registrar as demais tools e ajustar scripts e testes necessários.

Depois da implementação, o próximo objetivo é deixar de olhar apenas para os arquivos e realmente conectar um cliente ao servidor.

Configurando o MCP no ambiente de desenvolvimento

Como estou trabalhando com IDEs que possuem suporte a MCP, preciso registrar o servidor no ambiente.

No VS Code, preparo a configuração MCP na área correspondente do projeto. Informo que o transporte é stdio, qual comando deve iniciar o processo e quais argumentos devem ser passados para executar o script MCP do OpsPilot.

Depois percebo que o Cursor utiliza sua própria localização de configuração. Apenas criar o arquivo do VS Code não é suficiente para que o Cursor descubra o servidor.

Por isso, crio também a configuração específica dentro da pasta esperada pelo Cursor. O conteúdo é muito parecido porque continua apontando para o mesmo servidor e para o mesmo comando de inicialização.

A diferença está na forma como cada cliente organiza sua configuração. O backend MCP continua sendo único.

Reload, nova sessão e descoberta das tools

Depois de alterar a configuração, faço reload da janela para que o ambiente tente descobrir o novo servidor.

Um detalhe que aparece na prática é que o reload nem sempre faz uma conversa já aberta reconhecer imediatamente as novas tools. Em alguns casos, preciso iniciar uma nova janela de contexto.

Isso acontece porque a lista de ferramentas disponíveis pode ter sido carregada quando a sessão começou. Adicionar um servidor depois não significa necessariamente que aquela conversa vai atualizar seu conjunto de capacidades automaticamente.

Ao abrir uma nova sessão, consigo verificar novamente se o MCP foi descoberto.

Resolvendo um problema de conexão

Na primeira tentativa com o Cursor, o servidor aparece com erro de conexão.

A causa está relacionada à forma como o comando de inicialização tenta encontrar o runtime necessário para executar o TypeScript. Em vez de tentar adivinhar toda a correção manualmente, utilizo o próprio agente de código para analisar a mensagem de erro e ajustar a configuração.

Depois da correção, reinicio o servidor MCP pelo ambiente e confirmo que ele aparece conectado.

Esse tipo de problema é importante porque mostra que disponibilizar uma tool não termina na implementação do servidor. O processo também precisa ser inicializável pelo cliente, encontrar as dependências e carregar o ambiente correto.

Chamando o OpsPilot pelo Cursor

Com o MCP conectado no Cursor, abro uma nova janela de agente e faço um pedido em linguagem natural para criar um incidente no serviço de notifications.

Dessa vez, o trace mostra que o agente escolhe a tool do OpsPilot exposta por MCP. Ele prepara os parâmetros e chama a operação de abertura de incidente.

Esse é o resultado que eu buscava. O Cursor não precisou conhecer a implementação da SQLiteOpsStore, não precisou importar LangChain e não precisou recriar a regra de abertura de incidentes.

Ele apenas descobriu uma ferramenta MCP com um contrato conhecido e utilizou essa ferramenta.

Testando também no VS Code

Em paralelo, faço o mesmo tipo de teste no VS Code com Copilot.

Aqui aparece outro detalhe operacional: o agente estava trabalhando em uma branch diferente daquela que eu esperava.

Isso faz com que o ambiente pareça inconsistente. Em determinado momento, o agente acredita ter criado o incidente, mas o resultado não aparece na persistência que estou consultando.

Depois de identificar a diferença de branch e alinhar o contexto correto do projeto, o Copilot passa a enxergar o MCP esperado e utiliza a tool de abertura de incidente.

Esse problema não estava no protocolo MCP. Era uma questão do estado do ambiente de desenvolvimento. A distinção é importante para não começar a alterar o servidor quando a causa está em outra camada.

Verificando os incidentes persistidos

Depois de executar chamadas pelo Cursor e pelo VS Code, volto ao terminal e consulto os incidentes abertos através da aplicação.

Os registros criados pelas tools MCP aparecem no mesmo conjunto persistido que já vinha sendo utilizado pela API.

Essa verificação confirma a decisão arquitetural principal desta aula. O MCP é realmente uma segunda porta para o OpsPilot, não um backend isolado.

Quando o Cursor abre um incidente via MCP e depois a API HTTP lista os incidentes, ambos enxergam o mesmo estado porque compartilham a mesma OpsStore e o mesmo banco SQLite.

Também consigo observar que algumas tentativas interrompidas durante os testes não geraram o resultado esperado, enquanto as chamadas concluídas corretamente aparecem na listagem. Isso ajuda a diferenciar chamadas efetivamente persistidas de interações que ficaram apenas no contexto do agente.

Compatibilidade sem duplicar a lógica

Ao final dessa integração, consigo utilizar o mesmo OpsPilot por diferentes clientes.

No Cursor, o agente chama as tools MCP. No VS Code, Copilot pode utilizar o mesmo servidor. A API HTTP continua funcionando. A arena e o benchmark continuam utilizando as estratégias do projeto.

Tudo isso converge para as mesmas operações de domínio e para a mesma persistência.

Essa é a vantagem prática do protocolo. Se eu adicionar outro agente compatível com MCP, o objetivo é não precisar recriar manualmente todas as integrações. O novo cliente descobre as ferramentas e conversa com o servidor usando o contrato comum.

O caminho percorrido pelo projeto também fica mais claro. Começamos com estado em memória, passamos por uma representação simples dos dados, introduzimos persistência SQLite, adicionamos resiliência para fronteiras externas e agora expomos capacidades através de um protocolo reutilizável por diferentes agentes.

As mesmas preocupações continuam valendo: contratos tipados, validação com Zod, prepared statements, timeout, retry, limites de interação e revisão humana. O MCP acrescenta interoperabilidade, mas não substitui os guardrails que já construímos ao redor do agente.

NA PRÁTICA

Abra o snapshot da Unidade 3 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “MCP como protocolo entre agentes e ferramentas” no fluxo apresentado?
- Que decisão de engenharia está associada a “O MCP não é um novo backend” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- MCP como protocolo entre agentes e ferramentas.
- O MCP não é um novo backend.
- Especificando o servidor MCP.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 3

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE

Agente →	Tools →	Store / SQLite →	APIs externas →	HTTP / MCP →	Validação
----------	---------	------------------	-----------------	--------------	-----------

CHECKLIST DA UNIDADE

- Revisar a Aula 1: Especificando a Integração com Banco de Dados
- Revisar a Aula 2: Criando e Consultando Incidentes no Banco com Nosso Agente
- Revisar a Aula 3: Validando Disponibilidade de Provedores Externos
- Revisar a Aula 4: Disponibilizando Nosso Agente Via MCP

GITHUB DA UNIDADE 3

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Unidade 4

Memória e Reflexão em Agentes Autônomos

Evolução do OpsPilot com memória episódica e memória semântica, embeddings locais, recall e aprendizagem refletida.

FLUXO VISUAL DA UNIDADE					
Mensagem →	Memória episódica →	Memória semântica →	Recall →	Resposta →	Learning Reflector

GITHUB DA UNIDADE 4
Abrir no GitHub • Ler UNIDADE.md

Aula	Tema
1	Memória Episódica Para Nosso Agente
2	O Vector Store: Spec, Implementação e a Memória Funcionando

Aula 1 — Memória Episódica Para Nosso Agente

Unidade 4 • Memória e Reflexão em Agentes Autônomos

OBJETIVOS DE APRENDIZAGEM

- Explicar por que um assistente sem memória deixa de ajudar.
- Compreender que o modelo não possui memória própria.
- Compreender as camadas de memória que vamos construir.

GITHUB DA UNIDADE 4

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Por que um assistente sem memória deixa de ajudar
- O modelo não possui memória própria
- As camadas de memória que vamos construir
- Working Memory
- Memória de longa duração
- Memória episódica

Conteúdo da aula

Nesta aula, eu começo a unidade 4 com um problema que fica cada vez mais evidente conforme o OpsPilot ganha capacidades. Neste ponto, o agente já raciocina, utiliza um banco de dados, consulta o mundo externo por meio de provedores como GitHub e Cloudflare e também expõe operações via MCP. Mesmo assim, ele ainda apresenta uma limitação básica para qualquer assistente: ele não se lembra da conversa.

Para deixar esse problema explícito, eu faço um teste simples. Primeiro digo ao agente como quero ser chamado e, na mesma mensagem, peço a abertura de um incidente. Ele responde corretamente e executa a operação. Logo em seguida, envio outra requisição perguntando qual é o meu nome. O agente responde que não sabe e pede que eu informe novamente.

Isso acontece mesmo quando as duas mensagens estão muito próximas no tempo. O problema não é a distância entre as requisições. O problema é que cada request é atendida como uma nova execução, sem o histórico anterior. Cada chamada ao LLM recebe apenas o contexto enviado naquela requisição.

Por que um assistente sem memória deixa de ajudar

Em uma aplicação de plantão, esse comportamento seria especialmente ruim. Imagine que eu esteja no meio de um incidente de produção e precise explicar novamente, a cada mensagem, quem eu sou, qual incidente estou tratando, o que já foi feito e quais decisões já foram tomadas.

Nesse cenário, eu gasto mais tempo reconstruindo o contexto para o agente do que recebendo ajuda dele. O assistente deixa de funcionar como assistente porque toda interação recomeça do zero.

Produtos como GPT, Gemini e outros assistentes conseguem dar a impressão de continuidade justamente porque existe alguma forma de memória sendo construída ao redor do modelo. Eles podem usar histórico de conversa, fatos persistidos, preferências e mecanismos de recuperação que colocam informações relevantes de volta no contexto.

O modelo, isoladamente, não faz isso sozinho. Essa é a ideia mais importante desta unidade: toda memória de agente é engenharia nossa.

O modelo não possui memória própria

Uma chamada ao LLM nasce, lê o contexto que enviamos e termina. Ela não mantém uma lembrança automática da chamada anterior.

Quando queremos que um agente se lembre de alguma coisa, precisamos decidir o que guardar, onde guardar, como recuperar e em que momento colocar essa informação novamente no prompt.

Por isso, eu gosto de pensar em memória como gestão de contexto com persistência. Persistir informação não é suficiente. Se eu guardar tudo e nunca recuperar, o agente continua sem saber. Se eu recuperar informação demais, o contexto cresce, o custo aumenta e a qualidade pode piorar.

A arquitetura de memória precisa equilibrar retenção, relevância e tamanho do contexto.

As camadas de memória que vamos construir

A partir desta unidade, quero evoluir o OpsPilot em duas frentes principais. A primeira é o fio de conversa persistente. O agente precisa lembrar do que foi dito anteriormente, inclusive depois que a aplicação for reiniciada.

A segunda é a memória semântica. Nesse caso, não quero guardar apenas mensagens em ordem cronológica. Quero extrair fatos e preferências do usuário e conseguir recuperá-los por significado.

Também vamos introduzir um refletor de aprendizado. Depois de uma resposta, uma chamada adicional poderá analisar a conversa e destilar aquilo que realmente vale a pena lembrar em uma frase durável.

Por fim, também precisamos considerar o esquecimento. Se o usuário pedir que determinada informação seja apagada, o sistema deve conseguir remover aquilo que aprendeu.

Nesta aula, porém, começo pela base: a memória episódica sustentada por um histórico persistente de conversa.

Working Memory

Antes da implementação, eu organizo alguns conceitos de memória que vamos usar ao longo das próximas aulas.

A primeira categoria é a Working Memory. Ela representa a conversa em andamento, aquilo que está disponível dentro do contexto atual.

Quando abro um chat no Copilot e continuo conversando na mesma janela, por exemplo, existe um conjunto de mensagens recentes que compõem essa memória de trabalho. Enquanto aquela conversa permanece aberta e o contexto continua sendo enviado, o agente consegue relacionar o que eu disse agora com aquilo que apareceu alguns turnos antes.

No OpsPilot atual, essa continuidade não existe entre as requisições HTTP. Cada chamada recebe apenas a nova mensagem. É justamente essa lacuna que vamos começar a preencher.

Memória de longa duração

Outra categoria é a memória que sobrevive em disco e representa decisões ou informações que queremos manter por mais tempo.

Dentro do nosso fluxo de desenvolvimento, o arquivo de Copilot Instructions é um bom exemplo de memória declarada. Nós decidimos explicitamente que determinadas informações do projeto precisam ser consultadas de forma recorrente e as registramos em um arquivo persistente.

Essa memória não surgiu espontaneamente. Nós escolhemos quais princípios deveriam sobreviver ao encerramento de um chat e os colocamos em uma fonte que novas sessões conseguem consultar.

O mesmo princípio vale para um agente de produto. Algumas informações precisam sobreviver à execução atual e continuar disponíveis depois.

Memória episódica

A memória episódica representa um diário do que aconteceu e da ordem em que aconteceu.

No OpsPilot, a tabela de mensagens será a forma mais direta dessa memória. Cada interação entra no banco associada a uma conversa. Depois consigo recuperar uma sequência de mensagens anteriores e reconstruir parte daquele episódio.

Históricos de incidentes também podem funcionar como memória episódica. Eles registram eventos, decisões e mudanças de estado ao longo do tempo.

Esse tipo de memória preserva contexto temporal. Não é apenas um conjunto de fatos soltos. A ordem das mensagens ajuda a entender o que levou a determinada resposta ou decisão.

Memória semântica

A memória semântica funciona de outra forma. Ela guarda fatos e preferências que foram destilados de episódios específicos.

Se eu digo em uma conversa que meu nome é Thiago, esse fato pode nascer dentro de um episódio, mas depois ser convertido em uma memória semântica que diz que o usuário prefere ser chamado de Thiago.

O mesmo pode acontecer com preferências operacionais. Ao longo de várias interações, o sistema pode aprender que eu prefiro ver alertas críticos antes dos demais.

Nesse caso, não preciso necessariamente recuperar a conversa inteira em que essa preferência apareceu. Posso recuperar o fato já consolidado.

Mais adiante, vamos trabalhar com embeddings para buscar essas memórias por significado, sem depender de correspondência exata de palavras.

O verdadeiro desafio é relevância

Guardar dados é relativamente simples. O ponto mais difícil é descobrir o que deve voltar para o contexto em cada momento.

Se eu disser apenas "organize o meu plantão", o sistema precisa descobrir quais memórias podem ajudar a interpretar essa solicitação. Talvez eu prefira incidentes críticos primeiro. Talvez eu queira priorizar os mais fáceis de resolver. Talvez exista uma preferência relacionada ao serviço que estou acompanhando.

A qualidade da memória não depende apenas de quantidade. Ela depende da capacidade de recuperar a informação certa para aquela situação.

Nesta primeira etapa, vamos começar com uma regra simples e previsível: recuperar as últimas mensagens da conversa. Depois evoluímos para mecanismos mais seletivos.

Primeiro passo: conversa persistente

O primeiro objetivo da implementação é fazer com que o endpoint de chat consiga manter um fio de conversa.

Para isso, vou introduzir uma Conversation Store com operações para criar uma conversa, acrescentar mensagens e recuperar as últimas mensagens associadas a um identificador.

Essa store também será persistida no SQLite que já utilizamos para incidentes, alertas e runbooks. Eu não quero criar outro mecanismo de persistência apenas para memória.

A nova estrutura adiciona uma tabela de mensagens ao banco e mantém as conversas associadas a um identificador único.

Conversation ID na API

O endpoint de chat passa a aceitar um conversationId opcional.

Quando o cliente ainda não possui uma conversa, a aplicação cria um novo identificador e o devolve na resposta.

Nas mensagens seguintes, o cliente envia esse mesmo conversationId. Isso permite que a aplicação saiba qual histórico precisa recuperar antes de executar a estratégia.

Esse desenho é importante porque a memória precisa estar associada a uma conversa específica. Se eu misturar mensagens de usuários ou incidentes diferentes, o contexto se torna incorreto e o agente começa a responder com informações que pertencem a outro episódio.

Uma janela limitada de histórico

Eu não quero enviar a conversa inteira para o modelo a cada nova mensagem.

Se uma conversa durar horas ou dias, o histórico pode crescer indefinidamente. Enviar tudo aumenta o tamanho do contexto, pode elevar o custo e ainda introduz informações antigas que talvez já não sejam relevantes.

Por isso, defino uma janela. Nesta etapa, quero recuperar as últimas doze mensagens.

Esse número é uma decisão inicial e pode virar uma configuração mais adiante. O conceito importante é que existe um limite explícito.

A memória episódica persistida pode ser grande, mas a memória de trabalho injetada em cada execução continua controlada.

Especificando a conversa persistente com o Spec Kit

Como em todas as features anteriores, começo pelo Spec Kit.

Na nova especificação, defino a Conversation Store, a operação de append, a recuperação das últimas mensagens e a nova tabela de messages dentro da SQLiteOpsStore.

Também especifico que o endpoint /chat deve aceitar o conversationId opcional e devolvê-lo na resposta.

A estratégia deve receber as doze mensagens mais recentes como parte do contexto.

Além disso, quero uma nova métrica: history messages. Ela informa quantas mensagens do histórico foram realmente injetadas naquela execução.

Essa métrica é importante porque quero medir o peso da memória desde o início, em vez de adicionar contexto sem nenhuma visibilidade sobre quanto estou carregando.

Testes com SQLite em memória

Os testes continuam utilizando SQLite em modo:memory:.

Essa escolha, que já fizemos na camada de persistência, continua útil aqui. Cada teste pode criar uma conversa, gravar mensagens, recuperar o histórico e depois descartar todo o banco.

Assim, consigo verificar a lógica de memória sem escrever no banco real do desenvolvedor e sem deixar conversas artificiais acumuladas entre execuções.

O banco em memória preserva o comportamento real do SQLite, mas mantém a suíte rápida e isolada.

Revisando a especificação antes do plano

Depois que o Spec Kit gera a spec, eu abro o documento e reviso as user stories e os requisitos funcionais.

Um dos cenários centrais é justamente o plantonista trocar várias mensagens sobre o mesmo incidente e esperar que o OpsPilot mantenha a continuidade.

A lógica geral dessa feature é simples: guardar mensagens no banco e recuperá-las antes de cada nova execução.

Mesmo sendo simples, o limite de histórico precisa estar explícito porque eu não quero transformar memória persistente em crescimento ilimitado de contexto.

Com a especificação alinhada, avanço para plan e tasks e deixo o framework gerar os artefatos necessários.

Referência para o fluxo de memória episódica

Antes do implement, preparo uma pequena referência para mostrar ao agente de código como quero que o fluxo de chat seja organizado.

A execução começa obtendo o conversationId recebido pelo cliente. Se ele não existir, a aplicação cria uma conversa nova.

Depois, recupera as últimas doze mensagens daquele histórico.

A nova mensagem do usuário é persistida na conversa e a estratégia é executada recebendo tanto a mensagem atual quanto o histórico recuperado.

Quando a estratégia termina, a resposta do assistente também é persistida.

Por fim, a API devolve o `conversationId`, o resultado da estratégia e as métricas, incluindo a quantidade de mensagens históricas utilizadas.

Esse fluxo é pequeno, mas representa a essência da memória episódica que queremos construir: recuperar, injetar, executar e registrar novamente.

Medindo o peso da memória

Eu faço questão de colocar a quantidade de mensagens históricas nas métricas porque memória tem custo.

Cada item recuperado aumenta o contexto enviado ao modelo. Se eu decidir ampliar a janela de doze para vinte, cinquenta ou cem mensagens, isso muda o comportamento e pode mudar a latência e o consumo.

Medir history messages desde o começo permite correlacionar esse crescimento com outros indicadores que já temos, como chamadas ao LLM e tempo de execução.

Assim como fizemos com ReAct, Plan-and-Execute e Reflection, quero transformar decisões de arquitetura em algo observável.

Implementando com o Spec Kit

Depois de preparar a referência, executo a etapa de implement do Spec Kit.

O agente já possui a especificação, o planejamento, as tarefas e o exemplo do fluxo esperado. Ele completa a Conversation Store, integra a nova tabela ao SQLite e ajusta o endpoint de chat e as estratégias para receber o histórico.

Quando a implementação termina, parto direto para uma validação prática. O teste precisa reproduzir exatamente o problema que mostrei no começo da aula.

Validando a memória entre duas requisições

Eu inicio novamente o servidor e faço a primeira chamada para o endpoint de chat.

Nessa requisição, informo meu nome e guardo o `conversationId` devolvido pela API.

Esse identificador é o vínculo entre aquela mensagem e as próximas interações.

Na segunda chamada, envio o mesmo `conversationId` e pergunto qual é o meu nome.

Diferentemente do que aconteceu no início da aula, agora o agente recupera o histórico da conversa antes de responder.

A mensagem anterior volta para o contexto, a estratégia consegue relacionar a pergunta atual ao que já foi dito e responde corretamente com o nome informado.

Essa é a primeira evidência de que o OpsPilot deixou de tratar cada request como um episódio completamente isolado.

Persistência como base para as próximas memórias

Com esse mecanismo, eu resolvo apenas a primeira camada do problema.

A conversa agora possui continuidade porque as mensagens são persistidas e uma janela recente é reinjetada no prompt. Isso já permite que o agente lembre do que foi dito anteriormente e mantém o fio da interação.

Ainda não estamos extraindo fatos duráveis nem buscando preferências por significado. Também não estamos usando embeddings ou o refletor de aprendizado.

Essas capacidades entram nas próximas etapas. A base, porém, já está pronta: o agente agora consegue guardar episódios, recuperar parte deles e utilizar esse histórico como contexto para novas decisões.

NA PRÁTICA

Abra o snapshot da Unidade 4 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que um assistente sem memória deixa de ajudar” no fluxo apresentado?
- Que decisão de engenharia está associada a “O modelo não possui memória própria” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Por que um assistente sem memória deixa de ajudar.
- O modelo não possui memória própria.
- As camadas de memória que vamos construir.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — O Vector Store: Spec, Implementação e a Memória Funcionando

Unidade 4 • Memória e Reflexão em Agentes Autônomos

OBJETIVOS DE APRENDIZAGEM

- Explicar o que é um embedding.
- Compreender similaridade entre vetores.
- Explicar por que não usar um serviço vetorial externo.

GITHUB DA UNIDADE 4

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- O que é um embedding
- Similaridade entre vetores
- Por que não usar um serviço vetorial externo
- Especificando a memória semântica
- Top 3 e gate de relevância
- Persistência e modelo local

Conteúdo da aula

Na aula anterior, eu resolvi uma primeira parte do problema de memória do OpsPilot. Com a memória episódica, consigo manter um fio de conversa persistente e recuperar as últimas mensagens a partir de um `conversationId`. Isso já permite que o agente saiba o que foi dito anteriormente dentro daquele episódio. Mesmo assim, ainda falta uma capacidade importante: recuperar fatos e preferências relevantes sem depender de um identificador de conversa e sem precisar reler um histórico inteiro.

Se eu disser que prefiro ver eventos críticos primeiro, quero que essa preferência possa influenciar uma solicitação futura como "organize o meu plantão". As duas frases não são iguais e talvez nem compartilhem as mesmas palavras, mas possuem relação de significado. É justamente esse tipo de recuperação que vamos começar a construir com memória semântica e embeddings.

O que é um embedding

Um embedding é uma representação numérica de um texto. Em vez de trabalhar apenas com palavras, transformamos a informação em um vetor, ou seja, em uma lista de números.

No modelo que vou utilizar nesta etapa, essa representação tem algumas centenas de dimensões, por volta de 384. O número exato importa menos do que a propriedade que buscamos: textos semanticamente parecidos tendem a produzir vetores próximos.

Uma frase como "organize o meu plantão" pode ficar próxima de uma memória como "prefiro ver os críticos primeiro". Em contrapartida, um pedido sem relação com operações, como uma receita de pizza, deve ficar distante.

Essa proximidade permite buscar memória por significado, e não apenas por correspondência exata de palavras. É isso que diferencia a nova camada da simples consulta ao histórico de mensagens.

Similaridade entre vetores

Para comparar esses vetores, precisamos de uma medida de similaridade. Um conceito comum é a similaridade de cosseno, que observa a relação angular entre os vetores.

Nesta implementação, a busca utiliza produto escalar sobre embeddings normalizados, chegando ao mesmo tipo de objetivo prático: atribuir um score de proximidade entre a pergunta atual e cada memória do usuário.

Eu não vou aprofundar toda a matemática de vetores e cossenos porque o foco do módulo continua sendo a arquitetura do agente. O que preciso compreender é que cada memória recebe uma representação numérica e que, diante de uma nova consulta, consigo ordenar essas memórias pela proximidade semântica.

Por que não usar um serviço vetorial externo

Uma alternativa bastante comum seria utilizar embeddings oferecidos por uma API externa, como a OpenAI, em conjunto com um banco vetorial gerenciado. Essa abordagem faz sentido em muitos projetos, principalmente quando existe uma quantidade muito grande de vetores ou requisitos de busca distribuída.

Para o nosso cenário, eu quero seguir outro caminho. Vamos utilizar um modelo de embeddings pequeno, o all-MiniLM-L6-v2, rodando localmente. Ele ocupa algo na ordem de dezenas de megabytes e pode ser executado na própria máquina.

A escala do nosso exemplo também é pequena. Estamos falando de dezenas de memórias por usuário. Nesse volume, uma busca em array pode ser extremamente rápida e não justifica introduzir uma infraestrutura vetorial externa.

Essa escolha reduz custos, evita chamadas adicionais para gerar embeddings e aumenta a privacidade, porque a preferência do usuário não precisa sair da máquina apenas para ser transformada em vetor.

Se estivéssemos falando de milhões de vetores em produção, a resposta arquitetural poderia ser diferente. Aqui, porém, quero a solução mais simples que preserve o conceito que precisamos aprender.

Especificando a memória semântica

Volto então ao Spec Kit e crio uma nova especificação para memória semântica.

Quero uma Memory Store organizada por `userId`. Isso significa que as memórias precisam ser isoladas por usuário. Quando eu faço `recall` para um usuário, não quero recuperar preferências pertencentes a outra pessoa.

Essa store terá pelo menos duas operações principais. A primeira é `remember`, responsável por persistir um fato. A segunda é `recall`, responsável por buscar as memórias mais relevantes para uma nova consulta.

A especificação também define um mecanismo de deduplicação. Se uma memória nova for semanticamente muito parecida com outra já salva, com `score` acima de 0,92, ela não deve ser armazenada novamente.

Esse detalhe evita que a mesma preferência seja gravada várias vezes com pequenas variações de frase. Sem deduplicação, eu poderia terminar com quinze versões de "prefere críticos primeiro", e o `recall` devolveria praticamente a mesma informação repetida.

Top 3 e gate de relevância

Para o `recall`, defino que quero no máximo as três memórias mais relevantes.

Além disso, não basta estar entre as três melhores. A memória precisa superar um limiar mínimo de relevância. Nesta etapa, utilizo 0,3 como `gate`.

Esse filtro é importante porque sempre existirão três melhores resultados quando houver memórias armazenadas, mesmo que todos sejam ruins. Se a maior similaridade for praticamente irrelevante para a pergunta atual, eu prefiro não injetar memória nenhuma.

Contexto continua sendo orçamento. Uma memória irrelevante não ajuda o modelo e ainda aumenta ruído, tokens e possibilidade de resposta enviesada por informação que não deveria estar naquele turno.

Persistência e modelo local

A especificação adiciona uma tabela de `memories` ao SQLite. Entre os dados previstos estão o identificador, o `userId`, a representação de `embedding` e a data de criação.

Para gerar os vetores, utilizo o pacote de `transformers` compatível com execução local no projeto. O modelo MiniLM fica dentro da área de memória da aplicação e o planejamento também define a estratégia de `pooling` necessária para produzir a representação final do texto.

Outro detalhe interessante é que, nesta feature, eu quero testes utilizando embeddings reais. Não vou substituir o modelo por um fake apenas para validar a busca. Como o modelo roda localmente, consigo exercitar o comportamento real da memória sem depender de rede ou pagar por uma API externa.

Revisando plano e tarefas

Depois do `specify`, executo `plan` e `tasks`.

O Spec Kit cria os artefatos técnicos, atualiza os pontos necessários do projeto e adiciona as tarefas para dependência, modelagem, persistência, geração de embeddings e integração com o chat.

Eu faço uma revisão geral, mas não paro em cada arquivo porque o objetivo da aula é chegar rapidamente ao comportamento da memória funcionando. Ainda assim, o fluxo permanece o mesmo: a implementação continua sendo guiada pela spec, pelo plano e pelas tarefas.

A referência do recall

Antes do `implement`, preparo uma referência pequena para mostrar o coração da busca vetorial.

A função recebe `userId`, a query atual e um limite de resultados, que por padrão será três.

Primeiro gero o embedding da consulta. Depois recupero as memórias daquele usuário e calculo um score entre o vetor da pergunta e o vetor de cada memória.

Em seguida, ordeno os resultados do maior score para o menor, seleciono os três primeiros e aplico o filtro de relevância mínima.

São poucas etapas, mas elas resumem todo o recall semântico que precisamos para esta escala: transformar a pergunta em vetor, comparar, ordenar, limitar e filtrar.

O filtro final não pode ser esquecido. Se nenhuma memória atingir o limiar, o resultado correto é não injetar nada no contexto.

Integrando recall ao endpoint de chat

A nova especificação também altera o endpoint de chat. Além do `conversationId` utilizado pela memória episódica, agora o request pode carregar um `userId`.

Antes de executar a estratégia, a aplicação usa a mensagem atual como query para recuperar as memórias semanticamente relevantes daquele usuário.

Essas memórias são injetadas no prompt junto com o contexto necessário para a estratégia. Assim, uma conversa nova pode se beneficiar de fatos aprendidos em episódios anteriores, mesmo que não compartilhe o mesmo `conversationId`.

Esse ponto diferencia bem as duas camadas. A memória episódica reconstrói o fio recente de uma conversa. A memória semântica recupera fatos duráveis pela relevância do significado.

O refletor de aprendizado

Depois de implementar a store e o recall, ainda existe um problema. Se eu precisar chamar `remember` manualmente sempre que surgir uma preferência, a memória não aprende de forma natural.

Por isso, crio uma segunda especificação para um refletor de aprendizado.

Depois de cada resposta, uma etapa adicional analisa o último turno e decide se existe algum fato durável que vale a pena guardar. A ideia é destilar a conversa em uma frase curta que sobreviva ao episódio.

Eu não quero guardar qualquer coisa. Um pedido pontual não deve virar memória permanente. Também não quero persistir segredos. O refletor precisa distinguir entre algo circunstancial e um fato ou preferência que realmente pode ajudar em interações futuras.

Quando encontra um aprendizado válido, ele chama `memories.remember` de forma assíncrona.

Também incluo a ideia de esquecimento. O usuário precisa ter uma operação para pedir que uma memória seja removida. Memória útil não significa memória sem controle.

Executando a segunda spec

Para essa camada de aprendizado, executo novamente `specify`, `plan`, `tasks` e `implement`. Desta vez, não preciso fornecer uma referência extensa de código porque o comportamento já está bem delimitado pelos requisitos.

Ao final, o OpsPilot passa a ter não apenas armazenamento e recall, mas também um caminho para aprender fatos a partir das próprias interações.

Preparando um cenário para validar a preferência

Com a implementação pronta, preparo um cenário prático.

Escolho um `userId` específico para representar meu usuário e crio alguns incidentes com severidades diferentes, incluindo low, high e critical.

A ideia é deixar os dados fora de uma ordem que já favoreça a preferência. Assim, consigo observar depois se a memória realmente influencia a organização apresentada pelo agente.

Ao listar os incidentes inicialmente, percebo que o filtro por usuário ainda não se comporta exatamente como eu esperava e aparecem registros adicionais. Para o objetivo da aula, isso não impede a validação principal e pode ser ajustado posteriormente.

Gravando uma preferência

Em seguida, registro para esse usuário um fato simples: ele prefere alertas críticos primeiro.

Na primeira tentativa encontro um problema porque a rota de `memories` ainda não havia sido implementada corretamente na versão gerada pelo meu agente. Eu passo o erro para o Cursor e peço que ele corrija a integração.

Esse tipo de diferença pode não acontecer em outra execução. Como já vimos ao longo do projeto, agentes e modelos diferentes podem deixar lacunas diferentes mesmo partindo da mesma especificação.

Depois da correção, faço novamente a chamada para guardar o fato.

Deduplicação funcionando

Em uma nova tentativa de salvar a mesma preferência, a store informa que a memória não foi armazenada novamente.

Isso acontece porque o fato já estava persistido e a similaridade ultrapassou o limiar de deduplicação.

Esse resultado confirma o comportamento que definimos na spec. O sistema não precisa comparar apenas texto literal. Duas formulações semanticamente equivalentes podem ser tratadas como a mesma memória.

A deduplicação mantém a base mais limpa e evita que o recall seja dominado por repetições do mesmo aprendizado.

Recuperando a memória para organizar o plantão

Depois disso, faço a pergunta que realmente importa: peço ao agente para organizar o meu plantão.

Como a mensagem carrega o `userId`, o OpsPilot gera o embedding da solicitação e consulta a Memory Store.

A preferência sobre alertas críticos aparece como memória relevante e pode ser injetada no prompt antes da execução da estratégia.

Em uma primeira resposta, o formato ainda não fica ideal e o agente faz perguntas adicionais sobre como organizar a visualização. Eu ajusto a solicitação para deixar explícito que quero os incidentes abertos com suas severidades e uma ordenação coerente com a prioridade operacional.

Depois desse ajuste, a resposta passa a apresentar os itens críticos primeiro, alinhada à preferência que havia sido persistida.

Observando recall e memória nas métricas

Também ajusto a resposta da API para expor métricas relacionadas à memória.

Quero saber quantas memórias foram recuperadas para o turno e se algum aprendizado novo foi armazenado.

Na execução de validação, o OpsPilot mostra que uma memória foi recuperada para construir a resposta. Ao mesmo tempo, não foi necessário utilizar mensagens do histórico episódico daquela conversa.

Esse detalhe é importante porque demonstra que a resposta não dependeu da janela recente de conversation history. Ela foi influenciada por uma memória semântica pertencente ao usuário.

Agora consigo observar separadamente o peso do histórico e o peso do recall semântico, da mesma forma que já observava chamadas ao LLM e latência.

A memória funcionando no agente

Com essa etapa, o OpsPilot passa a combinar duas formas diferentes de contexto persistente.

A memória episódica mantém o fio da conversa por meio das mensagens recentes. A memória semântica guarda fatos e preferências e os recupera por proximidade de significado.

O refletor de aprendizado acrescenta um caminho para transformar correções e informações duráveis em novas memórias sem depender de cadastro manual em todos os casos.

Ainda existem melhorias possíveis. O formato das respostas pode ser mais consistente, a apresentação dos incidentes pode ser refinada e a política de relevância pode ser ajustada com base em novos testes.

Mesmo assim, o comportamento principal já está presente: o agente consegue guardar uma preferência, evitar duplicá-la, recuperar essa preferência em uma solicitação semanticamente relacionada e utilizá-la para produzir uma resposta mais adequada ao usuário.

NA PRÁTICA

Abra o snapshot da Unidade 4 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O que é um embedding” no fluxo apresentado?
- Que decisão de engenharia está associada a “Similaridade entre vetores” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- O que é um embedding.
- Similaridade entre vetores.
- Por que não usar um serviço vetorial externo.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 4

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE					
Mensagem →	Memória episódica →	Memória semântica →	Recall →	Resposta →	Learning Reflector

CHECKLIST DA UNIDADE
- Revisar a Aula 1: Memória Episódica Para Nosso Agente - Revisar a Aula 2: O Vector Store: Spec, Implementação e a Memória Funcionando

GITHUB DA UNIDADE 4
Abrir no GitHub • Ler UNIDADE.md

Unidade 5

Gerenciamento de Contextos

Tratamento do contexto como orçamento, com medição, sumarização, pruning e montagem controlada das seções do prompt.

FLUXO VISUAL DA UNIDADE					
System prompt →	Memórias →	Histórico →	Tools →	ContextBuilder →	Budget / Pruning / Summary

GITHUB DA UNIDADE 5	
Abrir no GitHub • Ler UNIDADE.md	

Aula	Tema
1	O Contexto como Orçamento: Anatomia, Medição e Sumarização
2	Context Stitching e os Paralelos com a Ferramenta

Aula 1 — O Contexto como Orçamento: Anatomia, Medição e Sumarização

Unidade 5 • Gerenciamento de Contextos

OBJETIVOS DE APRENDIZAGEM

- Compreender contexto ativo é tudo o que o modelo enxerga.
- Compreender context rot no agente que estamos construindo.
- Compreender mais tokens não significa uma resposta melhor.

GITHUB DA UNIDADE 5

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Contexto ativo é tudo o que o modelo enxerga
- Context rot no agente que estamos construindo
- Mais tokens não significam uma resposta melhor
- Contexto como orçamento
- Duas medições de tokens com papéis diferentes
- Instrumentando o consumo do contexto

Conteúdo da aula

Agora que o OpsPilot possui memória episódica e memória semântica, eu criei um novo problema que precisa ser tratado. Quanto mais a conversa cresce, mais histórico, memórias e informações podem voltar para o contexto do modelo. Isso aumenta o consumo de tokens e também pode piorar a qualidade da resposta. É o mesmo problema de context rot que discutimos no início do módulo, mas agora olhando para o agente que estamos construindo.

Para tornar esse crescimento visível, eu utilizo um script preparado para simular uma conversa longa durante um plantão. Ele executa vários turnos e mostra como o consumo vai se acumulando. A cada nova interação existe a mensagem atual, a resposta do modelo e o contexto anterior que precisa ser considerado. O resultado é que o custo por turno começa a crescer conforme a conversa fica maior.

Esse script é útil para demonstrar o problema, mas não é algo que eu recomendo executar repetidamente. Como ele faz várias chamadas ao modelo, pode consumir uma quantidade relevante de tokens e, dependendo do plano utilizado no OpenRouter, também pode aproximar a aplicação dos limites de uso. O objetivo é observar o comportamento uma vez e depois trabalhar sobre a arquitetura.

Contexto ativo é tudo o que o modelo enxerga

O primeiro conceito que eu preciso organizar é o de contexto ativo. Contexto ativo é tudo aquilo que eu monto e envio para o modelo em uma chamada.

No OpsPilot, esse contexto nasce de diferentes fontes. Existe o system prompt fixo, que contém instruções permanentes para o comportamento do agente. Existem as memórias do usuário recuperadas pelo mecanismo de recall. Existe o histórico recente da conversa. Existem as definições das tools que o agente pode utilizar. Por fim, existe a mensagem atual do usuário.

Essas fontes formam o orçamento que o modelo precisa processar antes de responder. No caso de uma estratégia como ReAct, existe ainda uma parcela adicional durante a própria execução: as observações das tools vão sendo acumuladas dentro daquele episódio e também passam a ocupar espaço no contexto.

Isso significa que a memória não é gratuita. Tool também não é gratuita. Histórico também não é gratuito. Mesmo informações fixas, como o system prompt e os schemas das ferramentas, entram em toda chamada e participam do custo total.

Context rot no agente que estamos construindo

Nas primeiras aulas, discutimos context rot olhando para ferramentas como Copilot. Quando uma conversa de desenvolvimento cresce demais, o agente começa a receber uma quantidade enorme de arquivos, instruções, mensagens e decisões anteriores. Em algum momento, parte desse conteúdo deixa de ajudar.

Agora o mesmo problema aparece dentro do OpsPilot. Se eu simplesmente continuar adicionando mensagens, memórias e observações sem nenhuma política de controle, o contexto vai inchar de forma contínua.

Esse crescimento tem duas consequências. A primeira é econômica, porque mais tokens no prompt significam mais processamento e potencialmente mais custo. A segunda é de qualidade, porque mais contexto não significa automaticamente mais inteligência.

Quando eu adiciono informação demais, a atenção do modelo fica diluída. Uma decisão importante pode ficar perdida no meio de dezenas de mensagens pouco relevantes. Esse comportamento é frequentemente associado ao efeito conhecido como *lost in the middle*: informações posicionadas no meio de um contexto muito grande podem receber menos atenção do que aquilo que está mais próximo do início ou do final.

Mais tokens não significam uma resposta melhor

É importante separar duas ideias. Gastar mais tokens não significa necessariamente que o modelo está raciocinando melhor. Ele pode simplesmente estar processando mais informação.

Se parte dessa informação não é necessária para responder à solicitação atual, eu estou pagando para o modelo ler ruído.

É como tentar executar uma tarefa importante enquanto, ao mesmo tempo, preciso prestar atenção em várias fontes sem relação direta com aquilo que estou fazendo. Quanto mais elementos competem pela atenção, mais difícil fica manter o foco naquilo que realmente importa.

Por isso, o objetivo não é construir o maior contexto possível. O objetivo é construir o menor contexto capaz de sustentar uma boa decisão.

A regra que passa a orientar o OpsPilot é simples: cada token no contexto precisa justificar a própria presença.

Contexto como orçamento

Quando eu começo a tratar contexto como orçamento, a pergunta muda. Em vez de perguntar tudo o que eu poderia colocar no prompt, eu passo a perguntar o que esta requisição realmente precisa para ser respondida.

Essa mudança de perspectiva influencia memória, histórico, tools e observações. Uma memória sem relação com a pergunta atual não deve entrar apenas porque existe. Uma mensagem antiga que já perdeu relevância não precisa continuar ocupando espaço integralmente. Uma conversa social pode ser descartada se não altera nenhuma decisão futura.

A partir daí entram técnicas como pruning, seleção de informações relevantes e costura de estado, ou *state stitching*. O objetivo é manter aquilo que sustenta a continuidade do agente e remover o que envelheceu mal.

Mas eu não quero otimizar baseado apenas em percepção. Antes de cortar o contexto, preciso medir o que está acontecendo.

Duas medições de tokens com papéis diferentes

Para administrar o orçamento, eu trabalho com dois tipos de medição de tokens.

A primeira é a medição real. Ela vem da própria resposta da API do modelo. O provedor informa um campo de *usage* com a quantidade de tokens contabilizada pelo tokenizador real utilizado naquela chamada.

Esse número é o que eu uso para saber quanto a requisição efetivamente consumiu.

A segunda medição é estimada. Ela acontece antes do envio. O objetivo não é substituir o valor real da API, mas me ajudar a decidir se o contexto está grande demais antes de pagar pela chamada.

Para essa estimativa, utilizo uma heurística simples de aproximadamente quatro caracteres por token. Não é uma contagem exata e pode variar conforme idioma, símbolos e tokenizador, mas é suficiente para orientar decisões de pruning antes do request.

Assim, a medição real serve para contabilização e observabilidade. A estimativa serve para planejamento e controle preventivo.

Instrumentando o consumo do contexto

Volto então ao Spec Kit para especificar uma nova feature de medição de contexto.

Quero uma função de estimativa de tokens dentro da área de context da aplicação, utilizando a heurística de quatro caracteres por token. Também quero capturar o usage real retornado pela integração com LangChain e pelo modelo.

No endpoint de chat, as métricas passam a incluir prompt tokens e um context breakdown.

O prompt tokens representa a contagem real disponível após a chamada. O context breakdown é uma estimativa dividida pelas fontes que formaram o contexto.

Dessa maneira, eu consigo saber não apenas que uma requisição utilizou milhares de tokens, mas também onde esse orçamento foi gasto. Posso enxergar quanto veio do sistema, quanto veio de memórias recuperadas, quanto veio do histórico de mensagens e quanto pertence às outras partes montadas para o turno.

Também atualizo o script de conversa longa para imprimir o consumo por turno e adiciono testes para essa instrumentação.

Medindo uma requisição real

Depois que a implementação termina, faço uma chamada ao endpoint de chat perguntando como está o plantão do usuário utilizado nos testes de memória.

A resposta agora inclui as novas métricas. Em uma das execuções da aula, o prompt chegou a aproximadamente 6.751 tokens.

Além do total, consigo observar o breakdown estimado das partes que formaram esse prompt.

Esse número muda completamente a discussão. Antes, eu sabia de forma abstrata que histórico e memória poderiam aumentar o contexto. Agora consigo medir uma requisição concreta e identificar a participação de cada fonte.

A partir dessa observabilidade, pruning deixa de ser uma otimização genérica e passa a ser uma decisão baseada em dados.

A janela recente já é uma forma de pruning

O OpsPilot já possui uma primeira forma de pruning porque a memória episódica não injeta a conversa inteira no prompt. Nós limitamos o histórico recente a uma janela de mensagens.

Essa regra impede que centenas de interações sejam enviadas integralmente a cada request.

O problema é que simplesmente cortar as mensagens antigas cria outro risco. Uma informação importante pode ter aparecido antes da janela atual.

Imagine que, em um turno anterior, a equipe decidiu que um freeze de deploy terminaria no dia 15. Se essa decisão estiver em uma mensagem que saiu da janela, o agente pode simplesmente esquecer que ela existe.

Pior ainda, imagine que mais tarde a equipe altere essa data para o dia 20. Se a informação antiga e a atualização forem cortadas sem nenhum mecanismo de retenção, o agente pode sugerir uma ação incompatível com a decisão operacional atual.

Portanto, pruning não pode significar apenas apagar. Eu preciso comprimir o passado sem perder aquilo que muda o comportamento futuro.

Sumarização do histórico

A solução que introduzo é a sumarização do histórico.

Quando mensagens antigas saem da janela recente, elas podem ser transformadas em um resumo compacto. O modelo deixa de receber cada frase completa daquele trecho, mas continua recebendo as decisões e fatos que realmente precisam sobreviver.

Esse resumo deve preservar decisões tomadas, fatos estabelecidos e pendências abertas. Também precisa manter nomes, datas, casos importantes, preferências do usuário e mudanças relevantes no estado de incidentes.

Cumprimentos, conversa social e informações que não alteram nenhuma decisão futura podem ser descartados.

A sumarização funciona como uma compressão semântica do episódio. Em vez de carregar dezenas de mensagens antigas, eu carrego um bloco curto que representa aquilo que ainda importa.

Especificando a sumarização incremental

Crio então uma nova especificação no Spec Kit para a sumarização de histórico.

A persistência ganha uma estrutura de conversation summaries. As mensagens recentes continuam disponíveis integralmente, enquanto blocos que saem dessa janela são resumidos.

Quero um resumo pequeno, por volta de 150 tokens.

Também existe uma regra importante: o resumo novo precisa incorporar o resumo anterior. Dessa forma, eu não perco uma decisão simplesmente porque ela foi registrada em um bloco de mensagens ainda mais antigo.

Esse comportamento cria uma sumarização incremental. O estado comprimido vai sendo carregado para frente conforme novas partes da conversa envelhecem.

Eu também não quero chamar o sumariador em toda request. Isso aumentaria custo desnecessariamente. A regra definida é atualizar o resumo apenas quando um novo conjunto de mensagens realmente sai da janela. Na especificação da aula, essa atualização acontece quando oito novas mensagens são expulsas do histórico recente.

O que o prompt de sumarização deve preservar

Antes do implement, forneço uma referência para o prompt do sumariador.

A instrução é comprimir o trecho de conversa em no máximo aproximadamente 150 tokens, preservando obrigatoriamente decisões tomadas, fatos estabelecidos, nomes, datas, casos e preferências.

Também quero preservar incidentes abertos ou resolvidos e pendências que ainda precisem de ação.

Ao mesmo tempo, peço para descartar cumprimentos e conversa social.

Se já existir um resumo anterior, o novo resultado deve incorporá-lo. O retorno precisa ser compacto, em tópicos telegráficos, porque o objetivo é justamente economizar contexto.

Essa referência orienta o agente de código a tratar o resumo como um estado operacional, e não como uma simples descrição genérica da conversa.

Mesclando o resumo ao contexto ativo

Depois de implementada, a sumarização passa a entrar no contexto junto com a janela recente.

Isso cria duas camadas complementares. As mensagens mais novas permanecem integrais porque contêm detalhes que ainda podem ser necessários. O passado mais distante é representado por um resumo compacto.

Quando novas mensagens envelhecem, o sistema gera uma nova síntese combinando o resumo anterior com o novo bloco que saiu da janela.

Esse mecanismo funciona como um rollup da conversa. A cada ciclo, eu carrego para frente apenas o estado relevante, em vez de carregar toda a transcrição desde o primeiro turno.

Validando a preservação de uma decisão antiga

Depois da implementação, utilizo novamente o cenário de conversa longa para gerar histórico suficiente para a sumarização.

Não preciso deixar o script rodar todas as interações possíveis. Algumas rodadas já são suficientes para verificar o comportamento.

Em seguida, reinicio o servidor e faço uma consulta que exige contexto daquele episódio.

O resumo recuperado preserva uma das decisões consideradas relevantes durante a simulação: o freeze de deploy permanece ativo até o dia 15.

Essa informação já não precisa estar disponível como mensagem integral dentro da janela recente. Ela sobrevive porque foi extraída para o resumo.

Na execução demonstrada, o sistema utilizou oito mensagens no processo que produziu essa síntese e descartou o restante do conteúdo que não precisava permanecer integralmente.

Medição, pruning e sumarização trabalhando juntos

Com essa feature, o gerenciamento de contexto do OpsPilot começa a ficar mais completo.

O contexto continua vindo do system prompt, das memórias recuperadas, do histórico recente, das definições de tools e da mensagem atual. Em estratégias iterativas, ainda existem as observações produzidas durante a execução.

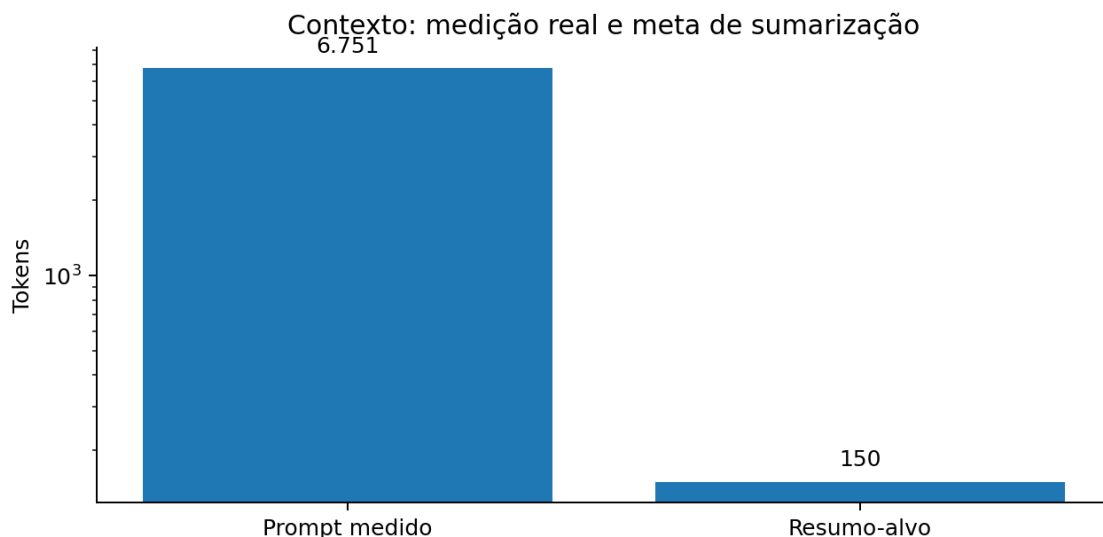
Agora, porém, eu consigo medir o consumo real da chamada e também estimar o peso de cada fonte antes do envio.

O histórico recente possui uma janela limitada, evitando o crescimento infinito. As mensagens que envelhecem não são simplesmente esquecidas: partes relevantes são comprimidas em um resumo incremental que continua sendo injetado no contexto.

Com isso, context rot deixa de ser apenas um risco abstrato. Eu tenho mecanismos concretos para observar o crescimento, cortar conteúdo, preservar estado importante e verificar quanto orçamento cada decisão está consumindo.

A recomendação para os exercícios continua sendo executar o script de conversa longa com moderação. Uma execução completa é suficiente para observar a evolução dos turnos e depois validar a sumarização. O objetivo não é gastar tokens repetindo a simulação, mas enxergar como o orçamento de contexto se comporta e como a arquitetura reage quando a conversa cresce.

Leitura visual do orçamento de contexto



O material registra aproximadamente 6.751 tokens em uma execução e uma meta de cerca de 150 tokens para o resumo.

A escala logarítmica ajuda a visualizar a diferença entre o prompt medido (aprox. 6.751 tokens) e a meta de cerca de 150 tokens para o resumo.

NA PRÁTICA

Abra o snapshot da Unidade 5 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Contexto ativo é tudo o que o modelo enxerga” no fluxo apresentado?
- Que decisão de engenharia está associada a “Context rot no agente que estamos construindo” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Contexto ativo é tudo o que o modelo enxerga.
- Context rot no agente que estamos construindo.
- Mais tokens não significam uma resposta melhor.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Context Stitching e os Paralelos com a Ferramenta

Unidade 5 • Gerenciamento de Contextos

OBJETIVOS DE APRENDIZAGEM

- Compreender a montagem de contexto como orçamento com prioridades.
- Compreender que cada fonte precisa de uma política própria.
- Compreender orçamento por seção.

GITHUB DA UNIDADE 5

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Montagem de contexto como orçamento com prioridades
- Cada fonte precisa de uma política própria
- Orçamento por seção
- Configurando budgets sem engessar a aplicação
- Especificando o ContextBuilder com o Spec Kit
- A referência para montagem das seções

Conteúdo da aula

Na aula anterior, eu comecei a tratar o contexto do OpsPilot como um orçamento. Passei a medir quanto cada requisição consome, separei o uso real informado pelo modelo da estimativa feita antes da chamada e introduzi uma política de sumarização para impedir que conversas longas cresçam indefinidamente. Agora falta uma peça que organiza todas essas fontes: o context stitching.

Context stitching é a costura do contexto. A cada requisição, eu não envio apenas uma mensagem isolada para o modelo. Eu monto um prompt a partir de diferentes retalhos de informação. No nosso caso, esses retalhos podem vir do system prompt, das memórias semânticas, do resumo persistido da conversa, da janela recente de mensagens e da mensagem atual.

Ter várias fontes é útil porque cada uma carrega um tipo diferente de informação. O problema aparece quando a soma de tudo isso ultrapassa o espaço que eu decidi reservar. A partir daí, não basta saber quanto custa. Eu preciso decidir o que permanece, o que encolhe e o que é descartado primeiro.

Montagem de contexto como orçamento com prioridades

A forma mais profissional de tratar essa montagem é pensar em cada seção como uma parte de um orçamento com prioridades.

Cada seção do contexto passa a ter três atributos. O primeiro é um teto de tokens. O segundo é uma prioridade, que ajuda a definir quem deve ser sacrificado primeiro quando o orçamento aperta. O terceiro é uma regra de corte, que determina como aquela seção deve encolher.

Esses três elementos resolvem problemas diferentes. O teto impede o crescimento sem controle. A prioridade protege informações que considero essenciais. A regra de corte evita aplicar o mesmo tipo de redução para fontes que possuem naturezas diferentes.

Eu não quero, por exemplo, cortar o system prompt da mesma forma que corto uma lista de memórias. Também não faz sentido remover aleatoriamente mensagens recentes quando já sei que as mais antigas são candidatas naturais a sair primeiro.

Cada fonte precisa de uma política própria

O system prompt é uma seção especial. Ele contém instruções que definem o comportamento do agente e, dentro do desenho desta aula, é tratado como intocável. Isso significa que ele não entra em uma política normal de pruning.

O resumo da conversa também recebe uma proteção maior. Ele já existe justamente porque comprimimos o passado. Se eu continuar cortando esse resumo sem critério, corro o risco de eliminar exatamente as decisões e fatos que a sumarização tentou preservar.

O histórico recente, por outro lado, pode encolher removendo primeiro as mensagens mais antigas. A própria ordem temporal nos dá uma regra natural de corte.

As memórias semânticas possuem outra característica. Elas já vêm acompanhadas por score de relevância. Quando preciso reduzir essa seção, posso descartar primeiro as memórias com menor score e manter aquelas que estão mais relacionadas à solicitação atual.

Essa diferença entre as seções é central no context stitching. O contexto não é um bloco homogêneo. Ele é composto por fontes com semânticas e níveis de importância diferentes.

Orçamento por seção

Na especificação desta aula, eu quero um ContextBuilder responsável por montar o prompt utilizado pelas estratégias.

Esse builder recebe orçamentos por seção. Os valores podem ser configurados por variáveis de ambiente para que eu consiga ajustar o comportamento sem alterar a implementação.

O system prompt continua protegido. O resumo recebe um orçamento próprio. O histórico recente recebe outro limite. As memórias semânticas recebem um teto separado.

Durante a explicação, utilizo valores pequenos para deixar o comportamento fácil de observar. O resumo pode trabalhar com uma faixa de aproximadamente duzentos tokens. A janela recente também recebe um teto. As memórias podem ter um orçamento em torno de trezentos tokens, embora esses números possam ser ajustados depois.

O importante não é decorar um valor específico. O importante é que cada fonte deixa de competir livremente com todas as outras. Eu sei antecipadamente quanto espaço estou disposto a conceder a cada parte do contexto.

Configurando budgets sem engessar a aplicação

Colocar os budgets em configuração é importante porque diferentes modelos e ambientes podem exigir limites diferentes.

Um modelo com uma janela de contexto muito maior não elimina a necessidade de orçamento, mas pode permitir limites mais confortáveis. Um ambiente com custo mais sensível pode trabalhar com budgets menores.

Também posso ajustar o comportamento conforme o tipo de agente. Uma aplicação de plantão pode dar mais espaço a incidentes e fatos recentes, enquanto outro domínio pode privilegiar documentos ou memória semântica.

Por isso, eu quero que a política esteja explícita, mas não codificada de forma rígida em todos os pontos da aplicação.

Especificando o ContextBuilder com o Spec Kit

Volto então ao Spec Kit e crio uma nova especificação para o ContextBuilder.

O objetivo é centralizar a montagem do prompt de todas as estratégias. Em vez de cada estratégia costurar system prompt, histórico, resumo e memórias por conta própria, quero uma única responsabilidade para esse trabalho.

Essa centralização evita divergências. Se ReAct montar o contexto de uma forma e Plan-and-Execute montar de outra, eu posso medir budgets diferentes para a mesma requisição sem perceber.

Com um builder compartilhado, as estratégias recebem o resultado de uma política única de contexto.

Também especifico testes com tetos propositalmente baixos. Assim, consigo verificar se o corte acontece na ordem correta e se cada seção respeita sua própria regra.

A referência para montagem das seções

Antes do implement, preparo uma referência para mostrar ao agente como quero que o contexto seja estruturado.

A primeira seção é o system prompt. Ela recebe o conteúdo fixo, o budget correspondente e uma política de não cortar.

A segunda seção é o resumo persistido da conversa. Ele é recuperado a partir do conversationId e também recebe proteção contra cortes arbitrários.

Depois vem o histórico. A aplicação recupera apenas uma quantidade limitada das mensagens recentes e, caso essa seção ultrapasse seu teto, as mais antigas são removidas primeiro.

A última seção principal são as memórias semânticas. O recall considera o userId e a mensagem atual, recupera um conjunto pequeno de memórias relevantes e, se precisar reduzir, elimina primeiro aquelas com menor score.

A mensagem atual do usuário continua sendo a entrada que a estratégia precisa responder. Ela não deve desaparecer apenas para que o contexto anterior caiba.

Fit to budget

Depois de reunir as seções, aplico uma operação de fit to budget em cada uma delas.

Esse é o ponto em que o teto e a regra de corte deixam de ser apenas metadados e passam a controlar o conteúdo real.

Se o histórico estiver acima do limite, o builder reduz a quantidade de mensagens respeitando a ordem temporal definida.

Se as memórias ultrapassarem o orçamento, o builder elimina as menos relevantes.

Se uma seção protegida estiver acima de um valor esperado, o comportamento precisa respeitar o design daquela seção em vez de aplicar um corte genérico.

Depois desse ajuste, as seções são montadas em um contexto final que pode ser entregue às estratégias.

Breakdown por seção

O ContextBuilder também precisa devolver um breakdown por seção.

Na aula anterior, eu já comecei a medir o contexto por fonte. Agora essa medição fica ainda mais útil porque cada fonte possui orçamento próprio.

Eu consigo comparar o que foi permitido com aquilo que efetivamente entrou. Posso ver quanto system, summary, history e memories consumiram naquele request.

Esse breakdown transforma o budget em algo auditável. Se uma seção sempre fica muito abaixo do teto, talvez eu esteja reservando espaço demais. Se outra vive no limite, talvez eu precise rever a política de corte ou o valor configurado.

Implementando com o Spec Kit

Com a referência pronta, executo o implement do Spec Kit.

O agente cria a estrutura de budgets, resolve as seções, formata as memórias e aplica as regras de corte antes de montar o contexto final.

Também aparecem funções auxiliares para interpretar as configurações vindas do ambiente. Isso é importante porque variáveis de ambiente chegam como texto e precisam ser transformadas em valores utilizáveis de forma segura.

Depois da implementação, eu reviso principalmente o ContextBuilder para confirmar se as regras que defini realmente aparecem no fluxo gerado.

Ajustando as variáveis de ambiente

Na sequência, atualizo o arquivo de ambiente com os valores de budget que quero utilizar no teste.

Durante essa etapa encontro uma divergência comum em desenvolvimento com agentes: os nomes que a implementação gerou não ficaram exatamente iguais aos nomes que eu havia imaginado para as variáveis.

Esse tipo de detalhe reforça a importância da revisão. Se eu simplesmente adicionar variáveis com nomes diferentes daqueles que o código lê, a aplicação continua usando defaults e eu posso achar que estou testando um budget que nunca foi aplicado.

Faço os ajustes necessários para alinhar os nomes e deixo de considerar um budget específico para system, porque no design daquela implementação o system continua intocável.

Validando o stitching com uma conversa existente

Para testar o comportamento real, reutilizo um conversationId que havia sido criado pelo script de conversa longa.

Isso é útil porque aquela conversa já possui histórico, resumo e informações suficientes para exercitar diferentes fontes do contexto.

Faço uma chamada perguntando como está o plantão associado ao usuário utilizado nas aulas anteriores e observo o breakdown retornado.

Agora não estou apenas perguntando se o agente lembra. Estou verificando quanto de cada fonte foi costurado para que ele consiga responder.

Lendo os resultados dos budgets

Na execução demonstrada, o system prompt permanece acima de alguns valores menores que haviam sido usados em testes porque ele é protegido pela política escolhida. Isso é coerente com o design: eu não quero truncar instruções fundamentais apenas para obedecer cegamente a um número.

O histórico fica dentro do orçamento configurado. O resumo também consegue se manter no limite definido. As memórias utilizam apenas uma pequena parcela do espaço reservado porque o recall trouxe pouco conteúdo relevante naquele turno.

Esse resultado mostra uma característica importante: budget é teto, não meta. Se eu reservar cem tokens para memórias e a requisição só precisa de dez, eu não devo preencher os outros noventa com informação irrelevante.

O objetivo não é usar todo o espaço disponível. O objetivo é impedir que qualquer fonte cresça além daquilo que considero aceitável.

Context stitching e continuidade da conversa

O conversationId continua sendo importante nesse desenho porque ele identifica o episódio que quero continuar.

Quando envio esse identificador, o sistema consegue recuperar o resumo e a janela recente daquela conversa. Ao mesmo tempo, o userId permite buscar as memórias semânticas relacionadas ao usuário.

O ContextBuilder costura essas fontes com o system prompt e a nova mensagem.

Em termos de experiência, é parecido com voltar a uma conversa anterior de um assistente e continuar de onde parei. A diferença é que, no OpsPilot, estou construindo explicitamente as regras que decidem quanto daquela conversa, quanto de memória e quanto de resumo vai voltar para o modelo.

Paralelos com as ferramentas que utilizamos

Esse mecanismo cria um paralelo direto com as ferramentas de agentes que usamos ao longo do módulo.

Quando trabalho em uma sessão de Copilot ou outro agente de código, aquela ferramenta também precisa decidir quais mensagens, instruções, arquivos e resultados anteriores permanecem disponíveis no contexto.

Quando uma conversa cresce demais, ela não pode simplesmente despejar tudo indefinidamente no modelo sem nenhum gerenciamento. As ferramentas também precisam selecionar, resumir, recuperar e priorizar.

No OpsPilot, estou implementando de forma explícita os mesmos tipos de decisões que antes eu observava apenas como usuário da ferramenta.

Isso ajuda a entender por que uma nova janela de contexto pode se comportar de maneira diferente, por que determinadas instruções precisam estar persistidas e por que informações antigas às vezes desaparecem quando não foram resumidas ou recuperadas corretamente.

Prioridade e qualidade do raciocínio

O budget não existe apenas para economizar tokens. Ele protege a qualidade do raciocínio.

Se eu permitir que histórico e memórias ocupem todo o espaço disponível, posso empurrar a informação atual para um contexto excessivamente ruidoso.

Quando defino prioridades, estou dizendo ao sistema quais informações têm maior chance de mudar a decisão.

O system prompt preserva comportamento e regras. O resumo mantém decisões duráveis do passado. O histórico recente mantém detalhes do episódio atual. As memórias trazem preferências e fatos semanticamente relevantes.

A mensagem atual define o problema que precisa ser resolvido agora. O stitching existe para equilibrar essas fontes em vez de permitir que uma delas domine todas as outras.

Budgets como decisão de arquitetura

Os valores utilizados na aula são exemplos e podem ser alterados.

Posso aumentar o orçamento do resumo, reduzir memórias, trabalhar com uma janela maior de histórico ou adaptar a política conforme o modelo escolhido.

O ponto que eu não quero perder é o desenho. Um agente precisa saber de onde o contexto vem, quanto cada fonte pode consumir, qual informação é protegida e qual regra será usada quando o limite for atingido.

Depois que essas decisões ficam explícitas e mensuráveis, eu consigo evoluir o agente sem depender apenas de tentativa e erro.

A memória global do OpsPilot passa a ter custos e limites definidos. Isso reduz a chance de crescimento descontrolado, melhora a previsibilidade do consumo e ajuda o modelo a receber um contexto mais concentrado naquilo que realmente importa para o turno atual.

NA PRÁTICA

Abra o snapshot da Unidade 5 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Montagem de contexto como orçamento com prioridades” no fluxo apresentado?
- Que decisão de engenharia está associada a “Cada fonte precisa de uma política própria” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Montagem de contexto como orçamento com prioridades.
- Cada fonte precisa de uma política própria.
- Orçamento por seção.

Revisão da Unidade 5

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE					
System prompt →	Memórias →	Histórico →	Tools →	ContextBuilder →	Budget / Pruning / Summary

CHECKLIST DA UNIDADE
- Revisar a Aula 1: O Contexto como Orçamento: Anatomia, Medição e Sumarização - Revisar a Aula 2: Context Stitching e os Paralelos com a Ferramenta

GITHUB DA UNIDADE 5
Abrir no GitHub • Ler UNIDADE.md

Unidade 6

LangGraph e Workflows Complexos

Orquestração do fluxo de produção com LangGraph, roteamento automático entre estratégias e fallback de modelos.

FLUXO VISUAL DA UNIDADE					
Entrada →	Router →	LangGraph →	Estratégia →	Fallback de modelo →	Resposta

GITHUB DA UNIDADE 6	
Abrir no GitHub	• Ler UNIDADE.md

Aula	Tema
1	LangGraph e Fallback de modelo

Aula 1 — LangGraph e Fallback de modelo

Unidade 6 • LangGraph e Workflows Complexos

OBJETIVOS DE APRENDIZAGEM

- Explicar por que usar um grafo de execução.
- Compreender o roteador deixa de exigir conhecimento do cliente.
- Compreender um grafo unificado para produção.

GITHUB DA UNIDADE 6

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Por que usar um grafo de execução
- O roteador deixa de exigir conhecimento do cliente
- Um grafo unificado para produção
- Roteamento com saída estruturada
- A estratégia continua podendo ser sobrescrita
- Montando o grafo com LangGraph

Conteúdo da aula

Neste ponto, o OpsPilot já possui bastante estrutura. Eu tenho estratégias de raciocínio, tools que operam sobre um banco SQLite, integração via MCP, memória episódica, memória semântica e um mecanismo de context stitching com orçamento por seção. O problema é que a composição ainda está muito manual.

Até agora, quem escolhe a estratégia de raciocínio é o próprio cliente. No corpo da requisição eu informo se quero ReAct ou Plan-and-Execute. Para um exercício controlado isso funciona, mas não é uma boa experiência de produção. Um plantonista às três horas da manhã não deveria precisar saber qual padrão de raciocínio é mais adequado para resolver uma solicitação.

Existe ainda outro problema. Se o modelo falhar no meio de uma execução, eu preciso decidir o que acontece em seguida. Até este momento, a aplicação ainda não possui uma política de fallback suficientemente explícita para trocar de modelo quando o principal deixa de responder.

A partir daqui, eu quero organizar essas decisões em um grafo de produção usando LangGraph. O grafo passa a representar de forma explícita as etapas que compõem uma execução do agente.

Por que usar um grafo de execução

Um grafo é um mapa de etapas ligadas por transições. Eu posso imaginar cada etapa como um nó e as ligações entre elas como caminhos possíveis de execução.

O estado percorre esses nós do início ao fim. Um nó pode tomar uma decisão e encaminhar a execução para um caminho diferente dependendo do valor que foi produzido.

Isso é particularmente útil para o OpsPilot porque eu já tenho comportamentos distintos que antes estavam ligados por condicionais e composição manual. Com LangGraph, consigo transformar essas decisões em uma estrutura visível e rastreável.

A primeira vantagem é que o fluxo se desenha no próprio código. A segunda é que o roteamento passa a ter entradas e saídas bem definidas. A terceira é o tracing, porque consigo saber por quais nós a execução passou.

Também ganho um lugar mais claro para aplicar resiliência. Posso configurar retry ou fallback em um ponto específico do grafo, em vez de espalhar tratamento de falha por toda a aplicação.

Outro benefício é o paralelismo mais declarativo. Ramos independentes podem, em determinados desenhos, executar em paralelo. Nesta aula, porém, eu não aprofundo o raio X do grafo nem o paralelismo em ondas. Esses pontos ficam como extensão e exercício para quem quiser avançar.

Eu poderia montar parte desse comportamento com uma sequência de if e else. O problema é que, conforme o fluxo cresce, esse tipo de composição fica mais difícil de visualizar, testar e instrumentar. O grafo não elimina condicionais, mas dá a elas um lugar explícito dentro de um modelo de execução.

O roteador deixa de exigir conhecimento do cliente

O primeiro nó importante será o roteador.

Esse roteador funciona como um classificador. Ele recebe o pedido do usuário, analisa a natureza da solicitação e escolhe automaticamente qual estratégia deve ser utilizada.

Com isso, o cliente deixa de ser responsável por decidir se uma pergunta exige ReAct, Plan-and-Execute ou Reflection.

Uma consulta simples e pontual pode ser encaminhada para o ReAct. Um pedido que exige planejamento estruturado e execução de múltiplas etapas pode seguir para Plan-and-Execute. Outra classe de solicitação pode ser encaminhada para o fluxo de Reflection.

Essa decisão fica dentro do sistema, que possui mais contexto sobre as capacidades de cada estratégia que o usuário final deveria precisar conhecer.

Um grafo unificado para produção

Volto então ao Spec Kit e crio uma nova especificação para um grafo unificado de produção.

Quero um arquivo dedicado para esse ProductionGraph e defino os nós principais que vão compor a execução.

O fluxo começa com um nó de contexto. Depois vem o roteador. A partir dele, o estado pode seguir para ReAct, Plan-and-Execute ou Reflection. Independentemente da estratégia selecionada, a execução converge depois para um nó de resposta e então termina.

Essa convergência é importante porque as estratégias podem raciocinar de formas diferentes, mas a aplicação ainda precisa entregar uma resposta com um contrato consistente.

Também quero que cada evento de trace identifique o nó em que aconteceu. Assim, o trace deixa de mostrar apenas ações ou observações soltas e passa a indicar em qual parte do grafo cada evento foi produzido.

O estado compartilhado é a peça que conecta os nós. Informações como entrada original, rota escolhida, trace e resultado intermediário podem viajar pelo grafo sem que cada nó precise reconstruir todo o contexto do zero. Cada etapa lê aquilo de que precisa e devolve as alterações que serão consumidas nas etapas seguintes.

Roteamento com saída estruturada

Para que o roteador seja previsível, eu não quero que ele responda com texto livre.

A especificação utiliza saída estruturada. O modelo deve devolver uma rota e uma justificativa curta para essa escolha.

A rota fica restrita às estratégias conhecidas. A justificativa é uma frase que explica por que aquela solicitação foi encaminhada para aquele caminho.

Mais uma vez, Zod entra como fronteira de validação. O resultado do modelo não é tratado como confiável apenas porque foi produzido por uma LLM. Eu defino um schema e obrigo a resposta a respeitar os valores aceitos pelo grafo.

Essa justificativa também melhora a observabilidade. Quando olho uma execução, não vejo apenas que o sistema escolheu ReAct. Consigo ver por que o roteador considerou o ReAct apropriado naquele pedido.

A estratégia continua podendo ser sobrescrita

Mesmo com o roteamento automático, eu não elimino totalmente a possibilidade de o cliente informar uma estratégia.

O campo strategy no endpoint de chat passa a ser opcional. Se ele não vier, o roteador toma a decisão. Se vier preenchido, funciona como um override.

Isso é útil para testes, benchmark, depuração e cenários em que eu quero forçar uma estratégia específica para comparar comportamento.

O override precisa aparecer no trace. Eu não quero olhar para uma execução depois e acreditar que o roteador escolheu aquela estratégia quando, na verdade, ela foi imposta externamente.

Esse cuidado preserva a auditabilidade do sistema.

Montando o grafo com LangGraph

Depois de specification, plan e tasks, preparo uma referência antes do implement.

A estrutura que quero mostrar ao agente de código começa pela criação de um StateGraph associado ao estado do grafo.

Em seguida, registro os nós de contexto, roteador, ReAct, Plan-and-Execute, Reflection e resposta.

Depois defino as transições. O início aponta para o contexto. O contexto encaminha para o roteador. A partir do roteador existe uma aresta condicional que consulta a rota produzida e escolhe o próximo nó.

ReAct, Plan-and-Execute e Reflection apontam para o nó de resposta. O nó de resposta, por sua vez, aponta para o fim da execução.

O grafo é então compilado e passa a representar formalmente o fluxo que antes estava disperso em condicionais e composição manual.

Lendo o estado para decidir o próximo nó

A parte mais interessante é a aresta condicional do roteador.

O nó de roteamento produz um valor no estado. A transição lê esse valor e devolve o nome do próximo nó.

Se a rota for ReAct, o estado segue para ReAct. Se for Plan-and-Execute, segue para o respectivo nó. Se for Reflection, percorre o caminho correspondente.

Esse desenho deixa explícito que o grafo não é apenas um diagrama decorativo. O próprio estado controla a navegação.

Como cada etapa tem nome, também consigo associar eventos de trace ao nó atual e reconstruir o caminho percorrido durante uma execução.

Implementando o roteador

Na referência do roteador, começo pelo schema de decisão.

Ele contém a rota e uma razão textual. A razão precisa ser curta e justificar a escolha.

O router node recebe o estado atual, chama o modelo configurado com saída estruturada e fornece como entrada o system prompt do roteador e o pedido original do usuário.

Depois da resposta, adiciono ao trace um evento do nó roteador contendo a rota e a justificativa. Por fim, atualizo o estado com a rota escolhida.

Com esse exemplo, volto ao agente e executo o implement do Spec Kit para que ele complete o grafo seguindo a arquitetura especificada.

Validando a escolha automática de ReAct

Depois da implementação, faço uma chamada sem forçar manualmente a estratégia.

A pergunta é simples: quantos alertas críticos existem para o usuário que estamos utilizando nos testes.

Além da resposta normal, peço as métricas de roteamento, incluindo a rota escolhida e o motivo.

O grafo seleciona ReAct. A justificativa informa que se trata de uma consulta pontual para listar alertas críticos.

Esse resultado faz sentido porque não existe necessidade de um planejamento longo. O agente precisa consultar a informação e responder.

Validando a escolha de Plan-and-Execute

Em seguida, altero a solicitação. Peço que o agente monte um plano de resposta para um incidente de checkout, respeitando uma ordem.

Essa vez a execução demora mais, o que já é coerente com o comportamento que observamos anteriormente em Plan-and-Execute.

O roteador escolhe Plan-and-Execute e registra a justificativa. O pedido exige um plano de resposta com múltiplos passos, execução estruturada e uma ordem de atividades.

A decisão deixa de depender de o plantonista conhecer o nome do padrão. Ele expressa a intenção em linguagem natural e o grafo escolhe o caminho de raciocínio.

Esse é o objetivo principal desta etapa: tirar conhecimento de implementação do cliente e colocar essa decisão dentro da arquitetura do agente.

Roteamento e disponibilidade são problemas diferentes

É importante não misturar duas decisões. O roteador escolhe a estratégia de raciocínio adequada para a intenção do usuário. O fallback de modelo resolve outro tipo de problema: o que fazer quando a infraestrutura de inferência que deveria executar aquela estratégia não está disponível.

Eu posso escolher corretamente ReAct e ainda assim falhar porque o modelo principal ficou indisponível. Da mesma forma, posso ter um modelo saudável e selecionar uma estratégia inadequada. Por isso, roteamento e resiliência aparecem como preocupações separadas dentro do desenho de produção.

O grafo me ajuda a tornar a primeira decisão explícita. A fábrica de modelos e a política de retry e fallback cuidam da segunda. Mantendo essas responsabilidades separadas, consigo observar e evoluir cada comportamento sem transformar todo erro em um problema genérico do agente.

Resiliência de modelo

Depois do roteamento, introduzo o próximo ponto de produção: resiliência de modelo.

O fato de o grafo escolher corretamente uma estratégia não significa que o modelo utilizado naquela execução estará sempre disponível.

Como estamos trabalhando com OpenRouter e, em muitos testes, utilizando seleção de modelos gratuitos, essa falha pode não aparecer de forma tão evidente. Mesmo assim, uma aplicação real precisa possuir uma política explícita para quando o modelo primário falha.

A ideia é configurar um modelo principal e um modelo de fallback. A fábrica de modelos passa a conhecer os dois.

O modelo primário recebe uma política de retry limitada. Se continuar falhando depois das tentativas previstas, a execução pode cair para o modelo secundário, que também possui seu próprio limite de tentativas.

Retry limitado antes do fallback

Na referência apresentada, tanto o modelo principal quanto o backup trabalham com um número limitado de retries, em torno de duas tentativas.

Esse limite segue a mesma filosofia de todos os outros guardrails do projeto. Eu não quero retry infinito.

Uma indisponibilidade persistente não deve transformar uma chamada em um processo que fica tentando indefinidamente. Retry precisa existir para falhas transitórias, mas sempre dentro de um teto conhecido.

Se o primário esgota suas tentativas, o fallback recebe a oportunidade de continuar a execução.

Trace e métricas do modelo utilizado

A troca de modelo também precisa ser observável.

Quero um evento de trace quando o fallback acontece. Assim, consigo saber que determinada resposta não foi gerada pelo modelo normalmente configurado.

Também quero uma métrica indicando qual modelo foi efetivamente utilizado.

Essas informações são importantes para análise de qualidade, custo e estabilidade. Se o fallback começa a aparecer com frequência, talvez o problema não esteja no agente, mas na disponibilidade do provedor principal.

Da mesma forma, respostas diferentes podem ser explicadas pela troca de modelo durante uma falha.

Falha total e status 503

Existe ainda o cenário em que nem o modelo principal nem o fallback conseguem responder.

Nesse caso, a aplicação não deve fingir que existe uma resposta válida. A especificação prevê que a falha total seja convertida em um erro de indisponibilidade, usando status HTTP 503.

Esse status representa melhor a natureza do problema do que um erro genérico. O serviço está temporariamente incapaz de concluir a solicitação porque os modelos necessários não estão disponíveis.

O importante é que essa condição também esteja prevista no desenho, em vez de aparecer como uma exceção inesperada em produção.

Nesta aula, essa política de fallback fica especificada e exemplificada como o próximo ajuste da arquitetura. O grafo de roteamento é o comportamento que eu efetivamente implemento e valido com as chamadas de teste. A parte de fallback é preparada para a evolução seguinte, com os limites e sinais de observabilidade já definidos.

NA PRÁTICA

Abra o snapshot da Unidade 6 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que usar um grafo de execução” no fluxo apresentado?
- Que decisão de engenharia está associada a “O roteador deixa de exigir conhecimento do cliente” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Por que usar um grafo de execução.
- O roteador deixa de exigir conhecimento do cliente.
- Um grafo unificado para produção.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 6

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE					
Entrada →	Router →	LangGraph →	Estratégia →	Fallback de modelo →	Resposta

CHECKLIST DA UNIDADE
Revisar a Aula 1: LangGraph e Fallback de modelo

GITHUB DA UNIDADE 6
Abrir no GitHub • Ler UNIDADE.md

Unidade 7

Observabilidade e Limites de Autonomia

Observabilidade, auditoria e governança: traces persistidos, logs estruturados, métricas e limites explícitos de autonomia.

FLUXO VISUAL DA UNIDADE					
Request ID →	Trace →	Logs →	Métricas →	Matriz de autonomia →	Auditoria

GITHUB DA UNIDADE 7	
Abrir no GitHub • Ler UNIDADE.md	

Aula	Tema
1	Criando Estratégias de Observabilidade

Aula 1 — Criando Estratégias de Observabilidade

Unidade 7 • Observabilidade e Limites de Autonomia

OBJETIVOS DE APRENDIZAGEM

- Compreender observabilidade em um sistema probabilístico.
- Compreender três sinais principais: traces, logs e métricas.
- Compreender a matriz de autonomia.

GITHUB DA UNIDADE 7

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Observabilidade em um sistema probabilístico
- Três sinais principais: traces, logs e métricas
- A matriz de autonomia
- Request ID como eixo da auditoria
- Especificando traces persistidos e logs JSON
- Consultando uma execução pelo requestId

Conteúdo da aula

Neste ponto, o OpsPilot já executa bastante coisa de forma autônoma. Ele possui roteamento entre estratégias, fallback de modelo, persistência em SQLite, memória, context stitching, tools e integração via MCP. Quanto mais autonomia eu adiciono, mais importante fica responder a uma pergunta básica de produção: quem está observando essa máquina e como eu reconstruo aquilo que aconteceu dentro de uma requisição.

Quando uma execução demora, consome muitos tokens, escolhe uma estratégia inesperada ou altera o estado de um incidente, eu preciso enxergar o caminho percorrido. Não quero depender de abrir o código ou conectar um debugger para entender cada caso. Quero que o próprio sistema emita sinais suficientes para que eu consiga reconstruir o que aconteceu.

É aqui que entra observabilidade. No software tradicional, a observabilidade é a capacidade de compreender o estado interno de um sistema a partir dos sinais que ele produz. Em agentes, esse problema ganha uma camada adicional porque não basta auditar o que aconteceu. Eu também preciso registrar evidências sobre por que o modelo escolheu determinado caminho.

Observabilidade em um sistema probabilístico

Em uma aplicação convencional, muitas decisões são determinísticas. Se uma condição é verdadeira, determinado trecho de código é executado. Em um agente, parte da decisão vem de um modelo probabilístico. Duas execuções parecidas podem produzir trajetórias diferentes.

Por isso, a auditoria precisa guardar mais do que status de sucesso ou erro. Quero saber qual rota foi escolhida, qual modelo foi usado, quais tools foram chamadas, que observações retornaram e quais justificativas influenciaram as transições do grafo.

Esse desenho não começa nesta aula. Ao longo das etapas anteriores, eu já fui adicionando pequenos elementos de explicabilidade. O roteador registra uma reason para justificar por que escolheu ReAct ou Plan-and-Execute. O crítico produz critique e feedback. Reflection registra se aprovou ou rejeitou uma resposta. Esses elementos agora passam a ter valor ainda maior porque podem compor uma trilha persistente.

Não estou tentando expor raciocínio interno irrestrito do modelo. O que quero são justificativas operacionais e eventos de execução que possam ser auditados depois.

Três sinais principais: traces, logs e métricas

Para tornar o OpsPilot observável, eu quero trabalhar com três sinais principais.

O primeiro são traces persistidos. O trace representa a sequência de eventos de uma requisição. Ele permite reconstruir quais nós do grafo foram executados, quais decisões aconteceram e em que ordem.

O segundo são logs estruturados. Em vez de mensagens livres espalhadas pelo terminal, quero eventos em JSON, com campos conhecidos e fáceis de processar por ferramentas de observabilidade.

O terceiro são métricas. Elas respondem perguntas agregadas: quantas requisições aconteceram, quantos erros tivemos, quantos tokens foram consumidos, qual foi a latência, quais rotas foram mais utilizadas e qual modelo atendeu cada execução.

Esses sinais se complementam. Métrica mostra tendência. Log mostra um evento. Trace mostra a trajetória completa de uma execução.

A matriz de autonomia

Antes de implementar a persistência da observabilidade, eu introduzo uma ideia de governança que ajuda a classificar as ações do agente: a matriz de autonomia.

Toda ação possível pode cair em uma de quatro faixas.

Na primeira faixa, o agente decide sozinho. São operações de baixo risco e normalmente reversíveis, como consultar alertas ou buscar informações.

Na segunda faixa, o agente também pode decidir e executar sozinho, mas precisa registrar de forma auditável o que fez. Abrir ou resolver um incidente entra bem nessa categoria. A ação é reversível, mas muda o estado do sistema e, por isso, eu quero um rastro persistente.

Na terceira faixa, o agente precisa pedir aprovação. Ações destrutivas, remoções, silenciamento de recursos, execução de comandos sensíveis, exclusão de histórico ou gastos acima de um teto são exemplos de capacidades que não deveriam acontecer sem confirmação.

A quarta faixa representa ações proibidas. Apagar a própria trilha de auditoria ou vaziar dados de usuário não deve ser algo que o agente consiga fazer nem mesmo mediante uma simples aprovação.

Essa matriz ajuda a ligar observabilidade com autonomia. Quanto maior o impacto de uma ação, mais importante fica controlar permissão, registro e evidência.

Request ID como eixo da auditoria

Para reconstruir uma execução, preciso de um identificador que conecte todos os sinais relacionados à mesma requisição.

Por isso, o endpoint de chat passa a trabalhar com requestId. Esse identificador pode vir no corpo ou em um header específico e também é devolvido ao cliente.

A partir dele, consigo ligar a resposta HTTP, o registro persistido, os eventos de trace, os logs e as métricas daquela execução.

Sem esse vínculo, eu poderia ter centenas de eventos corretos individualmente, mas não conseguiria dizer quais pertencem ao mesmo pedido do usuário.

O requestId funciona como a chave de correlação da observabilidade.

Especificando traces persistidos e logs JSON

Volto ao Spec Kit e crio uma nova especificação para traces persistidos e logs estruturados.

No SQLite, quero uma estrutura para requests e outra para trace events.

O registro principal da request deve guardar informações como usuário, rota escolhida, modelo utilizado, métricas de tokens, latência e status.

Os eventos de trace ficam relacionados ao requestId e precisam registrar a sequência, o nó em que aconteceram, o tipo do evento e um payload com os metadados relevantes.

Também quero um logger dedicado na área de observabilidade da aplicação. Cada evento deve ser emitido como uma linha JSON.

A regra é registrar apenas metadados necessários. Eu não quero transformar o log estruturado em vazamento de prompt, segredo ou conteúdo sensível. Observabilidade precisa respeitar a mesma disciplina de segurança que aplicamos às outras fronteiras.

Consultando uma execução pelo requestId

Além de persistir os dados, quero conseguir consultá-los depois.

A especificação adiciona uma rota que recebe o requestId e devolve o registro da execução junto com seus eventos de trace ordenados.

Esse endpoint funciona como uma visão forense da requisição. Eu consigo partir de um identificador retornado ao cliente e recuperar a sequência que levou à resposta.

A ordenação dos eventos é essencial. Um trace sem sequência temporal perde grande parte do valor, porque eu preciso saber qual decisão aconteceu antes de qual ação.

Referência no nó de resposta

Depois de specification, plan e tasks, preparo uma referência para mostrar onde quero persistir a auditoria.

O nó de resposta é um ponto conveniente porque a execução já possui resultado, métricas, rota e trace consolidados.

Nesse momento, salvo o requestId, userId, rota escolhida, modelo utilizado, prompt tokens, latência e o status da execução.

Depois percorro os eventos de trace e persisto cada um associado ao mesmo requestId, preservando a sequência, o nó, o tipo e o payload.

Também emito um log estruturado com informações como requestId, nó atual, tipo de evento, rota e tokens.

A referência não precisa cobrir toda a implementação. Ela apenas mostra ao agente de código o formato de auditoria que eu quero que ele replique.

Persistência e reconstrução do caminho

Depois de executar o implement do Spec Kit, reinicio o servidor e faço uma chamada prática.

Peço ao agente para resolver o incidente mais antigo. Esse é um cenário interessante porque exige mais de uma etapa e tende a passar por Plan-and-Execute.

Quando a requisição termina, guardo o requestId retornado. Em seguida, faço uma segunda chamada consultando exatamente aquele identificador.

Agora consigo enxergar a execução completa sem precisar reproduzir o problema ou abrir o debugger.

Vejo quando a requisição foi criada, seu status, qual conversationId estava associado, quem era o usuário e quais métricas foram produzidas.

Auditando rota, modelo e orçamento de contexto

Na execução demonstrada, consigo verificar que a estratégia escolhida foi Plan-and-Execute.

A justificativa também aparece: o pedido exigia primeiro listar incidentes para encontrar o mais antigo e depois resolver o item selecionado. Isso caracteriza uma tarefa com múltiplas etapas.

Também consigo confirmar qual modelo foi utilizado. No exemplo, o registro mostra o modelo gratuito selecionado pelo OpenRouter.

As métricas de contexto permanecem disponíveis na auditoria. O context breakdown mostra quanto foi usado em cada seção e permite verificar se os budgets configurados nas aulas anteriores foram respeitados.

A janela de mensagens, por exemplo, continua dentro do limite previsto. Isso conecta observabilidade ao trabalho de context engineering: eu não apenas defini orçamentos, agora consigo provar em uma execução real se eles foram cumpridos.

Contando chamadas e latência

A auditoria também registra o número de chamadas ao LLM e a latência total.

Na execução da aula, a requisição levou aproximadamente um minuto e meio e utilizou várias chamadas ao modelo.

Esses números ajudam a explicar por que uma estratégia pode parecer lenta. Não preciso dizer apenas que Plan-and-Execute demora mais. Posso observar quantas interações ele realizou e comparar esse comportamento com outras rotas.

Essa informação também será útil mais tarde para decisões de custo e otimização.

Lendo o trace nó a nó

A parte mais útil da auditoria aparece quando observo o trace completo.

O roteador identifica que o pedido é multipasso e seleciona Plan-and-Execute.

Depois o fluxo registra o planejamento: listar os incidentes, identificar o mais antigo e resolver aquele incidente usando o identificador obtido.

Os eventos seguintes mostram as ações executadas, o contexto utilizado e as transições internas até a resposta final.

Por último, vejo a resposta entregue ao usuário, informando qual incidente foi encontrado e resolvido, incluindo serviço, severidade e estado.

Esse trace transforma uma decisão probabilística em uma trajetória auditável. Eu não preciso conhecer todos os mecanismos internos do modelo para saber quais decisões operacionais foram registradas pelo agente.

Observabilidade sem abrir o código

Esse é o comportamento que eu procuro quando falo de observabilidade.

Eu consigo investigar uma requisição apenas pelos sinais emitidos pelo sistema. Não preciso abrir o arquivo do roteador para adivinhar a estratégia. Não preciso colocar breakpoint na store para descobrir se um incidente foi alterado.

A requisição persistida e seus eventos contam a história necessária para reconstruir o caso.

Essa capacidade é especialmente importante quando o agente começa a operar em produção, porque o problema pode ter acontecido horas antes e em uma execução que eu não consigo reproduzir exatamente.

Métricas agregadas para as últimas 24 horas

Depois de validar a auditoria por requestId, adiciono uma visão agregada.

Peço ao agente de código uma rota de estatísticas sobre as requests das últimas 24 horas.

Quero informações como total de chamadas, quantidade de erros, tokens consumidos e custo estimado.

Também quero latências em percentis, principalmente p50 e p95, agrupadas por rota e modelo.

Essas medidas ajudam a sair do caso individual e observar comportamento operacional.

O p50 mostra uma região mais típica da latência. O p95 ajuda a enxergar a cauda, onde estão as requisições mais lentas que podem afetar a experiência do plantonista.

Custo, rota e modelo

Ao consultar a nova rota, consigo enxergar quantas requests aconteceram na janela observada, quantos erros existiram e quantos tokens foram consumidos.

Também consigo agrupar por rota e descobrir quantas vezes Plan-and-Execute foi utilizado em comparação com outras estratégias.

O agrupamento por modelo mostra qual provedor ou modelo respondeu às requisições.

Com uma tabela de preços, posso transformar tokens em estimativa financeira e começar a acompanhar custo ao lado de latência e taxa de erro.

Essa visão permite decisões mais maduras. Se uma rota cara aparece com frequência em tarefas simples, talvez o roteador precise ser ajustado. Se um modelo possui p95 muito alto, talvez o fallback ou a configuração principal precise ser revisto.

Da auditoria individual para operação contínua

Com os traces persistidos e as métricas agregadas, o OpsPilot começa a emitir sinais suficientes para uma camada de observabilidade mais completa.

Eu poderia conectar essas informações a ferramentas como Grafana ou a outro stack de monitoramento, mas isso fica como extensão. O ponto desta aula é garantir que os dados existam de forma estruturada e possam ser consultados.

A partir daí, consigo reconstruir decisões individuais e também acompanhar tendências das últimas horas.

As justificativas adicionadas em aulas anteriores, os eventos do grafo, o uso de modelo, o orçamento de contexto, a latência, os tokens e o estado final da operação passam a fazer parte da mesma história auditável.

Isso cria uma base importante para melhorar o agente posteriormente. Se eu quiser entender por que uma rota está lenta, por que o custo aumentou ou por que uma decisão foi tomada, agora tenho evidências persistidas para investigar em vez de depender apenas da memória de quem acompanhou a execução.

NA PRÁTICA

Abra o snapshot da Unidade 7 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Observabilidade em um sistema probabilístico” no fluxo apresentado?
- Que decisão de engenharia está associada a “Três sinais principais: traces, logs e métricas” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Observabilidade em um sistema probabilístico.
- Três sinais principais: traces, logs e métricas.
- A matriz de autonomia.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 7

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE					
Request ID →	Trace →	Logs →	Métricas →	Matriz de autonomia →	Auditoria

CHECKLIST DA UNIDADE
Revisar a Aula 1: Criando Estratégias de Observabilidade

GITHUB DA UNIDADE 7
Abrir no GitHub • Ler UNIDADE.md

Unidade 8

Projeto Prático: OpsPilot Publicado

Construção e publicação da War Room, conectando uma interface web ao backend existente sem duplicar a lógica do agente.

FLUXO VISUAL DA UNIDADE					
War Room →	API →	OpsPilot →	Memória + Store + Tools →	GitHub Pages →	Cloudflare Tunnel

GITHUB DA UNIDADE 8	
Abrir no GitHub • Ler UNIDADE.md	

Aula	Tema
1	Implementando a War Room e Modificando a Memória do Agente
2	Publicando a War Room no GitHub Pages

Aula 1 — Implementando a War Room e Modificando a Memória do Agente

Unidade 8 • Projeto Prático: OpsPilot Publicado

OBJETIVOS DE APRENDIZAGEM

- Criar instruções de design para o agente de código.
- Especificar a War Room.
- Compreender trace, ações e controles visuais.

GITHUB DA UNIDADE 8

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Criando instruções de design para o agente de código
- Especificando a War Room
- Trace, ações e controles visuais
- A War Room como nova porta de entrada
- Planejamento, tarefas e implementação
- Primeira versão da interface

Conteúdo da aula

Neste ponto, o OpsPilot já possui vários mecanismos de produção. Ele tem guardrails, limites para controlar gastos, memória, persistência, contexto com orçamento, observabilidade, roteamento automático entre estratégias e integração via MCP. Mesmo com tudo isso, ainda existe um problema bastante visível para quem precisa usar o sistema no dia a dia: a interface continua sendo praticamente o terminal.

Até agora, eu consigo conversar com o agente por chamadas HTTP e também consigo expor suas capacidades por MCP. Tecnicamente isso funciona, mas não é a experiência que eu quero entregar para um plantonista. A pessoa não deveria precisar montar requisições no terminal sempre que quiser abrir um incidente, consultar o plantão ou acompanhar o que o agente decidiu.

Por isso, nesta aula eu começo a construir uma War Room web. A ideia é dar uma interface para o OpsPilot sem criar uma nova lógica de negócio. O front-end continua chamando a API que já existe e passa a apresentar de forma mais acessível a conversa, o trace e as ações realizadas pelo agente.

Criando instruções de design para o agente de código

Antes de especificar a interface, eu quero preparar o agente de código para trabalhar com algumas regras de design.

Crio um arquivo de design instructions dentro da área de instructions do projeto e aplico essas orientações aos arquivos da camada web.

Eu mantenho essas instruções genéricas de propósito. Não quero definir uma identidade visual complexa nesta etapa. O objetivo é registrar princípios que devem ser respeitados quando o agente criar ou modificar a interface.

Entre esses princípios estão hierarquia visual, espaçamento, escala, tratamento de estados vazios e de erro, suporte a dark mode e acessibilidade.

Esses pontos ajudam a evitar que o agente produza apenas uma tela funcional sem considerar aspectos básicos de experiência. Assim como criamos instructions para arquitetura, testes e serviços, agora criamos uma instrução específica para o domínio visual.

Também deixo espaço para uma configuração equivalente no Cursor, caso a ferramenta precise de um escopo próprio para instruções. A ideia continua sendo a mesma: registrar orientação persistente para que novas sessões do agente mantenham o mesmo padrão.

Especificando a War Room

Com as design instructions criadas, parto para uma nova especificação no Spec Kit.

Quero uma aplicação web para funcionar como War Room do OpsPilot. Para essa camada, utilizo React com uma estrutura de front-end baseada em Vite.

A interface deve ter um chat conectado ao endpoint /chat que já existe no backend. O usuário escreve em linguagem natural e recebe a resposta do agente sem precisar montar manualmente uma chamada HTTP.

Também quero uma área lateral para apresentar o trace. Esse ponto é importante porque a observabilidade que construímos na aula anterior não deve ficar escondida apenas em uma rota técnica. A interface pode mostrar, de forma visual, os passos percorridos pelo agente.

Ações relevantes podem aparecer como cartões. A especificação também prevê espaço para operações de aprovação ou negação quando chegarmos aos fluxos que exigem intervenção humana.

Por fim, quero uma área de configuração com a URL base da API do OpsPilot. Isso permite apontar o front-end para o backend adequado sem acoplar toda a interface a um endereço fixo.

Trace, ações e controles visuais

A War Room não deve mostrar apenas a resposta final. Uma das vantagens de todo o trabalho de observabilidade é que agora eu tenho informações para apresentar ao usuário de forma mais compreensível.

O trace lateral pode mostrar etapas como ação, observação e resposta. Em vez de o plantonista enxergar somente uma frase final, ele consegue acompanhar os principais eventos que levaram até aquele resultado.

Quando existir uma ação que exige intervenção humana, a interface também pode apresentar cartões específicos com controles de aprovação e negação. A arquitetura de autonomia que construímos anteriormente já diferencia ações que o agente executa sozinho daquelas que precisam de confirmação. A War Room passa a ser um lugar natural para materializar essa interação.

A engrenagem de configuração também tem uma função prática. Ela permite alterar a URL base da API, algo útil quando o front-end é executado em um endereço e o backend está em outro ambiente.

Nada disso exige duplicar o raciocínio do agente no navegador. O front apenas transforma contratos e sinais que já existem no backend em uma experiência visual.

A War Room como nova porta de entrada

Eu continuo tratando a War Room como mais uma interface sobre o mesmo sistema.

Não quero duplicar abertura de incidentes, listagem de alertas ou memória dentro do front-end. A aplicação web chama a API e reutiliza exatamente o que já construímos.

Esse desenho é semelhante ao que fizemos com MCP. A regra de negócio permanece centralizada. O que muda é a porta pela qual o usuário acessa essas capacidades.

Isso significa que uma operação realizada na War Room precisa chegar à mesma SQLiteOpsStore, às mesmas strategies, à mesma memória e ao mesmo grafo de produção utilizados pelas chamadas feitas pelo terminal.

Planejamento, tarefas e implementação

Depois do specify, executo plan e tasks como nas features anteriores.

Desta vez, não considero necessário fornecer uma referência extensa antes do implement. A camada web é relativamente convencional e as design instructions já ajudam a orientar o agente.

Por isso, deixo o Spec Kit seguir até a implementação e depois parto para a validação prática da interface.

A aplicação web é criada em uma pasta separada e ganha seu próprio comando de desenvolvimento. Para testar o conjunto, mantenho o backend do OpsPilot em execução e inicio também o servidor do front-end.

Primeira versão da interface

Quando abro a War Room no navegador, a primeira versão é bastante simples.

Isso não é um problema para o objetivo da aula. Eu quero primeiro provar que a interface está conectada ao agente e que consigo executar as mesmas operações que antes estavam restritas ao terminal.

A tela já oferece a área de chat e uma visualização para informações relacionadas ao trace. A hierarquia ainda pode ser refinada, e eu observo que a responsividade também não está ideal.

Eu não tento resolver todos os detalhes visuais agora. A primeira entrega funciona como um MVP da War Room. O objetivo é validar integração, fluxo de conversa e visibilidade das ações antes de investir tempo em acabamento.

Também identifico algumas oportunidades de evolução, como adicionar um estado de loading durante as chamadas e renderizar respostas em Markdown para que títulos, listas e trechos destacados apareçam de forma mais próxima da experiência oferecida por assistentes como Copilot e Gemini.

Esses pontos ficam como melhorias do projeto. A interface funcional já é suficiente para continuar o exercício e testar as capacidades do agente em um formato mais próximo do uso real.

Abrindo um incidente pela War Room

Para validar a integração, faço um primeiro pedido pela própria interface.

Solicito a abertura de um incidente de severidade média para o serviço de checkout.

A War Room envia a mensagem para o endpoint de chat, o agente processa a solicitação e o incidente é criado.

Na resposta, consigo enxergar o resumo da operação e também parte do raciocínio operacional representado pelo trace.

A interface mostra elementos como ação, observação e resposta, aproveitando os sinais que já estavam sendo gerados pelo agente.

Isso confirma que o front-end não é uma simulação. Ele está realmente utilizando o OpsPilot que construímos nas unidades anteriores.

Consultando o resumo do plantão

Depois da criação do incidente, faço uma segunda pergunta pela interface: peço um resumo do plantão até aquele momento.

O agente consulta os dados existentes e retorna uma visão do estado atual, incluindo a quantidade de incidentes abertos por severidade.

Na execução da aula, aparecem incidentes críticos, altos, médios e baixos. O agente também apresenta os críticos primeiro em parte da resposta.

Essa consulta é importante porque começa a aproximar a War Room do tipo de uso que imaginamos desde o começo do projeto. O plantonista não precisa conhecer a estrutura da API. Ele faz uma pergunta operacional e recebe uma resposta consolidada.

Ao mesmo tempo, eu observo que a apresentação ainda pode ser melhorada. Markdown renderizado, loading e ajustes de layout deixariam a experiência mais clara, mas esses aprimoramentos ficam como extensões possíveis.

A memória precisa conhecer a preferência do usuário

Durante o teste do resumo, surge novamente um ponto relacionado à memória.

O agente consegue organizar informações, mas naquele fluxo a interface ainda não está fornecendo um contexto de usuário suficientemente claro para recuperar automaticamente todas as preferências que já construímos.

Por isso, eu informo diretamente uma preferência: quero que alertas e incidentes críticos sejam sempre listados primeiro.

Essa frase é exatamente o tipo de informação que deve virar memória semântica. Ela não representa apenas um pedido pontual para aquela resposta. Ela descreve uma preferência operacional que pode influenciar interações futuras.

Depois de informar essa preferência, faço uma nova solicitação pedindo para organizar o plantão e mostrar os incidentes em aberto.

A expectativa é que o agente use a memória para devolver a lista na ordem preferida, sem que eu precise repetir a instrução em todas as mensagens.

Por que não ordenar diretamente no backend

Existe uma alternativa mais determinística para esse caso. Eu poderia simplesmente alterar a consulta no backend e ordenar os incidentes por severidade antes de entregá-los ao agente.

Em uma aplicação de produção, dependendo da regra, essa pode inclusive ser a escolha que eu faria.

Nesta aula, porém, quero exercitar a memória do agente. Por isso, mantenho a ordenação como uma preferência aprendida e peço que o próprio agente utilize essa memória quando organizar o plantão.

Essa escolha é didática. Ela permite observar a diferença entre uma regra fixa do sistema e uma preferência específica do usuário.

Uma regra global como "incidentes críticos sempre vêm primeiro para todos" pode pertencer ao backend. Uma preferência como "este usuário prefere críticos primeiro" faz sentido dentro da camada de memória semântica.

Ajustando o comportamento da memória

Durante os testes, percebo que a organização ainda não fica totalmente coerente em todas as respostas.

Faço então um pequeno ajuste no comportamento da memória para alinhar a implementação com aquilo que definimos nas aulas anteriores.

O objetivo é garantir que a preferência aprendida seja recuperada quando o usuário pedir para organizar o plantão.

Como estamos usando agentes de código e modelos diferentes podem produzir implementações diferentes, os arquivos alterados nessa correção podem variar. O importante é preservar o fluxo conceitual: a preferência é armazenada, recuperada e injetada no contexto antes da resposta.

Depois do ajuste, reinicio a aplicação e faço novamente o pedido para organizar o plantão.

Validando a memória pela interface

Na nova execução, o comportamento fica alinhado ao esperado.

A War Room envia a solicitação, o OpsPilot consulta o estado dos incidentes e utiliza a preferência armazenada para ordenar a resposta.

Os itens críticos aparecem primeiro, seguidos pelas demais severidades.

Essa validação é interessante porque junta várias partes do projeto em uma única experiência. A interface web chama a API. O grafo decide o fluxo. As tools consultam o banco. A memória recupera a preferência. O contexto é montado com seus budgets. A resposta volta para o front-end.

O usuário final não precisa conhecer nenhuma dessas camadas. Ele apenas pede para organizar o plantão.

Preparando o deploy da interface

Depois que a War Room está funcional localmente, começo a preparar o próximo passo: publicar essa interface.

Crio uma nova especificação no Spec Kit para o deploy da pasta web utilizando GitHub Pages e GitHub Actions.

A automação precisa preparar o artefato do Pages, publicar esse artefato e possuir as permissões necessárias para a execução do workflow.

Também quero atualizar a documentação do projeto para explicar como essa publicação funciona.

Executo novamente `specify`, `plan`, `tasks` e `implement` para que o agente prepare os arquivos de workflow e os ajustes necessários.

Nesta aula, porém, eu paro antes de validar a publicação final no GitHub Pages. O objetivo aqui é deixar a War Room implementada e o deploy preparado para ser concluído na sequência.

Versionando a War Room

Com a parte local concluída, interrompo os servidores de desenvolvimento e verifico o estado do Git.

A nova interface, as instruções de design, os ajustes de memória e os arquivos relacionados à preparação do deploy aparecem como mudanças do projeto.

Faço o stage dessas alterações e preparo um commit referente à War Room.

Esse fechamento mantém o mesmo processo que seguimos ao longo do módulo: cada feature importante fica registrada em artefatos de especificação, código e histórico de versão.

A próxima etapa parte dessa base para concluir a configuração do GitHub Pages e validar a interface publicada fora do ambiente local.

NA PRÁTICA

Abra o snapshot da Unidade 8 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Criando instruções de design para o agente de código” no fluxo apresentado?
- Que decisão de engenharia está associada a “Especificando a War Room” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Criando instruções de design para o agente de código.
- Especificando a War Room.
- Trace, ações e controles visuais.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Publicando a War Room no GitHub Pages

Unidade 8 • Projeto Prático: OpsPilot Publicado

OBJETIVOS DE APRENDIZAGEM

- Ativar o GitHub Pages no repositório.
- Criar o workflow de deploy.
- Compreender permissões necessárias para o Pages.

GITHUB DA UNIDADE 8

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Ativando o GitHub Pages no repositório
- Criando o workflow de deploy
- Permissões necessárias para o Pages
- Configurando o base path do Vite
- Corrigindo falhas do build com o próprio agente
- Acessando a War Room publicada

Conteúdo da aula

Na aula anterior, eu deixei a War Room funcionando localmente e preparei a estrutura de deploy. Agora falta completar essa etapa e publicar a interface no GitHub Pages. A ideia é disponibilizar o front-end de forma simples, usando o próprio repositório como origem da publicação.

É importante separar bem as duas partes da aplicação. O GitHub Pages vai hospedar apenas os arquivos estáticos gerados pelo front-end. O backend do OpsPilot continua sendo uma aplicação Node, com banco, agentes, memória e integrações externas. Nesta aula, eu não vou publicar esse backend em um serviço de nuvem. Para conseguir testar a War Room publicada, vou apontá-la temporariamente para o backend local por meio de um túnel.

Essa separação é útil porque me permite validar o fluxo de publicação do front-end sem adicionar mais uma camada de infraestrutura ao projeto. Depois, como exercício, o backend pode ser hospedado em outro serviço e a War Room pode passar a consumir uma URL permanente.

Ativando o GitHub Pages no repositório

O primeiro passo é entrar no repositório do projeto no GitHub e habilitar o Pages.

Essa configuração é necessária porque o workflow que vamos criar precisa publicar um artefato no ambiente do GitHub Pages. Se a funcionalidade não estiver habilitada ou configurada corretamente, o deploy pode falhar mesmo que o build do front-end esteja correto.

Também é importante observar a área de Actions. Cada push na branch principal pode disparar o workflow de publicação. Se alguma etapa falhar, eu consigo abrir a execução e olhar exatamente em qual ponto aconteceu o problema.

Como estamos trabalhando com agente de código, essa mensagem de erro pode ser copiada e devolvida ao próprio agente para que ele ajuste o workflow. Isso mantém o mesmo padrão de trabalho que usamos durante todo o módulo: especificar, executar, observar a falha e corrigir com base em evidência.

Criando o workflow de deploy

Para publicar a War Room, quero um workflow dedicado dentro da estrutura de GitHub Actions do repositório.

Esse workflow deve ser disparado quando houver push na branch principal. A publicação passa, então, a fazer parte do fluxo normal de versionamento. Quando uma alteração da War Room chega à main, uma nova versão pode ser construída e enviada ao Pages.

No job de build, utilizo Node 22 para manter a mesma linha de runtime que já adotamos no projeto.

A instalação das dependências acontece dentro da pasta web, seguida pelo comando de build do front-end. Como essa aplicação foi criada com Vite, o resultado esperado é uma pasta de distribuição contendo os arquivos estáticos prontos para publicação.

Depois do build, o workflow envia essa pasta como artefato do GitHub Pages. Um segundo job é responsável pelo deploy efetivo do artefato para o ambiente do Pages.

A divisão entre build e deploy é útil porque deixa claro onde cada falha acontece. Se a aplicação não compilar, o problema aparece na etapa de build. Se o artefato estiver correto mas a publicação falhar, a investigação fica concentrada no job de deploy.

Permissões necessárias para o Pages

O workflow também precisa declarar as permissões adequadas.

A leitura do conteúdo do repositório continua sendo necessária para montar o build. Além disso, o job que publica precisa de permissão de escrita no Pages e da permissão usada para emitir a identidade necessária durante o deploy.

Essas permissões fazem parte do contrato do workflow. Se estiverem ausentes ou incorretas, a Action pode executar parte das etapas e falhar justamente na hora de publicar.

Por isso, eu prefiro especificar esse ponto desde o início em vez de depender de ajustes manuais depois.

Configurando o base path do Vite

Existe um detalhe importante quando publico uma aplicação Vite no GitHub Pages: o caminho base.

Em um projeto publicado dentro de um repositório, a aplicação normalmente não fica na raiz absoluta do domínio. Ela fica em um caminho correspondente ao nome do repositório.

Por isso, a configuração do Vite precisa conhecer esse prefixo. No nosso caso, o base path precisa refletir o caminho usado pelo repositório do OpsPilot.

Se essa configuração estiver errada, o HTML pode até abrir, mas os arquivos JavaScript, CSS ou outros assets podem ser procurados em URLs incorretas. O resultado é uma página quebrada ou vazia mesmo que o build tenha sido concluído com sucesso.

Esse é um exemplo clássico de diferença entre uma aplicação funcionando localmente e funcionando em hospedagem estática. Localmente, a raiz costuma ser simples. No Pages, preciso considerar o subcaminho de publicação.

Corrigindo falhas do build com o próprio agente

Durante a publicação, aparecem alguns problemas no build.

Eu não tento esconder esse tipo de situação porque ela é normal em um pipeline real. Uma Action pode falhar por configuração, dependência, caminho de diretório ou diferença entre o ambiente local e o runner do GitHub.

A abordagem que sigo é abrir a execução, copiar a mensagem de erro e enviar para o agente de código.

O agente ajusta os arquivos necessários e eu faço um novo commit e push. O push dispara outra execução do workflow.

Depois de algumas correções, a Action passa e a publicação é concluída.

Esse fluxo também reforça uma ideia importante do módulo: o agente de código não substitui observabilidade. Ele consegue ajudar muito mais quando eu entrego para ele uma falha concreta, com logs e contexto, em vez de pedir genericamente para "consertar o deploy".

Acessando a War Room publicada

Com o workflow concluído, o GitHub Pages disponibiliza uma URL pública para a War Room.

O endereço segue o padrão da conta do GitHub combinado com o caminho do repositório.

Ao abrir essa URL, eu encontro a mesma interface que vinha executando localmente. Agora, porém, o front-end está sendo servido pela infraestrutura estática do GitHub Pages.

Isso resolve apenas a publicação da interface. Ainda precisamos lembrar que o chat depende do backend do OpsPilot. Sem uma URL acessível para essa API, a War Room publicada não consegue conversar com o agente.

Por que o backend não vai para o GitHub Pages

O GitHub Pages é adequado para arquivos estáticos. Ele não executa nosso servidor Node, não mantém SQLite aberto, não executa LangGraph e não hospeda as estratégias de agente.

Por isso, eu não tento colocar o backend no Pages.

Uma alternativa para uma arquitetura mais completa seria hospedar a API em algum serviço próprio para aplicações backend e então configurar a War Room com essa URL.

Isso pode ser feito como exercício depois. O ponto desta aula é manter a publicação simples e usar um túnel para conectar o front-end público ao servidor que ainda está rodando na minha máquina.

Expondo o backend local com Cloudflare Tunnel

Para tornar o backend local acessível temporariamente, utilizo Cloudflare Tunnel através do utilitário cloudflared.

Primeiro, mantenho o servidor Node do OpsPilot funcionando normalmente na porta 3000.

Depois inicio um túnel apontando para esse localhost. O cloudflared cria uma URL pública temporária que encaminha as requisições recebidas para o servidor local.

Essa URL costuma ser gerada automaticamente e pode ter um nome aleatório. Isso é esperado. Como estamos usando esse túnel apenas para teste, não preciso de um domínio permanente.

A instalação do cloudflared depende do sistema operacional. O importante é seguir a documentação correspondente ao ambiente, instalar o utilitário e criar o túnel apontando para a porta em que a API está rodando.

Cada vez que eu iniciar um túnel temporário, a URL pode mudar. Por isso, a War Room precisa permitir alterar a URL base da API, algo que já deixamos previsto na interface.

Configurando a War Room para usar o túnel

Depois que o cloudflared gera a URL pública, copio esse endereço e volto para a War Room publicada.

Na área de configuração, substituo o endereço anterior da API pela URL do túnel.

A partir desse momento, o navegador acessa a War Room no GitHub Pages, mas as chamadas de chat são encaminhadas para o endereço público do Cloudflare Tunnel, que por sua vez redireciona a requisição para o backend local.

O caminho completo fica fácil de visualizar: navegador, GitHub Pages, chamada para a URL do túnel, Cloudflare Tunnel e servidor Node local.

Essa solução é temporária e adequada para demonstração. Ela não substitui uma hospedagem real do backend, mas permite testar a integração ponta a ponta sem precisar provisionar uma infraestrutura adicional nesta aula.

Escolhendo uma solicitação curta para validar o deploy

Para validar a integração, eu evito começar com uma solicitação muito longa.

O túnel, a rede, o modelo gratuito e os limites de timeout podem aumentar a latência. Se eu pedir uma tarefa que exige Plan-and-Execute e várias chamadas ao LLM, fico com mais variáveis para investigar caso algo demore.

Por isso, escolho uma operação simples: listar os incidentes.

Essa solicitação tende a ser interpretada como uma consulta pontual. O grafo pode selecionar ReAct, que normalmente resolve esse tipo de pedido com menos etapas do que Plan-and-Execute.

Essa escolha não é apenas para economizar tempo. É uma boa estratégia de validação. Quando introduzo uma nova camada de infraestrutura, começo com o menor cenário capaz de provar que a integração funciona.

Confirmando a requisição no backend

Quando envio a solicitação pela War Room publicada, observo o terminal onde o backend Node está rodando.

A aplicação recebe uma nova requisição e gera o requestId. Só esse evento já é um bom sinal de que o caminho entre o front-end público, o túnel e o servidor local está funcionando.

Depois o grafo começa a processar a mensagem. O roteador escolhe a estratégia adequada e o agente consulta as tools necessárias.

Como a pergunta é apenas uma listagem, o fluxo permanece relativamente simples. O resultado volta pelo mesmo caminho até o navegador.

A War Room exibe então os incidentes encontrados.

Com isso, eu valido não apenas que o site está publicado, mas que a interface publicada consegue conversar com o OpsPilot real.

O que foi efetivamente publicado

É importante entender exatamente o que está funcionando neste momento.

O front-end está hospedado no GitHub Pages e pode ser aberto por uma URL pública.

O backend continua local. Ele só está acessível porque o Cloudflare Tunnel cria uma ponte temporária.

O banco SQLite, a memória, o LangGraph, as strategies, os traces e todas as demais capacidades continuam executando na máquina em que o servidor Node foi iniciado.

Se eu encerrar o servidor ou o túnel, a War Room continua carregando no GitHub Pages, mas as operações que dependem da API deixam de funcionar.

Essa separação é importante para evitar a impressão de que o Pages hospedou a aplicação inteira. O que publicamos foi a camada web.

Separando publicação estática de execução do agente

Esse deploy também ajuda a visualizar uma separação arquitetural importante entre apresentação e execução.

O GitHub Actions constrói uma versão estática da interface. Depois do build, o conteúdo publicado é composto por arquivos que o navegador consegue baixar e executar. Não existe uma sessão do agente rodando dentro do Pages.

Quando o usuário envia uma mensagem, a interface precisa encontrar uma API ativa em outro lugar. É essa API que monta o contexto, consulta a memória, escolhe a estratégia, chama o modelo, executa tools e persiste os resultados.

Essa separação permite evoluir as duas camadas de forma independente. Posso redesenhar a War Room sem reimplementar o agente. Também posso trocar a infraestrutura do backend sem precisar reconstruir toda a interface, desde que o contrato da API continue compatível.

Na prática, a configuração da URL base é o ponto de ligação entre essas duas partes. Durante o desenvolvimento ela pode apontar para localhost. Na demonstração publicada, aponta para o Cloudflare Tunnel. Em uma implantação definitiva, apontaria para a URL estável do backend hospedado.

Esse desenho evita transformar a interface em dona das regras do OpsPilot. A War Room permanece uma cliente do agente, assim como o terminal e as integrações anteriores.

Próximos passos possíveis para a infraestrutura

A partir desse ponto, existem algumas evoluções naturais que podem ser feitas como exercício.

A primeira é publicar o backend em um serviço adequado para aplicações Node. Nesse caso, a War Room passa a apontar para uma URL estável e deixa de depender do túnel temporário.

A segunda é criar um pipeline de CI também para o backend, executando type check, testes e build antes de qualquer publicação.

Outra possibilidade é associar um domínio próprio e configurar variáveis por ambiente para separar desenvolvimento, homologação e produção.

Essas extensões não são necessárias para validar a feature desta aula. O objetivo imediato é comprovar que a War Room pode ser construída automaticamente, publicada pelo GitHub Actions e conectada ao agente através de uma URL externa.

Com o Pages funcionando e o túnel encaminhando para o backend local, consigo demonstrar a experiência completa fora do localhost do front-end e manter o foco no agente, que continua sendo o núcleo do projeto.

NA PRÁTICA

Abra o snapshot da Unidade 8 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Ativando o GitHub Pages no repositório” no fluxo apresentado?
- Que decisão de engenharia está associada a “Criando o workflow de deploy” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Ativando o GitHub Pages no repositório.
- Criando o workflow de deploy.
- Permissões necessárias para o Pages.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 8

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE

War Room →	API →	OpsPilot →	Memória + Store + Tools →	GitHub Pages →	Cloudflare Tunnel
------------	-------	------------	------------------------------	----------------	----------------------

CHECKLIST DA UNIDADE

- Revisar a Aula 1: Implementando a War Room e Modificando a Memória do Agente
- Revisar a Aula 2: Publicando a War Room no GitHub Pages

GITHUB DA UNIDADE 8

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Unidade 9

Multi-Agent Systems

Evolução para uma arquitetura multiagente com supervisor, papéis especializados, handoffs, blackboard e consenso.

FLUXO VISUAL DA UNIDADE					
Supervisor →	Analista →	Planejador →	Executor →	Tools →	Blackboard + Handoffs

GITHUB DA UNIDADE 9	
Abrir no GitHub	• Ler UNIDADE.md

Aula	Tema
1	Implementando Multiagentes

Aula 1 — Implementando Multiagentes

Unidade 9 • Multi-Agent Systems

OBJETIVOS DE APRENDIZAGEM

- Explicar por que dividir o trabalho entre agentes.
- Compreender o supervisor como coordenador do time.
- Compreender o analista como papel somente de leitura.

GITHUB DA UNIDADE 9

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

CONCEITOS-CHAVE

- Por que dividir o trabalho entre agentes
- O supervisor como coordenador do time
- O analista como papel somente de leitura
- O planejador transforma fatos em mitigação
- O executor coloca o plano em prática
- Handoff como passagem de trabalho

Conteúdo da aula

Neste ponto, o OpsPilot já está próximo da forma final que eu queria construir ao longo do módulo. Ele já passou por uma simulação de deploy da War Room no GitHub Pages, continua conectado ao backend por uma URL externa, controla orçamento de contexto, persiste dados, mantém memória, registra observabilidade, executa benchmark e possui diferentes estratégias de raciocínio. A próxima evolução não é adicionar apenas mais uma tool. Eu quero mudar a forma como o trabalho complexo é distribuído dentro do agente.

Para entender o problema, eu parto de uma solicitação de incidente que não é simples. Imagine pedir ao OpsPilot para investigar latência alta no checkout e erros 500 no payments, levantar o que está disparando, montar um plano de mitigação e abrir os incidentes necessários. Essa solicitação contém atividades diferentes dentro do mesmo pedido.

Primeiro existe uma etapa de análise factual. Depois existe uma etapa de planejamento. Por fim, existe uma etapa de execução. Quando um único agente, com um único papel e um único prompt, tenta assumir todas essas responsabilidades ao mesmo tempo, a tendência é produzir um resultado mediano em cada uma delas. É justamente esse tipo de cenário que motiva a entrada de um sistema multiagente.

Por que dividir o trabalho entre agentes

Agora eu quero montar um time de agentes com papéis realmente distintos. Cada agente recebe uma função específica, um conjunto de capacidades compatível com essa função e uma responsabilidade clara sobre aquilo que deve produzir.

Essa especialização melhora a qualidade porque reduz a quantidade de objetivos concorrentes dentro do mesmo prompt. O agente responsável por analisar não precisa pensar simultaneamente em como executar uma ação. O planejador não precisa consultar todas as ferramentas enquanto está tentando estruturar uma mitigação. O executor recebe um plano já preparado e pode se concentrar em aplicá-lo.

O supervisor como coordenador do time

No topo do time eu coloco um supervisor.

O supervisor é o coordenador. A regra mais importante é que ele não executa as ações operacionais. Ele observa o andamento do trabalho, lê o estado compartilhado e decide quem deve agir em seguida.

O supervisor precisa saber quais papéis existem, quais critérios determinam a passagem de trabalho e quando a tarefa já pode ser considerada concluída.

Por isso, sua saída também precisa ser estruturada. Quero que ele devolva qual é o próximo papel e um brief. Esse brief contém a instrução de trabalho que será entregue ao próximo agente ou, quando o fluxo terminar, o resumo final do trabalho.

O analista como papel somente de leitura

O primeiro trabalhador do time é o analista.

O analista tem uma responsabilidade deliberadamente restrita: levantar os fatos do plantão.

Ele pode consultar ferramentas de leitura para identificar alertas disparando, severidades, datas, incidentes recentes, incidentes abertos e resolvidos, runbooks relevantes, status de provedores e outros fatos disponíveis para o time.

Ao mesmo tempo, eu deixo claro que ele não deve propor soluções e não deve abrir ou resolver incidentes.

Também peço que o analista seja cético. Se determinada informação não aparece nas observações das tools, ele não deve afirmá-la como fato. O formato esperado é compacto, com tópicos telegráficos que possam ser reutilizados pelo restante do time.

O planejador transforma fatos em mitigação

O segundo trabalhador é o planejador.

Ele recebe os fatos levantados pelo analista e transforma esse material em um plano de mitigação.

Nesta etapa, eu quero que o planejador se concentre em organizar a resposta, definir sequência, dependências e próximos passos. Por isso, ele não precisa assumir as mesmas tools operacionais do executor.

O planejador trabalha sobre aquilo que já está disponível no quadro compartilhado. Em vez de refazer toda a investigação, ele usa o diagnóstico produzido anteriormente como entrada para construir um plano coerente.

O executor coloca o plano em prática

O terceiro trabalhador é o executor.

O executor recebe o plano e executa as ações necessárias, como abrir e conduzir incidentes.

É aqui que entram as operações com efeito sobre o estado do sistema. Como já construímos persistência, trace, matriz de autonomia e observabilidade, essas ações continuam passando pelos mesmos contratos e guardrails do OpsPilot.

Eu não estou criando um novo backend para o modo multiagente. O executor reutiliza as tools e a infraestrutura que já existiam.

Essa é uma ideia importante desta aula: multiagente não significa reescrever o projeto. Significa reorganizar capacidades existentes em papéis especializados.

Handoff como passagem de trabalho

Cada vez que o supervisor passa o trabalho de um agente para outro, acontece um handoff.

O handoff é a transição explícita de responsabilidade. O supervisor pode enviar o trabalho para o analista, depois para o planejador, depois para o executor e, quando considerar que o objetivo foi cumprido, encerrar o fluxo.

Eu quero que cada handoff seja registrado no trace.

O evento precisa indicar de onde o trabalho veio, para quem está indo e qual brief foi entregue.

Isso melhora bastante a auditoria. Em vez de enxergar apenas uma sequência longa de chamadas ao modelo, eu consigo reconstruir como o time se organizou e em que momento cada papel assumiu a responsabilidade.

Blackboard como estado compartilhado

Para que os agentes cooperem, utilizo o padrão de blackboard.

O blackboard funciona como um quadro compartilhado. Cada agente escreve aquilo que produziu e os agentes seguintes conseguem ler esse material.

No nosso caso, esse quadro pode ser representado pelo próprio estado do grafo. Isso significa que não preciso introduzir uma tecnologia completamente nova apenas para compartilhar informações entre os trabalhadores.

O analista deixa o diagnóstico. O planejador lê esse diagnóstico e acrescenta o plano. O executor lê o que foi produzido anteriormente e registra o resultado das ações.

Esse estado compartilhado evita que cada agente precise reconstruir o problema inteiro a partir da mensagem original. O time trabalha sobre uma memória comum da execução atual.

Consenso quando agentes discordam

Existe ainda um padrão útil para cenários em que dois agentes produzem pareceres diferentes.

Em vez de escolher arbitrariamente uma das respostas, posso pedir dois pareceres independentes e colocar um terceiro papel como juiz.

Esse juiz recebe os dois resultados, aplica critérios definidos e decide qual proposta deve prevalecer.

Reaproveitando tudo que já foi construído

Uma das razões pelas quais essa etapa fica mais simples é que grande parte da infraestrutura já existe.

O supervisor entra como mais um nó no grafo que construímos anteriormente. O blackboard reutiliza o estado compartilhado desse grafo. Os trabalhadores usam as tools que já estão prontas. A memória já existe. O trace já sabe registrar eventos. A observabilidade já persiste uma execução.

Até o roteador que criamos antes pode ser ampliado para ganhar uma nova rota de team.

Agent-to-Agent como possibilidade de evolução

Existem também protocolos específicos para comunicação entre agentes.

Um exemplo citado na aula é o Agent-to-Agent, da Google. Ele pode ser explorado como exercício para quem quiser implementar uma comunicação padronizada entre agentes usando um SDK próprio.

Especificando o modo equipe

Volto ao Spec Kit e crio uma nova especificação para o modo equipe.

Quero uma área própria para o team, com supervisor, blackboard e os três papéis especializados.

A saída do supervisor deve ser estruturada com next e brief.

O analista fica somente em leitura e não propõe ações. O planejador monta o plano sem assumir as tools de execução. O executor atua sobre os incidentes e demais operações previstas para colocar o plano em prática.

Também especifico um evento de handoff no trace e uma nova rota team no roteador de produção.

Como guardrail, defino um teto de oito transições no fluxo do time. Essa limitação segue a mesma filosofia aplicada ao ReAct, Plan-and-Execute, retry e Reflection. Um time também não pode ficar passando trabalho indefinidamente entre agentes.

Revisando plano, spec e tarefas

Depois de gerar specify, plan e tasks, reviso os artefatos produzidos.

O planejamento introduz explicitamente o modo em equipe e mantém as validações de fronteira que já utilizamos no projeto.

As tarefas preveem a criação do blackboard, do supervisor, dos papéis e do grafo específico do time.

Antes do implement, preparo uma referência para reduzir a ambiguidade em duas partes: o supervisor e o prompt do analista.

Saída estruturada do supervisor

Na referência do supervisor, defino um schema com os próximos destinos possíveis: analista, planejador, executor ou done.

O campo brief descreve a instrução que deve ser entregue ao próximo papel. Quando o supervisor escolhe done, esse mesmo campo pode funcionar como resumo final.

O supervisor chama o modelo com structured output, recebe a decisão e registra no trace um evento de handoff.

Depois atualiza o estado do time com o próximo papel e o brief.

A aresta condicional do grafo lê esse next e encaminha a execução para o nó correspondente.

O padrão é muito parecido com o roteador que já implementamos. A diferença é que agora a decisão acontece várias vezes durante a mesma execução, coordenando trabalhadores especializados.

Definindo a personalidade operacional do analista

Também forneço uma referência para o prompt do analista.

Eu o descrevo como o analista do plantão e deixo sua única função explícita: produzir um diagnóstico factual do estado atual.

Ele pode listar alertas disparando, datas e severidades, incidentes recentes, runbooks, status de provedores e fatos conhecidos pelo time.

A instrução também proíbe propostas de solução e qualquer abertura ou resolução de incidente.

Peço um formato telegráfico e reforço que dados não observados não devem ser afirmados.

Essa definição de personalidade não é decorativa. Ela funciona como um contrato de responsabilidade para impedir que o papel ultrapasse sua função.

Implementando o time no grafo de produção

Com as referências prontas, executo o implement do Spec Kit.

A implementação leva mais tempo porque o agente precisa alterar várias partes do projeto.

Depois da geração, reviso os arquivos principais. O roteador passa a conhecer a rota de team. O grafo de produção ganha o nó correspondente e conecta o modo equipe ao fluxo de resposta.

Os prompts dos papéis ficam separados, o supervisor segue a estrutura solicitada e a Team Strategy passa a coordenar analista, planejador e executor.

Colocando o time em ação

Com backend e War Room em execução, faço um teste mais próximo do cenário que motivou a feature.

Informo que existe latência alta no checkout e erro 500 no payments. Peço para conduzir a resposta ao incidente, levantar o que está disparando, montar o plano e abrir os incidentes.

Agora o roteador reconhece que esse pedido exige o modo equipe.

A execução demora mais do que uma consulta simples porque existem vários papéis e chamadas ao modelo. Na demonstração, o time realiza cerca de dez chamadas ao LLM.

Esse aumento de custo e latência é esperado. O que eu preciso avaliar é se a tarefa complexa passa a ser executada com uma qualidade e uma organização que um único agente não conseguiria oferecer da mesma forma.

Lendo o blackboard produzido pelo time

Ao final, consigo observar o conteúdo acumulado no blackboard.

O analista registra a investigação e organiza os alertas ativos por severidade.

Ele apresenta primeiro os críticos, depois os altos, médios e baixos, respeitando o tipo de organização que o OpsPilot já aprendeu a utilizar.

Também aparecem runbooks relevantes para checkout e payments, verificações relacionadas a taxa de erros e outras informações úteis para o diagnóstico.

O status de provedor também é consultado. Na execução apresentada, o GitHub aparece operacional.

Quando não existem fatos adicionais conhecidos pelo time, essa ausência também fica explícita em vez de o agente inventar informações para preencher o quadro.

Auditando a colaboração pelo trace

O trace agora mostra uma execução significativamente mais rica.

Consigo enxergar o routing inicial, os handoffs entre papéis, as actions disparadas e as observations devolvidas pelas tools.

O analista consulta o estado do plantão, verifica incidentes e recupera runbooks. O trabalho passa para os papéis seguintes com os briefs registrados.

Como cada passagem fica auditável, consigo investigar não apenas o que o OpsPilot respondeu, mas como o time dividiu a tarefa.

Esse comportamento se conecta diretamente à observabilidade construída anteriormente. Multiagente aumenta complexidade, então trace e auditoria se tornam ainda mais importantes.

Custos e qualidade no modo multiagente

Um sistema com vários agentes naturalmente pode consumir mais chamadas ao LLM do que uma execução simples.

Na prática, eu espero mais latência e maior consumo porque supervisor e trabalhadores precisam produzir decisões próprias.

Isso não significa que o modo equipe deva ser usado para qualquer pergunta.

Uma consulta pontual continua fazendo mais sentido com ReAct. Um pedido realmente complexo, que combina investigação, planejamento e execução, é que justifica pagar pelo trabalho adicional do time.

A decisão passa a ser semelhante às outras escolhas de arquitetura que fizemos: eu não otimizei apenas para o menor custo. Eu observo se o custo adicional compra uma capacidade ou uma qualidade que o fluxo anterior não conseguia entregar.

Aplicando os mesmos princípios em novos agentes

Eu já tenho um processo de especificação com Spec Kit, guardrails com allow e deny, limites para chamadas e contexto, memória, persistência, ferramentas, observabilidade, benchmark, interfaces e agora coordenação multiagente.

No modo multiagente, esse mesmo controle se estende à delegação. Cada papel precisa ter responsabilidade clara, acesso compatível com sua função e uma passagem de trabalho registrada. É isso que transforma um grupo de agentes em um sistema coordenado, em vez de apenas várias chamadas independentes ao modelo.

NA PRÁTICA

Abra o snapshot da Unidade 9 usando o link do GitHub exibido no início desta aula e localize os artefatos citados. Reproduza a evolução apresentada e, depois, altere uma decisão pequena e reversível para observar o impacto em testes, trace, estado ou comportamento do agente.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que dividir o trabalho entre agentes” no fluxo apresentado?
- Que decisão de engenharia está associada a “O supervisor como coordenador do time” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório ou nos testes para confirmar que o comportamento da aula está realmente funcionando?

RESUMO DA AULA

- Por que dividir o trabalho entre agentes.
- O supervisor como coordenador do time.
- O analista como papel somente de leitura.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 9

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.

FLUXO VISUAL DA UNIDADE

Supervisor →	Analista →	Planejador →	Executor →	Tools →	Blackboard + Handoffs
--------------	------------	--------------	------------	---------	-----------------------

CHECKLIST DA UNIDADE

- Revisar a Aula 1: Implementando Multiagentes

GITHUB DA UNIDADE 9

[Abrir no GitHub](#) • [Ler UNIDADE.md](#)

Revisão final da disciplina

Ao concluir a disciplina, o OpsPilot deixa de ser apenas uma demonstração de chamada a LLM e passa a reunir decisões de engenharia sobre contexto, estado, memória, ferramentas, protocolos, resiliência, observabilidade, interface e coordenação entre múltiplos papéis. Use o mapa abaixo para revisar o encadeamento completo.

FLUXO VISUAL DA UNIDADE					
Context Engineering →	Spec-Driven →	Reasoning →	Tools + MCP →	Memory →	Context Budget

FLUXO VISUAL DA UNIDADE					
LangGraph →	Observability →	Autonomy →	War Room →	Multi-Agent →	Auditability

REVISÃO DE DOMÍNIO

- Explique a diferença entre instrução e permissão e por que guardrails determinísticos continuam necessários.
- Compare ReAct, Plan-and-Execute e Reflection usando qualidade, chamadas ao modelo, latência e estado real da aplicação.
- Mostre como HTTP e MCP reutilizam a mesma aplicação, store, schemas e regras em vez de criarem backends independentes.
- Explique como memória episódica, memória semântica, sumarização e ContextBuilder disputam o orçamento de contexto.
- Descreva como LangGraph, fallback, observabilidade e a matriz de autonomia aumentam o controle sobre um sistema probabilístico.
- Explique como supervisor, analista, planejador e executor se coordenam por handoffs e blackboard em uma arquitetura multiagente.

REPOSITÓRIO OFICIAL DA DISCIPLINA

[Abrir no GitHub](#) — use este repositório como parte do estudo e acompanhe os snapshots por unidade.